



Software Testing di progetti Apache

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA
INGEGNERIA INFORMATICA

Autore	Alessandro Finocchi
Supervisore	Guglielmo De Angelis
Programma di studio	Software testing

1 Introduzione

In questo report sono descritte le varie attività di testing compiute nei progetti Apache di Bookkeeper ed Avro. Le classi scelte sono state `BufferedChannel` e `WriteCache` per bookkeeper, `Schema` e `SchemaCompatibility` per avro.

I test sono stati concepiti partendo da una analisi della documentazione, dalla quale sono nati una serie di ragionamenti che hanno instradato verso la generazione di test da **Category Partition**; una volta progettati, i test sono stati implementati con l'ausilio dei framework di **JUnit** per lo sviluppo degli unit test e **Mockito** per la creazione di mock. Grazie ai tool di **Jacoco** e **Badua** è stata analizzata la branch coverage e la all-uses coverage per evolvere la test suite originaria, e infine si è utilizzato **PIT** per effettuare mutation testing in modo da stimare la bontà della test suite e migliorarla ulteriormente.

I test sono stati inclusi nel ciclo di build del progetto tramite **Surefire** e **Failsafe**, e nel processo di Continuous Integration tramite i workflow delle **GitHub Actions**.

L'implementazione dei test è avvenuta in un branch a parte denominato "ale_branch" per ciascun progetto, si possono trovare nel link della sezione "Package di test" di ciascuna classe.

2 Testing Bookkeeper

2.1 BufferedChannel

2.1.1 Package di test

https://github.com/AlessandroFinocchi/bookkeeper/tree/ale_branch/bookkeeper-server/src/test/java/org/apache/bookkeeper/bookie

2.1.2 Introduzione

Dalla documentazione si legge che `BufferedChannel` è un wrapper di `FileChannel` al quale aggiunge funzionalità, in particolare permettendo la bufferizzazione dei dati in input ed output; sempre da essa si trova una descrizione minimale dei suoi attributi:

- **position**: la posizione assoluta della prossima operazione di write
- **unpersistedBytes**: dal nome si evince che indica il numero di byte non ancora persistenti, ovvero che dal buffer di scrittura e dalla cache del file channel non sono ancora stati resi persistenti
- **unpersistedBytesBound**: dal nome si evince che indica un estremo superiore per gli unpersistedBytes, oltre il quale quindi si vuole fare il flush dei byte scritti nel file channel
- **writeBuffer**: il buffer usato per le operazioni di scrittura
- **writeBufferStartPosition**: la posizione del puntatore di scrittura del file channel
- **writeCapacity**: la capacità del buffer di scrittura
- **readBuffer**: il buffer usato per le operazioni di lettura
- **readBufferStartPosition**: la posizione del puntatore di lettura del file channel
- **readCapacity**: la capacità del buffer di lettura
- **fileChannel**: il file channel di cui fare il wrapping

La strategia di testing adottata per questa classe sarà la seguente: si inizierà testando il metodo **costruttore** per esaminare la sua robustezza a parametri d'inizializzazione di vario tipo e per assicurarsi che non siano generate istanze invalide, e una volta fatto ciò si passerà ad eseguire il test dei due metodi principali per cui è nata questa classe, ovvero i metodi di **read** e **write**, a partire dalle istanze del costruttore che ce lo permettono. Ai fini dello studio aiuterà definire la variabile "`xContentLen`" come il numero di byte contenuti dentro l'oggetto x, dove x assumerà un nome diverso a seconda di cosa riferisce (es. per la lunghezza del contenuto del file channel si userà `fcContentLen`).

2.1.3 Generazione test da Category Partition

Costruttore

```
public BufferedChannel(ByteBufAllocator allocator, FileChannel fc, int writeCapacity, int readCapacity, long unpersistedBytesBound) throws IOException
```

Identificazione delle classi di equivalenza

- *allocator*: deve allocare i buffer $\Rightarrow$ {istanze valide}, {istanze non valide}, {null}
- *fc*: è il file channel, le categorie possono essere estratte conoscendo i vari stati e caratteristiche di un `FileChannel`, ci si concentrerà su:
 - Modalità: se è aperto o chiuso
 - Permessi: read-only, write-only o read-write
 - Validità: fc può avere una posizione non valida o può essere nullo

Ulteriori considerazioni sulla posizione di `fc` non servono poiché se si cercasse di scrivere oltre la fine del file questo si estenderebbe in automatico, e lo stesso vale per le considerazioni sulla dimensione¹. L'istanza valida sarà quella di un `FileChannel` aperto con il file già scritto (per un totale di `fcContentLen` byte), con permessi `read-write` e posizione valida, mentre tutte le altre partiranno da questa configurazione e la modificheranno $\Rightarrow$ {istanza valida}, {istanza chiusa}, {istanza read-only}, {istanza write-only}, {istanza con posizione non valida}, {null}²

- `writeCapacity`: $\{< 0\}, \{\geq 0\}$
- `readCapacity`: $\{< 0\}, \{\geq 0\}$
- `unpersistedBytesBound`: i byte non persistenti sono quelli scritti nella cache del file channel o nel buffer ancora non scaricati nel file, a primo impatto si potrebbe pensare che debbano essere meno della `writeCapacity`, ma come si diceva anche il file channel ha una cache, e quando si scrive nel file channel finché non avviene una *force* i byte non sono resi persistenti, quindi anche i byte scritti nel file channel sono da considerare non persistenti, e siccome il file channel può ampliarsi non c'è limite al numero di byte non persistenti che può avere $\Rightarrow \{< 0\}, \{\geq 0\}$

Boundary analysis

- `allocator`:
 - istanza valida: allocator può essere istanziato in diversi modi, ad esempio con meccanismi di pooling o senza, questo però è trasversale ai fini dell'attività di testing della nostra classe per cui ne sarà considerato solo uno, quello senza pooling.
 - istanza non valida: questo parametro deve solo allocare un buffer, per cui si usa un allocatore che invece non lo fa: a tal proposito si sfrutta Mockito per creare un mock di `ByteBufferAllocator` il quale istanzia buffer nulli come si può vedere in [Figura 4.1](#).
 - null.
- `fc`:
 - istanza valida: il file channel è aperto, ha permessi `read-write`, la sua posizione è valida e il file a cui è associato ha al suo interno scritti `fcContentLen` byte.
 - istanza chiusa: come la valida, ma il file channel è chiuso.
 - istanza read-only: come la valida, ma il file channel non ha permessi in scrittura.
 - istanza write-only: come la valida, ma il file channel non ha permessi in lettura.
 - istanza con posizione invalida: Mockito ci permette un mock parziale, tramite *spy*, dell'istanza valida che però restituisce una posizione negativa come si può vedere in [Figura 4.2](#).
 - null.
- `writeCapacity`: -1, 0, 1, 100³.
- `readCapacity`: -1, 0, 1, 100³.
- `unpersistedBytesBound`: -1, 0, 1, 50³

Write

```
public void write(ByteBuf src) throws IOException
```

Questo metodo scrive i dati di `src` prima nel buffer di scrittura e poi nel file channel, a seconda della dimensione del buffer e dell'upperbound al numero di byte non persistenti `unpersistedBytesBound`.

Identificazione delle classi di equivalenza

- `src`: dalla javadoc si trova che è la sorgente dei dati che vogliamo scrivere nel file channel. I `ByteBuf` possiedono tre indici che devono rispettare la relazione in [Figura 4.3](#). Considerando ciò, il partizionamento del suo dominio può essere fatto anche in questo caso in base a diverse caratteristiche:
 - Il contenuto: se il buffer è vuoto o meno
 - Validità: il buffer può essere nullo o deallocato, o ancora l'indice di lettura può essere non valido $\Rightarrow$ {istanza vuota}, {istanza non vuota}, {istanza con indice invalido}, {istanza deallocata}, {istanza nulla}

Boundary analysis

- `src`:
 - istanza vuota: nel buffer non c'è scritto nulla

¹ Si è pensato anche al caso in cui il file associato al `FileChannel` fosse stato cancellato dopo la creazione dell'istanza, ma questo test è particolare perchè su ambienti Linux-like non avrebbe ritornato alcun problema dal momento che l'applicazione avrebbe mantenuto l'handle aperto impedendo al file di essere effettivamente rimosso dal disco, mentre su Windows, dal momento che per default non si può cancellare un file se un processo lo ha aperto, ciò avrebbe creato problemi quindi si è omesso questo caso.

² La categorizzazione non è stata pensata semplicemente in istanze valide o meno come per allocator, questo perchè alcune istanze possono essere valide o meno a seconda del contesto, come quella `read-only`, valida se il buffer vuole essere usato in lettura, invalida in caso contrario. Serve quindi distinguere a grana più fine i diversi casi.

³ Ai valori di confine ci si aggiunge anche questo numero per testare il funzionamento in casi reali.

- istanza non vuota: il buffer contiene dei byte
- istanza con indice invalido: il buffer ha un indice di lettura non valido; per far ciò si è usato Mockito il quale permette di restituire sempre come indice di lettura un numero negativo (tramite *spy*)
- istanza deallocata: il buffer è stato deallocato
- null

Read

public synchronized int read(ByteBuf dest, long pos, int length) throws IOException

Legge i byte che sono stati scritti: questi però possono essere ancora non persistenti, in particolare siccome *writeBufferStartPosition* punta alla posizione da cui fare la prossima write nel file channel, se *pos* è minore di questo valore si leggerà il contenuto del file channel, se sarà maggiore invece si leggerà il contenuto del buffer, mentre se sarà minore ma $pos + length > writeBufferStartPosition$ quello che succede è che si vuole leggere oltre la dimensione del file channel, quindi si dovrà leggere prima dal file channel e poi dal buffer.

Identificazione delle classi di equivalenza

- *dest*: è la destinazione dove si scriveranno i dati letti dal file channel; il partizionamento del suo dominio può essere fatto ispirandosi alle considerazioni fatte per il metodo *write*:
 - Il contenuto: se il buffer ha byte scrivibili o se non li ha (come nel caso in cui è pieno).
 - Validità: il buffer può avere l'indice di scrittura non valido, può essere deallocato o nullo.
 ⇒ {istanza scrivibile}, {istanza non scrivibile}, {istanza con indice invalido}, {istanza deallocata}, {istanza nulla}
- *pos* e *length*: sono prese in esame insieme poiché è la coppia che stabilisce la lettura, quindi il partizionamento verrà fatto in base a dove si vuole leggere. Più nello specifico per *pos*, la lettura può partire dal file channel o dal buffer di scrittura, ma se si legge dal buffer di scrittura bisogna porre attenzione a non andare oltre i byte leggibili; per quanto riguarda *length*, se si parte a leggere al file channel, se *length* è sufficientemente lunga si può arrivare anche ai byte del buffer di scrittura.⇒
 - C0= {coppie invalide} = {(pos, length) : pos < 0 ∨ length < 0},
 - C1= {(pos, length) : lettura valida solo dal file channel},
 - C2= {(pos, length) : lettura valida solo dal buffer},
 - C3= {(pos, length) : lettura valida sia dal file channel che dal buffer}
 - C4= {(pos, length) : si arrivano a leggere i byte non leggibili del buffer}

Boundary analysis

- *dest*:
 - istanza scrivibile: nel buffer non c'è scritto nulla, quindi tutti i byte sono scrivibili
 - istanza non scrivibile: il buffer è pieno, non ha più spazio libero per byte da scrivere
 - istanza con indice invalido: l'indice di lettura del buffer è maggiore di quello di scrittura, a tal proposito si è usato Mockito per permettere all'indice di lettura di essere maggiore
 - istanza deallocata: il buffer è stato deallocato
 - null
- (*pos*, *length*): Il valore *writeBufferStartPosition* indica il confine per *pos*, prima del quale si legge dal file channel e oltre il quale dal buffer, mentre *writeBuffer.writerIndex* indica la posizione del buffer da cui iniziano i byte scrivibili, i quali non possono essere letti.
 - C0 ⇒ (-1, 1), (1, -1), (0, 0)
 - C1 ⇒ (0, 1), (0, writeBufferStartPosition-1), (0, writeBufferStartPosition), (1, writeBufferStartPosition-1)
 - C2 ⇒ (writeBufferStartPosition, 1), (writeBufferStartPosition+writeBuffer.writerIndex-1, 1)
 - C3 ⇒ (0, writeBufferStartPosition+1), (0, writeBufferStartPosition+writeBuffer.writerIndex)
 - C4 ⇒ (0, writeBufferStartPosition+writeBuffer.writerIndex+1), (writeBufferStartPosition+writeBuffer.writerIndex, 1)

2.1.4 Definizione e risultati test

Per il **costruttore**, come valore di ritorno ci si aspetta un'istanza valida o un'eccezione se indicata: se l'istanza è valida, tutti i suoi attributi vengono asseriti per verificarne la coerenza con i parametri di inizializzazione usati. I test istanziati sono visibili in [Tabella 4.1](#) e [Tabella 4.2](#). In particolare i casi T2, T7 e T17 sono falliti, per cui nei prossimi test, oltre a prendere un'istanza valida di costruttore, si considereranno contestualmente tutti questi casi per assicurarsi del loro corretto funzionamento.

Per il metodo **write** i test istanziati sono visibili in [Tabella 4.3](#) e [Tabella 4.4](#): oltre ai casi definiti durante la category partition e i test falliti del costruttore che potrebbero portare problemi nell'utilizzo di questo metodo, sono stati reputati importanti ulteriori casi, tra cui osservare il comportamento del metodo con un file channel

read-only, in cui la write non deve essere possibile, e casi particolari per la writeCapacity e unpersistedBytesBound, logicamente legati al comportamento che la write dovrebbe avere.

Per i casi erranei si è attesa un'eccezione, mentre per quelli ritenuti corretti si è asserito anche lo stato dell'istanza, osservando che i campi position, unpersistedBytes e writeBufferStartPosition fossero stati aggiornati coerentemente al caso di test generato, e si sono validati anche i byte effettivamente scritti e il dove fossero stati scritti, discriminando se fossero da leggere nel file channel o nel write buffer.

Anche in questo si sono riscontrate anomalie che hanno portato il fallimento di alcuni test:

- W2 e W3, relativi ai casi di test falliti nel costruttore T7 e T17, sono falliti anche durante la write poiché non hanno generato alcuna eccezione.
- W6 ha riscontrato un comportamento anomalo dal momento che, quando il write buffer ha una capacità pari a 0, il metodo write entra in un loop infinito.
- W7 ha riscontrato un comportamento anomalo nel cercare di effettuare una write quando l'upperbound al numero di byte non persistenti è 0, ma i byte non vengono scritti nel file channel.

Per il metodo **read** i test istanziati sono visibili in [Tabella 4.5](#) e [Tabella 4.6](#): per quanto riguarda lo stato dell'istanza in [Tabella 4.5](#), quella valida ha un file channel scritto con fcContentLen byte e i buffer sono correttamente allocati (presa dai test sul costruttore andati a buon fine), le altre modificano questa istanza. Anche qui come prima, oltre ai casi di test falliti nel costruttore, si è voluto sollecitare il metodo con ulteriori test ritenuti impattanti, nello specifico si è voluto studiare l'effetto del metodo in questione quando il file channel è write-only e in contesti di readCapacity non valida.

Per i casi ritenuti non validi si è attesa un'eccezione, per i rimanenti si è asserito che il contenuto scritto in dest fosse quello atteso. Nel testare la read si sono scoperte alcune anomalie dai casi di test falliti:

- R1, relativo al caso di test fallito nel costruttore T2, non ha generato alcuna eccezione nonostante l'allocatore allocasse buffer nulli: il motivo è che il buffer di lettura è allocato dalla super classe, e questo non è specificato dalla documentazione, e siccome non c'è riferimento al fatto che l'oggetto *allocator* istanzia solo il buffer di scrittura, la documentazione andrebbe aggiornata.
- R7, la cui configurazione porta il metodo ad eseguire un ciclo infinito.
- R8, la cui configurazione porta il metodo ad eseguire un ciclo infinito.
- R12, passare una lunghezza di byte da leggere negativa non genera eccezioni, quando invece è logico che dovrebbe siccome una lunghezza è concettualmente una quantità non negativa.

2.1.5 Analisi adeguatezza dei test

Dall'analisi di **Jacoco** ci concentriamo sulla **branch coverage**: del 100% per il costruttore, dell'80% per la write e del 88% per la read.

Per quanto riguarda la **write**, il primo problema si presenta a LOC 129, dove non si entra mai nell'if: questo succede perchè, come il commento sopra a questa riga aiuta a capire, effettivamente il buffer non finisce mai lo spazio a disposizione in nessun test.

Aggiungiamo allora il test J-W1, una variante del test fallito W7, dove però la dimensione del buffer di scrittura è la metà di quella dei byte che si vorranno scrivere: in questo caso J-W1 passa, quindi i byte vengono scritti nel file channel, e questo ci fa capire che se la variabile unpersistedBytesBound è pari a 0 ma il buffer di scrittura non è pieno comunque i byte non verranno scaricati nel file channel. A riprova di ciò infatti il test J-W2 fallisce perchè, allocando uno spazio nel buffer di scrittura pari alla metà dei byte che si vogliono scrivere più 1, quando verrà scritta la porzione [0, srcContentLen+1] nel buffer, questo poi verrà scaricato nel file channel, ma la seconda parte, quella rimanente, no perchè appunto il buffer di scrittura non sarà pieno, questo nonostante l'upperbound ai byte non persistenti sia zero. Il problema si vede facilmente dal report di jacoco, perchè la variabile *doRegularFlushes* = *unpersistedBytesBound* > 0, ma questo è incoerente con il significato di unpersistedBytesBound, il quale può aver senso che sia 0. Grazie al test J-W1 riusciamo con successo a portare la branch coverage della write al 100% come si vede in [Figura 4.4](#).

Spostandoci invece sulla **read**, notiamo che si è perso un branch a LOC 260, relativo al caso in cui il buffer di scrittura è nullo e si vuole leggere da una posizione interna ad esso, per cui aggiungiamo il test J-R1 in cui allochiamo un buffer di scrittura nullo e leggiamo al suo interno, come specificato in [Tabella 4.9](#) e [Tabella 4.10](#): prima di ora però non si era considerato nei casi di test che il buffer di scrittura potesse essere nullo, quindi si migliora la struttura del test per contemplare anche questa evenienza. Un altro branch mancato è però uno di quelli a LOC 264, in questo caso non è proprio ovvio capire quale sia la combinazione di parametri per l'if che non sono coperti, quindi si rimanda questo miglioramento a badua. Così la branch coverage per la read è salita per ora al 94% come si vede in [Figura 4.5](#).

Dall'analisi di **Badua** si ottiene il 100% di **all-uses coverage** per il costruttore testato.

In relazione alla **write**, si ottiene una all-uses coverage pari al 96%: le uniche due coppie def-use non coperte delle 48 totali sono

```
<du var="shouldForceWrite" def="119" use="142" target="143" covered="0"/>
```

```
<du var="shouldForceWrite" def="138" use="142" target="145" covered="0"/>
```

La mancanza è fittizia: infatti la prima è una definizione della variabile a false, ma il target è raggiungibile solo se la variabile in questione ha valore true, e il viceversa vale anche per la seconda coppia, che quindi possono essere considerate irraggiungibili e dunque il risultato finale è una all-uses coverage pari al 100% anche per il metodo write. Il risultato è visibile in [Codice 4.1](#)

Infine per la **read**, si ottiene una all-uses coverage pari al 74%: ci sono infatti ben 31 coppie def-use non coperte delle 118 come si può vedere in [Codice 4.2](#). Per quanto riguarda il branch che non si era coperto con jacoco a LOC 264, si può notare dalle coppie def-use in [Codice 4.3](#) che le variabili pos e readBufferStartPosition non definiscono mai l'entrata verso il blocco else successivo a LOC 272, e questo significa che readBufferStartPosition è sempre minore o uguale a pos: dalla super classe si scopre infatti che readBufferStartPosition viene inizializzato al minimo valore long, e leggendo la read durante la sua esecuzione l'unica modifica a readBufferStartPosition avviene quando gli viene assegnato il valore del parametro pos, e siccome il parametro pos non viene decrementato, con una read è impossibile che readBufferStartPosition sia strettamente maggiore di pos, ma se si effettuasse una lettura ad una posizione p1 e poi un'altra in posizione p2, con $p1 > p2$, alla seconda lettura readBufferStartPosition sarebbe uguale a p1, e quindi maggiore di p2, quindi riusciremmo a coprire il branch. Implementiamo questo nel test B-R1 (vedere [Tabella 4.11](#) e [Tabella 4.12](#), in cui si testa una read dopo un'altra read) e riusciamo a coprire le 3 coppie def-use sopra riportate e ad ottenere una branch coverage del 100% come si vede in [Figura 4.6](#).

Prendiamo in esame ora [Codice 4.4](#): queste coppie ci indicano che non si entra mai nell'if a LOC 264 a partire dall'inizio del metodo: per rimediare a ciò aggiungiamo il test B-R2 (vedere [Tabella 4.11](#) e [Tabella 4.12](#)) che usa sempre il test con le due read, questa volta però si legge due volte a partire dalla stessa posizione il buffer di lettura, in modo che la seconda volta pos sia uguale a readBufferStartPosition (prima condizione dell'if rispettata) ma minore di readBufferStartPosition+readBuffer.writerIndex (seconda condizione dell'if rispettata), e così raggiungiamo anche queste coppie.

A questo punto osserviamo che la read prova sempre prima a leggere dal file channel e poi nel write buffer, perciò possiamo "dividere" il metodo in due parti logiche: la parte A(LOC 248-263), che prova a leggere dal write buffer, e la parte B(LOC 264-281) che vuole invece leggere dal file channel; il flusso di esecuzione quindi non può andare da un punto in A ad un punto in B, ma può andare da un punto in B ad uno in A, quindi escludiamo tutte le coppie def-use con $def \in A$ e use e $target \in B$, evidenziate in [Codice 4.5](#).

Possiamo essere ancora più precisi: dividiamo la parte A in due parti ulteriori, la A1 (LOC 249-259), in cui si legge dal buffer di scrittura con condizione necessaria che questo non sia null, e A2 (LOC 260-263) dove invece si esce con condizione necessaria che il buffer sia null; abbiamo detto che quando si entra nella parte A non si può più andare in B, e siccome la read non fa diventare il buffer di scrittura null, quando si entra in A si esegue sempre dentro una sola delle sue sottoparti, o A1 o A2, per cui escludiamo tutte le coppie def-use che hanno def in A1 (risp. A2) e use e target in A2 (risp. A1) come in [Codice 4.6](#).

Se prendiamo invece la coppia in [Codice 4.7](#) ci accorgiamo che il while della read non itera mai più di una volta di seguito per leggere il buffer di lettura, e questo vale effettivamente anche per quello di scrittura, per cui andiamo ad aggiungere il caso di test B-R3 con singola read (vedere [Tabella 4.11](#) e [Tabella 4.12](#)) che copre queste due casistiche.

Per quanto riguarda la coppia in [Codice 4.8](#), essa non è raggiungibile perchè cambiata la definizione di pos a LOC 258 questa non potrà mai essere usata a LOC 260: infatti, pos fa parte della seconda clausola delle condizioni degli if a LOC 249 e 260, le quali sono entrambe delle forme normali congiuntive, ovvero separate da AND, in cui nel primo caso si controlla che il buffer non sia nullo e nel secondo caso che lo sia, ma se pos è stato definito a LOC 258 questo significa che il buffer di scrittura non è null, ovvero che la prima clausola della forma normale congiuntiva a LOC 260 è sempre falsa, e se la prima clausola di una forma normale congiuntiva è falsa non si controlla mai la seconda, quindi questa coppia è irraggiungibile.

Con questi miglioramenti si raggiunge una all-uses coverage del 95%, con solo 6 coppie delle 118, e possiamo ritenerci soddisfatti.

2.1.6 Mutation testing

Arrivati a questo punto, dopo i miglioramenti introdotti da jacoco e badua andiamo a migliorare la bontà dei nostri test usando pitest.

Si osserva per il **costruttore** che tutti i mutanti vengono correttamente riconosciuti e uccisi.

Per quanto riguarda il metodo **write**, solo un mutante sopravvive a LOC 124 come si può vedere da [Figura 4.8](#):

leggendo questo branch si capisce che i test implementati avevano un difetto, erano cioè concepiti per controllare se il contenuto di `src` fosse stato inserito dentro il buffer di scrittura oppure il file channel, non si era contemplata la possibilità che alcuni byte rimanessero scritti nel write buffer mentre gli altri fossero scaricati nel file channel, in realtà questo può accadere quando ciò che vogliamo scrivere non entra nel buffer, quindi se ne scrive una prima porzione e poi, se pieno, si scarica nel file channel, ma se l'ultima porzione non riempie il buffer e la sua lunghezza non supera l'upperbound ai byte non persistenti, rimane dentro il buffer di scrittura. Per questo motivo si è attuato un miglioramento dei casi di test:

- **Prima** di `ptest` si controllava se fosse stato scritto o il file channel o il write buffer.
- **Dopo** `ptest` si sono effettuati dei controlli più precisi per asserire correttamente il comportamento della `write`; i nuovi controlli sono stati realizzati a partire dal diagramma di flusso in [Figura 4.9](#), il quale è stato utile da scrivere per comprendere il corretto comportamento che questo metodo deve implementare.

Inoltre, per completezza dei casi di test e al fine di uccidere il mutante rimasto in vita, si aggiungono i test P-W1 e P-W2 come si può vedere in [Tabella 4.13](#) e [Tabella 4.14](#) in accordo con i ragionamenti appena fatti: si cercherà di scrivere `srcContentLen > 4` byte con un write buffer di dimensione `srcContentLen-2`, quindi i primi `srcContentLen-2` byte verranno scritti nel write buffer e subito dopo scaricati nel file channel siccome il buffer sarà riempito, gli ultimi 2 byte invece sono scritti nel buffer, e non vengono scaricati nel file channel perchè il buffer non si riempirà momento che

$\text{srcContentLen} > 4 \Rightarrow \text{writeCapacity} = \text{srcContentLen} - 2 > 2$

quindi per P-W1 ci si aspetta che gli ultimi 2 byte vengano scaricati nel file channel siccome il numero di byte non persistenti supera il limite, viceversa per P-W2 ci si aspetta che restino nel buffer: il risultato è che il primo test va a buon fine mentre il secondo no, dal momento che invece anche gli ultimi 2 byte sono scritti nel file channel, e questo sottolinea un altro comportamento anomalo, o comunque non documentato.

Ad ogni modo, con questi miglioramenti si sono uccisi tutti i mutanti come si può vedere in [Figura 4.10](#).

Infine, per il metodo **read**, solo 1 mutante a LOC 266 sopravvive come si può vedere in [Figura 4.11](#): questo accade perchè si sostituisce il meno col più, ma comunque il minimo valore resta sempre il numero di byte scrivibili in dest: per fare ciò aggiungiamo il test in [Tabella 4.15](#) e [Tabella 4.16](#) configurato per fare due read, in cui la seconda volta si rilegge dal read buffer, ma il mutante leggerà oltre la dimensione del buffer (la quale è stata diminuita ad un valore $\text{fCL}/2$) siccome il numero di byte da leggere diventa

$\text{readBuffer.writerIndex() + positionInBuffer} = \text{fCL}/2 + \text{fC}/2 - 1 = \text{fCL} - 1 > \text{fCL}/2 = \text{readCapacity(P-R1)}$

A questo punto tutti i mutanti sono correttamente riconosciuti anche per il metodo `read` come si può vedere in [Figura 4.12](#).

2.1.7 Stima dell'affidabilità

Ipotizzando una distribuzione uniforme di utilizzo, rispetto all'insieme dei casi di test si può assumere una reliability pari al 82% per il costruttore siccome solo 3 casi su 17 sono falliti, del 68% per la `write` siccome sono falliti 6 test su 19, e infine del 85% per la `read` dal momento che sono falliti 4 test su 28.

2.2 WriteCache

2.2.1 Package di test

https://github.com/AlessandroFinocchi/bookkeeper/tree/ale_branch/bookkeeper-server/src/test/java/org/apache/bookkeeper/bookie/storage/ldb (Vedere le classi di test "*WriteCache*Test*")

2.2.2 Introduzione

Dalla documentazione si sa che questa classe implementa una cache di scrittura, allocata dalla direct memory e suddivisa in più segmenti. Le entry sono aggiunte in un buffer comune e indicizzate tramite una hashmap. Tra i suoi attributi troviamo:

- `ConcurrentLongLongPairHashMap index`: è l'indice delle entry scritte nei segmenti, implementato come un'hash map che mappa la chiave (`ledgerID`, `entryID`) nel valore (`offset`, `length`).
- `ConcurrentLongLongHashMap lastEntryMap`: è l'indice delle ultime entry scritte nei segmenti da parte di ciascun ledger, implementato come un'hashmap che mappa la chiave (`ledgerID`) nel valore (`entryID`).
- `ByteBufferAllocator allocator`: alloca i segmenti della cache.
- `ByteBuffer[] cacheSegments`: la cache vera e propria, suddivisa in segmenti, ognuno elemento del vettore.
- `ByteBufferAllocator allocator`: alloca dei `ByteBuffer`.
- `int segmentsCount`: numero di segmenti nella cache.
- `long maxCacheSize`: taglia massima della cache.
- `int maxSegmentSize`: taglia massima di un segmento (deve essere una potenza di 2).

- AtomicLong **cacheSize**: dimensione di tutte le entry.
- AtomicLong **cacheOffset**: offset della cache (deve essere un multiplo di 64).
- AtomicLong **cacheCount**: numero di entry attualmente nella cache.

In questo caso vogliamo testare solo l'effetto che la put ha sullo stato della classe: come per BufferedChannel si testerà il costruttore e da esso si ricaveranno le istanze valide per testare il metodo put.

2.2.3 Generazione test da Category Partition

Costruttore

```
public WriteCache(ByteBufAllocator allocator, long maxCacheSize, int maxSegmentSize)
```

Identificazione delle classi di equivalenza

- *allocator*: deve allocare i buffer $\Rightarrow$ {istanze valide}, {istanze non valide}, {null}
- *maxCacheSize*: indica la dimensione massima con cui sarà istanziata la cache $\Rightarrow$ $\{< 0\}$, $\{\geq 0\}$
- *maxSegmentSize*: indica la dimensione massima di ogni segmento della cache, quindi non potrà essere superiore alla dimensione della cache, teniamo presente anche questo fattore. Inoltre questo valore deve essere allineato ad una potenza di due.
 $\Rightarrow$ $\{< 0\}$, $\{\geq 0\}$, $\{\leq \text{maxCacheSize}\}$, $\{> \text{maxCacheSize}\}$, {potenza di due}, {non potenza di due}

Boundary analysis

- *allocator*:
 - istanza valida: prendiamo l'allocatore senza pooling.
 - istanza non valida: si riprende l'allocatore che alloca un buffer null (Figura 4.1).
 - null.
- *maxCacheSize*: -1, 0, 1, 512⁴
- *maxSegmentSize*: con maxCacheSize a 512 prendiamo -1, 0, 1, 100, 128, 512, 600

Put

```
public boolean put(long ledgerId, long entryId, ByteBuf entry)
```

Identificazione delle classi di equivalenza

- *ledgerId*: indica l'id del ledger in cui si vuole scrivere; in quanto chiave delle hash map della classe, dalla javadoc si sa che le chiavi di queste hashmap devono essere maggiori o uguali a zero, per cui ci aspettiamo problemi in caso di chiavi strettamente negative $\Rightarrow$ $\{< 0\}$, $\{\geq 0\}$
- *entryId*: indica l'id dell'entry che si vuole scrivere; in quanto chiave dell'hash map index della classe, come prima ci aspettiamo problemi in caso di chiavi strettamente negative $\Rightarrow$ $\{< 0\}$, $\{\geq 0\}$
- *entry*: contiene i byte che si vogliono scrivere in cache; partizioniamo il dominio come nel caso della write in BufferedChannel, ma in più consideriamo anche le istanze che entrano in un segmento di cache e quelle che non entrano
 $\Rightarrow$ {istanza vuota}, {istanza con size $\in (0, \text{maxSegmentSize}]$ }, {istanza con size $> \text{maxSegmentSize}$ }, {istanza con indice invalido}, {istanza deallocata}, {istanza nulla}

Boundary analysis

- *ledgerId*: -1, 0, 1
- *entryId*: -1, 0, 1
- *entry*
 - istanza vuota: nel buffer non c'è scritto nulla
 - istanza non vuota con size minore o uguale di maxSegmentSize
 - istanza non vuota con size strettamente maggiore di maxSegmentSize
 - istanza con indice invalido: il buffer ha un indice di lettura non valido; per far ciò si è usato Mockito il quale permette di restituire sempre come indice di lettura un numero negativo (tramite spy)
 - istanza deallocata: il buffer è stato deallocato
 - null

2.2.4 Definizione e risultati test

Per il **costruttore** ci aspettiamo sia ritornata un'istanza valida o un'eccezione a seconda della configurazione dei parametri con cui è inizializzato: i test implementati in questa fase sono visualizzabili in Tabella 4.17. Se ci si aspetta un'istanza valida, si effettua un controllo del suo stato, ponendo attenzione ai campi size e count (da inizializzare logicamente a 0), maxCacheSize, maxSegmentSize, (che devono essere uguali a quelli passati), segmentsCount che deve essere uguale ad $1 + \text{maxCacheSize}/\text{maxSegmentSize}$, inoltre si

⁴Ai valori di confine ci si aggiunge anche questo numero per testare il funzionamento in casi reali.

controlla che la dimensione di tutti i segmenti sia pari a quella comunicata in `maxSegmentSize`, tranne per l'ultimo segmento, che invece sarà pari al numero di byte disponibili rimanenti. I test falliti sono stati 3: T1 e T2, in cui si usa rispettivamente un allocatore che alloca buffer nulli e uno invece null (il fallimento di questi test non ha importanza per i test della put dal momento che i buffer che alloca l'allocator si scopre sono quelli che restituisce il metodo `get`, ma nella documentazione non è evidenziato questo fatto e andrebbe aggiornata perchè potrebbe portar a pensare che sia l'allocatore dei buffer della cache) e T4, in cui la dimensione massima della cache è zero, e questo non ha senso perchè non potrebbe contenere alcun segmento su cui scrivere. Per tali ragioni si osserverà solo l'effetto che può avere T4 nel contesto di una put.

Per il metodo **put**, oltre a considerare i test generati dalla category partition, esaminiamo anche il caso d'uso dell'istanza di costruttore fallita (T4): i test implementati in questa fase sono visualizzabili in [Tabella 4.18](#). I risultati possibili sono 3: se ci si aspetta un'eccezione si controlla che la put la generi, altrimenti si controlla se il risultato della sua chiamata è true o false, e ci si assicura che lo stato della classe sia correttamente aggiornato andando a verificare la coerenza dei campi `size`, `count` e delle hash map, e in caso si aspetti un inserimento valido, anche della stringa effettivamente scritta in cache. Nel caso di P1 ci aspettavamo un'eccezione nel costruttore, che però come si diceva non è stata lanciata, perciò dato che si assume che le cache di dimensione 0 siano ammissibili, ci aspettiamo che il metodo ritorni false.

2.2.5 Analisi adeguatezza dei test

Dall'analisi di **Jacoco** ci concentriamo sulla **branch coverage**: del 100% per il costruttore e del 75% per il metodo `put`.

Per il metodo **put** infatti come si può vedere in [Figura 4.14](#), l'approccio black box ci aveva impedito di osservare che per ogni ledger, se si vuole inserire un'entry con id minore dell'ultima inserita, tale inserimento non aggiorna l'hash map `lastEntryMap`, che quindi non raccoglie tutte le ultime entry inserite per ogni ledger, ma raccoglie tutte quelle con id maggiore. Quindi per coprire il branch mancato nell'if a LOC 171 serve avere un'istanza in cui è già stata inserita una entry, ma i test in [Tabella 4.18](#) non prevedono questa possibilità: vengono allora migliorati per prevedere anche la possibilità di fare una put precedente a quella che si vuole testare, e in più aggiungendo i test in [Tabella 4.19](#). Per quanto riguarda invece il branch a LOC 176, la chiamata a `compareAndSet` restituisce falso soltanto quando il valore corrente della chiave comunicato (`currentlastEntryId`) non coincide con quello realmente presente nell'hashmap: il fine ultimo è gestire aggiornamenti asincroni, ma come anche il commento ci fa notare nella pratica ciò non dovrebbe accadere, per cui decidiamo di non coprire anche quel branch e rimanere con una branch coverage del 87% come si può vedere in [Figura 4.15](#).

Riguardo **Badua**, le uniche coppie `def use` non coperte sono proprio quelle che riguardano il branch non coperto della `compareAndSet` di cui discusso poco sopra, per cui ci attestiamo su una **all-uses coverage** del 91% come si può vedere in [Codice 4.9](#).

2.2.6 Mutation testing

Per il **costruttore** pitest riporta diversi mutanti non uccisi come si può osservare in [Figura 4.16](#). Per quanto riguarda i 2 mutanti sopravvissuti a LOC 95, il primo che cambia la condizione da maggiore a minore o uguale e il secondo che rimuove la chiamata, questi sono mutanti equivalenti: infatti passando una `maxSegmentSize` positiva il metodo si comporta bene in tutti i casi, se gli si passa una `maxSegmentSize` pari a 0 si ottiene un'eccezione nel metodo SUT per la chiamata a `checkArgument` e nei mutanti poiché a LOC 106 `maxSegmentSize` divide `maxCacheSize`, infine se si usasse una `maxSegmentSize` minore strettamente di zero si otterrebbe un'eccezione nel metodo SUT sempre per via della chiamata a `checkArgument`, ma anche nei mutanti perchè a LOC 112 `Unpooled.directBuffer` restituirebbe un'eccezione.

Per i mutanti a LOC 103 e 104 invece si migliora i test aggiungendo anche i controlli delle variabili inizializzate in quelle righe, le quali da un'analisi black box erano sfuggite poiché campi privati⁵, e infine anche il mutante a LOC 110 che cambia la condition boundary da $< a \leq$ è un mutante equivalente dal momento che, anche se l'allocazione del `cacheSegments[segmentsCount-1]` si facesse dentro il `for`, la si rifarebbe comunque appena uscito, assegnandogli il valore corretto che gli assegnerebbe anche il metodo SUT.

Infine per il metodo **put** abbiamo 3 mutanti non uccisi nelle LOC 151 e 154 come ben si vede in [Figura 4.18](#). I primi due mutanti a LOC 151 sostituiscono rispettivamente il $+$ con il $-$ (M_1) e il $>$ con il $\geq$ (M_2): per uccidere M_1 si è aggiunto il test P-P1 (vedere [Tabella 4.20](#)), con il quale inseriamo una entry lunga 1024 byte in una cache vuota grande 512 byte ma con `maxSegmentSize` pari a 1024, per cui avrà un unico segmento, e mentre il test del metodo SUT passerà poiché il risultato atteso è false, in M_1 , dal momento che `offset` e `localOffset` saranno pari a zero, `offset - size = 0 - 1024 = -1024` che non sarà maggiore di `maxCacheSize`

⁵Vengono infatti acceduti grazie ad un getter annotato come `VisibleForTesting`

= 512, quindi arriverà al secondo if a LOC 154, per il quale $\text{maxSegmentSize} - \text{localOffset} = 1024$ non sarà minore di size , anch'essa pari a 1024, perciò si uscirà dall'if e quando si cercherà di scrivere i byte a LOC 164, dato che non ci sarà abbastanza spazio per l'entry nel segmento, si otterrà un'eccezione; per uccidere M_2 invece si implementa il test P-P2 (vedere Tabella 4.20), nel quale si inserisce una entry di 128 byte in una cache con maxCacheSize a 256 byte e maxSegmentSize a 128 byte, e con il primo segmento dei due della cache già pienamente scritto, quello che succederà è che mentre il test del metodo SUT passerà poichè il risultato atteso è true, in M_2 , dato che offset sarà pari a 128 e size anche, allora si avrà che $\text{offset} + \text{size} = 256 = \text{maxCacheSize}$, che non è maggiore di se stessa e quindi il metodo restituirà false.

Per quanto riguarda l'ultimo mutante a LOC 154, il quale sostituisce il $-$ con il $+$ si è prima implementato il test P-P3 (vedere Tabella 4.20) nel quale prendiamo una cache con $\text{maxCacheSize}=256$ e $\text{maxSegmentSize}=128$, e con il primo segmento scritto per metà (64 byte), quindi il secondo segmento è ancora interamente libero, dopo di ciò si effettua il test della put per una entry grande 128 byte, e siccome c'è ancora un segmento libero in cache ci si aspetta come risultato atteso true, il motivo per cui questo test deve riconoscere il mutante è che quando ci si trova nel primo ciclo del while a LOC 146, quello in cui si vede se c'è spazio nel primo segmento, quando si arriva a LOC 154, mentre il metodo SUT riconosce che non c'è abbastanza spazio in quel segmento e passa al prossimo, il metodo mutante riconosce lo spazio libero di quel segmento come sufficientemente grande da contenere la entry dal momento che $\text{maxSegmentSize} + \text{localOffset} = 128 + 64 = 192 > 128 = \text{size}$, quindi alla fine proverà ad inserire entry ad offset 64 del primo segmento, che però al momento della scrittura, essendo lungo solo 128 byte, restituirà un'eccezione; in realtà si scopre che con questa configurazione il risultato ritornato è false, come se non ci fosse abbastanza spazio nella cache, quindi per coprire il mutante si implementa il test P-P4, versione di P-P3 che ritorna false, con il quale lo si uccide con successo, ma siccome logicamente questo inserimento sarebbe dovuto andare a buon fine (si sarebbe potuto fare nel secondo segmento libero) e dalla javadoc non c'è alcun riferimento in merito, questo è un bug. Con questi miglioramenti tutti i mutanti riescono ad essere uccisi con successo, come si può vedere in Figura 4.19.

2.2.7 Stima dell'affidabilità

Supponendo una distribuzione uniforme di utilizzo, rispetto all'insieme dei casi di test si può assumere una reliability pari al 75% per il costruttore siccome sono falliti 3 test su 12, e del 94% per il metodo put con un solo test fallito su 17.

2.3 Integration testing: SingleDirectoryDbLedgerStorage e WriteCache

2.3.1 Package di test

https://github.com/AlessandroFinocchi/bookkeeper/tree/ale_branch/bookkeeper-server/src/test/java/org/apache/bookkeeper/bookie/storage/ldb (Vedere la classe di test "SingleDirectoryDbLedgerStorageWriteCacheIT")

2.3.2 Introduzione

Dopo aver analizzato la classe WriteCache, e in particolare il suo metodo *put*, possiamo andare a realizzare dei test di integrazione con la classe che fa uso di questa cache, ovvero sia **SingleDirectoryDbLedger**: in particolare, il metodo principale di questa classe con una dipendenza funzionale a WriteCache è il metodo *addEntry*, il quale aggiunge una entry al Ledger, ma prima di fare il flush in memory delle entry scritte, per motivi di performance vengono prima bufferizzate nella writeCache, dopo di che il flushing, oltre che periodicamente, viene fatto solo quando la cache si riempie, in modo da poterla svuotare.

È importante sottolineare come venga utilizzata la cache: ci si riferisce con l'espressione "ruotare la cache" all'atto di scambiare la cache attualmente in uso con un'altra per gestire in modo efficiente le scritture che stanno avvenendo e il flushing dei dati; nel ledger infatti esistono due istanze di WriteCache, *writeCache*, usata per scrivere nuovi dati, e *writeCacheBeingFlushed*, la cache precedente che sta venendo scaricata; quando *writeCache* è piena e deve essere svuotata su disco, invece di bloccare le nuove scritture e rallentare il sistema, si scambiano le due cache così le scritture possono continuare e nel frattempo si effettua il flushing della cache precedente che si era riempita (la rotazione avviene tramite il metodo *swapWriteCache*).

2.3.3 Metodologia

L'approccio di integrazione dei componenti è quello gerarchico Top-down.

Il metodo verrà analizzato per successivi raffinamenti: volendo osservare la connessione tra il ledger e la cache, dal momento che il primo fa uso di un metodo del secondo, c'è una dipendenza funzionale dal ledger verso la cache. Un semplice ORD che rappresenta questo fatto si può vedere in Figura 4.20.

Per seguire l'approccio Top-down, si andrà a testare prima il ledger con uno stub della cache per isolare il

componente dalla sua dipendenza, dopo di che si evolverà lo stub in un mock (tramite *spy* di Mockito) per verificare che l'interazione tra il componente e la sua dipendenza rispetti il comportamento atteso.

2.3.4 Implementazione e risultati

L'implementazione vede l'utilizzo di alcune classi di configurazione facenti parte dei test precedentemente presenti in modo da poter inizializzare l'istanza di ledger soggetta dei test.

All'interno del metodo `addEntry` del ledger, prima di effettuare una put, si ottiene un timestamp con una lettura ottimistica di un oggetto `StampedLock`, e dopo la put si valida il timestamp, il quale sarà valido se nel frattempo la cache non sarà ruotata, mentre se è invalido significa che la cache è ruotata e quindi la scrittura va rifatta, da cui la seconda chiamata a put; infine, se l'inserimento non è andato a buon fine, significa che la cache è piena e quindi tramite il metodo `triggerFlushAndAddEntry` si svuota la cache e si rieffettua la put.

Per controllare il numero di volte in cui la put è stata chiamata, si è usato il metodo `verify` di Mockito, mentre per la creazione degli stub si è usato il metodo `doReturn` sempre di Mockito (es in [Figura 4.21](#)) Per il testing con una `WriteCache` stub si configurano gli stub per assicurarci che queste casistiche siano coperte:

- La prima put ritorna true, quindi viene chiamata solo una volta
- La prima put ritorna false, e quindi viene simulato il non inserimento dovuto al fatto che la cache è piena, ovvero si faranno due put come prima spiegato.

Andiamo a simulare anche il caso di rotazione della cache, stavolta usando però un'istanza della cache reale, quindi prima la si riempie e poi si controlla che l'ultimo inserimento avvenga più volte: riguardo quest'ultimo dettaglio, l'inserimento avverrà sicuramente almeno 2 volte, ma potrebbe avvenire 3 volte a seconda del risultato della validazione del timestamp.

La test suite è visibile in [Tabella 4.21](#): tutti i test vanno a buon fine.

3 Testing Avro

3.1 Schema

3.1.1 Package di test

https://github.com/AlessandroFinocchi/avro/tree/ale_branch/lang/java/avro/src/test/java/org/apache/avro (Vedere le classi di test "*Schema*Test*")

3.1.2 Introduzione

Schema è il primo mattone nella serializzazione dei dati in Avro: quando i dati di Avro sono letti, lo schema usato al momento della scrittura è sempre presente insieme ad essi per renderli completamente auto-descrittivi, e di conseguenza al momento della scrittura lo schema è salvato insieme ai dati. Gli schemi in Avro sono definiti in formato JSON e possono rappresentare uno dei seguenti tipi:

- **record**, un insieme di campi ognuno con un nome e un tipo
- **enum**, contenente uno dei simboli di un piccolo insieme
- **array** di valori, tutti dello stesso schema
- **map**, contenente coppie stringa/valore, di uno schema dichiarato
- **union** di altri schemi
- **oggetto** binario di dimensione fissa
- **stringa** in UNICODE
- **sequenza di byte**
- **signed int** a 32 bit
- **signed long** a 64 bit
- **single-float** a 32 bit
- **double-float** a 64 bit
- **booleano**
- **null**

Il metodo `parse` di `Schema` è un metodo che converte un json in un oggetto `Schema`: essendo un metodo statico, che non deve avere effetti collaterali sullo stato della classe, testarlo in isolamento è utile e sensato quindi ci si concentrerà solo su questo metodo.

3.1.3 Generazione test da Category Partition

Parse

```
static Schema parse(JsonNode schema, Names names)
```

Identificazione delle classi di equivalenza

- *schema*: è un'astrazione di un nodo JSON rappresentato con una struttura ad albero, cioè contenente altri nodi JSON. Per ogni tipo che uno schema può avere il nodo JSON deve rispettare una specifica struttura come descritta dalla documentazione; per cui faremo i test usando delle istanze che rispettano il formato voluto (valide), istanze che non lo rispettano (non valide) e istanze nulle
⇒ {istanza valida}, {istanza non valida}, {istanza nulla}
- *names*: è una mappa che associa ad ogni schema un nome e un namespace in modo da tenere traccia degli schemi definiti e poterli recuperare con facilità in base ai nomi usati
⇒ {istanza valida}, {istanza non valida}, {istanza nulla}

Boundary analysis

- *schema*:
 - istanza valida: si prenderà un'istanza per ogni tipo di dato supportato per assicurarsi che il suo parsing funzioni
 - istanza non valida: si prendono istanze che non rispettano il formato specificato dalla documentazione, in particolare istanze senza un campo obbligatorio o istanze con un campo obbligatorio dal valore non valido.
 - null
- *names*:
 - istanza valida: si prende un namespace che rispetti le convenzioni, ovvero con stringhe separate da punti e senza caratteri speciali
 - istanza non valida: l'istanza lancia eccezioni quando i suoi metodi sono invocati. Non è stato possibile usare Mockito per via dei problemi che si hanno nel fare mocking di una classe che estende una collezione standard di Java, per cui si è creata invece una classe che estende Names e fa override dei suoi metodi come si può vedere in [Figura 4.22](#)
 - null

3.1.4 Definizione e risultati test

I test per il metodo **parse** sono stati descritti in [Tabella 4.22](#): ogni test previsto in figura è stato adottato per ciascun tipo di dato supportato dagli schemi di avro come da specifiche. I risultati attesi erano o un'eccezione in caso di configurazione di parametri sbagliata, oppure un'istanza valida per configurazioni di parametri validi. In particolare, il test per una union nel file *SchemaWOFIELDTest*, in cui si generava il nodo json di una union vuota, è risultato fallito: infatti si attendeva un'eccezione dal momento che una union non può essere vuota siccome dalla documentazione si legge che *"Note that when a default value is specified for a record field whose type is a union, the type of the default value must match with one element of the union"*, ma non ne viene generata nessuna.

3.1.5 Analisi adeguatezza dei test

Il metodo **parse** non fa altro che richiamare altri due metodi statici, ovvero **parseNamesDeclared** e **parseCompleteSchema**, perciò facciamo l'analisi della coverage di questi due.

Per quanto riguarda il metodo **parseNamesDeclared**, tramite **Jacoco** si vede che è ottenuta una **branch coverage** del 70%, con i branch mancanti visualizzabili in [Figura 4.24](#) e [Figura 4.25](#). Analizziamoli uno per uno alle rispettive LOC:

- 1880: nel branch mancato non si può mai entrare perchè il metodo **parse**, prima di invocare **parseNamesDeclared**, controlla che lo schema non sia null
- 1895: da qui si apprende che esiste un'ulteriore tipo di schema, il tipo *error*, che dalle specifiche non sembrava essere riportato; leggendo la documentazione più nel dettaglio si scopre che questo tipo è trattato come un record, per cui si migliorano i test per comprendere anche questo tipo di dato (*ERROR*), e gli si aggiunge un namespace per coprire anche il branch a LOC 1898
- 1907: si osserva che i tipi record ed error definiti hanno sempre dei field, e questi sono sempre degli array, per cui si migliorano i test aggiungendo anche i controlli per un record che non ha field (*NO_FIELD_RECORD*) e per un record con field che non è un array (*NO_ARRAY_FIELD_RECORD*)
- 1909: vediamo che non è coperto il caso in cui name è null, ma questo branch è non raggiungibile dal momento che la funzione *getRequiredText* lancia un'eccezione in quel caso, come scritto nel commento della funzione (vedere [Figura 4.26](#))
- 1913: si ha lo stesso problema che si aveva a LOC 1907 ma per il caso delle enum, perciò si aggiornano i test includendo anche nodi json di enum senza simboli (*NO_SYMBOLS_ENUM*) o con un simbolo che non è un array (*NO_ARRAY_SYMBOLS_ENUM*)
- 1920: ci si accorge che non si sono mai considerate enum con un valore di default, perciò aggiungiamo anche questo caso di test (*DEFAULT_VALUE_ENUM*)

- 1926: non si sono inclusi vettori senza il nodo items, perciò la suite è aggiornata aggiungendo anch'essi (*NO_ITEMS_ARRAY*)
- 1932: si aggiunge il caso in cui una map abbia il nodo values non specificato (*NO_VALUES_MAP*)
- 1938: per il tipo fixed si considera il caso in cui non c'è la size o questa non è un numero intero (rispettivamente *NO_SIZE_FIXED* e *NO_INT_SIZE_FIXED*)
- 1941: come per LOC 1909
- 1963: qui si controlla che lo schema sia testuale, fin'ora si erano sempre usati nodi json mappati a partire da delle stringhe, per cui si crea invece un nodo json mappato da un intero così da rispettare anche quest'ultimo branch (*NON_TEXTUAL*)

Con questi miglioramenti, riassunti in [Tabella 4.23](#) considerando che i branch non coperti sono irraggiungibili come spiegato sopra, si ottiene una **branch coverage** del 100%, come si può vedere in [Figura 4.27](#) e [Figura 4.28](#).

I risultati di **Jacoco** per il metodo **parseCompleteSchema** mostrano invece una **branch coverage** dell'82% con i branch mancanti visualizzabili in [Figura 4.29](#) e [Figura 4.30](#). Anche in questo caso studiamo come possiamo migliorare i nostri test vedendo i branch mancanti ad ogni LOC:

- 2020: come per la LOC 1880 di `parseNamesDeclared`
- 2054: `result` non è mai null, e non può esserlo dal momento il suo valore è restituito dalla mappa di namespace, la quale però è stata aggiornata precedentemente da `parseNamesDeclared`, e quindi è sempre presente
- 2059: i due branch mancanti non sono raggiungibili, il primo è il caso in cui `result` è null, ma questo non può succedere come detto prima, e il secondo invece è quando `result` non ha il campo `field`, ma questo non è possibile di nuovo perchè `parseNamesDeclared` fa già questo controllo, e ferma l'esecuzione se così è.
- 2062: analogo a 2059, `parseNamesDeclared` controlla anche che queste informazioni siano presenti
- 2069: analogo a 2059, `parseNamesDeclared` controlla anche che queste informazioni siano presenti

Siccome questi branch sono irraggiungibili, anche in questo caso si ottiene una **branch coverage** del 100%.

L'analisi di **Badua** per il metodo **parseNamesDeclared** è visibile in [Codice 4.10](#): 21 coppie sono state mancate in totale su 159, andiamole ad analizzare.

Consideriamo la coppia in [Codice 4.11](#) vale lo stesso ragionamento fatto con Jacoco per la LOC 1880, quindi questa coppia non può essere coperta.

Prendiamo invece in esame tutte le coppie in [Codice 4.12](#): qui la variabile `name` è definita null, ma come si diceva anche prima, grazie alla funzione `getRequiredText`, a riga 1900 gli viene assegnato un valore diverso da null (sennò viene lanciata un'eccezione), e questa funzione è invocata in tutti i casi in cui lo schema è di tipo enum, record, error o fixed, poi `name` viene usata da LOC 1902 fino alla fine del metodo solo se il tipo di schema è sempre uno di quei quattro, quindi negli utilizzi successivi `name` non viene mai usato da null, perciò tutte queste coppie non sono raggiungibili.

Per quanto riguarda le coppie in [Codice 4.13](#) inerenti la variabile `doc` inizializzata a null, gli utilizzi di queste coppie sono tutti sempre successivi al suo riassegnamento a LOC 1897, che avviene se lo schema è di nuovo una enum, record, error o fixed, e gli use di queste coppie avvengono precisamente in questi casi, quindi anch'esse sono coppie non raggiungibili.

Le coppie in [Codice 4.14](#) invece, inerenti alla definizione di `result` col valore null a LOC 1889, non sono raggiungibili perchè parte di un if che vuole `result` diverso da null.

Per le coppie invece in [Codice 4.15](#), queste sono irraggiungibili per lo stesso motivo per cui in Jacoco i branch corrispondenti non sono raggiungibili.

Infine le coppie in [Codice 4.16](#) sarebbero raggiungibili se `result` potesse essere null, ma nelle definizioni delle coppie sono usati metodi che, in caso di errore, non ritornano null bensì un'eccezione, per cui anche queste coppie non sono raggiungibili.

Quindi per questo metodo si è totalizzata una **all-uses-coverage** del 100%.

Infine per il metodo **parseCompleteSchema** **Badua** ci dà il risultato che si può osservare in [Codice 4.17](#): le coppie mancanti sono 28 su 118, raggruppate in [Codice 4.18](#), ma con ragionamenti del tutto analoghi a quelli del caso precedente si vede che tutte queste coppie non sono raggiungibili, per cui si ottiene anche qui una **all-uses coverage** del 100%.

3.1.6 Mutation testing

Andiamo infine ad analizzare la bontà dei nostri test usando **Pitest**.

Per il metodo **parseNamesDeclared** si hanno 4 mutanti sopravvissuti, come si può vedere in [Figura 4.31](#) e

Figura 4.32.

I primi due mutanti sono inerenti alla chiamata al metodo privato *exploreFields* a LOC 1909, il primo elimina questa chiamata mentre il secondo nega la condizione: questi mutanti ci permettono di notare l'assenza di test per uno schema record che ha come field un altro schema record, per cui si è aggiunta anche questa casistica (*RECORD_WITH_RECORD_FIELDS*, vedere Tabella 4.24) e così è stato possibile uccidere i due mutanti, come si può vedere in Figura 4.33.

Gli ultimi 2 mutanti sono quello a LOC 1946, in cui si nega la condizione, e a LOC 1951, in cui si rimuove la chiamata *addProperties*: per uccidere questi mutanti c'è stato bisogno di considerare quindi uno schema che avesse delle proprietà, cosa che fino ad adesso non era stata presa in esame dal momento che ogni schema aveva solo i campi richiesti; studiando più a fondo la documentazione, verso la fine si scopre che alcuni tipi, complessi o primitivi, possono diventare dei tipi logici aggiungendogli ulteriori campi, detti proprietà appunto; per uccidere questi mutanti quindi si è aggiunto un test (*LOGICAL_TYPE_DATE*, vedere Tabella 4.24) per un tipo logico *Date* aggiungendo una proprietà al tipo primitivo *int*, e si sono migliorati i test per controllare che il tipo logico dello schema creato fosse attribuito correttamente, ottenendo come risultato la corretta uccisione di questi due mutanti, come si può vedere in Figura 4.34.

Infine per il metodo **parseCompleteSchema** abbiamo 4 mutanti che sono rimasti in vita come possiamo vedere da Figura 4.35.

I mutanti a LOC 2034 e a LOC 2096 sostituiscono entrambi il valore di ritorno alla rispettiva riga con null, ma sono mutanti equivalenti nel nostro contesto, dal momento che dentro il metodo *parse* non vengono effettuati controlli sul valore di ritorno e queste LOC non alterano lo stato della classe, e tantomeno il suo comportamento.

Per quanto riguarda invece i due mutanti che alterano le condizioni a LOC 2069, questi non sono due mutanti equivalenti, ma siccome l'unico cambiamento tra il SUT e questi due mutanti è una chiamata ad un Logger statico, l'unico modo per ucciderli è controllare se la chiamata al logger è avvenuta o meno: purtroppo Mockito non permette di mockare campi statici, si è invece usata una sua estensione chiamata **PowerMockito** per farlo, come si può vedere in Figura 4.36¹: si prende l'istanza di logger che la classe *Schema* ha creato e la si sostituisce con la medesima, ma parzialmente mockata (tramite *spy()*), dopo di che si verifica che la chiamata avvenga per il numero di volte che ci si aspetta (nello schema implementato nel test specifico era 1), implementando il test visibile in Tabella 4.25. Questo permette di uccidere correttamente questi due mutanti come si può osservare in Figura 4.37.

3.1.7 Stima dell'affidabilità

Ipotizzando una distribuzione uniforme di utilizzo, rispetto all'insieme dei casi di test si può assumere una reliability pari al 9%, con soltanto un test fallito dei 102 implementati.

3.2 SchemaCompatibility

3.2.1 Package di test

https://github.com/AlessandroFinocchi/avro/tree/ale_branch/lang/java/avro/src/test/java/org/apache/avro (Vedere le classi di test "*Compatibility*Test*")

3.2.2 Introduzione

Come si legge dalla documentazione, chi legge dei dati di Avro può sempre farne il parsing perchè lo schema originale deve essere fornito insieme ai dati; tuttavia, il lettore può voler leggerli in uno schema diverso, non solo nel tipo, ma anche nella versione, magari con campi in più o in meno. Si definiscono:

- **writer schema**: lo schema usato da chi ha scritto i dati,
- **reader schema**: lo schema usato da chi legge i dati.

Lo scopo di questa classe è scoprire se gli schemi writer e reader sono compatibili.

Dalle specifiche troviamo che questi schemi sono compatibili nei seguenti casi:

- entrambi sono **array** dello stesso tipo di elementi
- entrambi sono **mappe** dello stesso tipo di valori
- entrambi sono **enumerazioni** con gli stessi nomi: se un simbolo nel writer non è presente nel reader e il reader ha un valore di default viene usato quel valore, altrimenti è segnalato un errore
- entrambi sono **fixed** con gli stessi nomi e size
- entrambi sono **record** con gli stessi nomi: l'ordine dei campi è indifferente e se il writer contiene un campo con un nome non presente nel reader viene ignorato; infine se il reader ha uno campo con un nome non presente nel writer, se c'è un valore di default viene preso quello, altrimenti è segnalato un

¹ I test con PowerMockito sono stati esclusi da Badua per problemi di incompatibilità tra i tools.

errore.

- almeno uno dei due schemi è una **union**:
 - se entrambi sono union, al primo schema della union reader che fa matching con lo schema della union writer selezionata sono compatibili, se nessuno fa matching è segnalato un errore
 - se solo il reader è una union, al primo schema della union reader che fa matching con lo schema writer sono compatibili, se nessuno fa matching è segnalato un errore
 - se solo il writer è una union, se lo schema reader fa matching con uno dei tipi della union sono compatibili, altrimenti è segnalato un errore
- entrambi sono dello stesso tipo **primitivo**, altrimenti lo schema writer può essere promosso a quello del reader, in particolare:
 - int può essere promosso a long, float o double
 - long può essere promosso a float o double
 - float può essere promosso a double
 - string può essere promosso a bytes
 - bytes può essere promosso a string

Inoltre, i campi doc e aliases di uno schema sono ignorati ai fini della compatibilità.

Per questa classe si testerà il metodo **checkReaderWriterCompatibility**, il punto da cui inizia il processo di scoperta della compatibilità, ma per l'analisi della coverage e mutation testing, siccome questo metodo è piuttosto semplice e un 100% di coverage o mutation score non rispecchierebbe appieno quanto si è esplorato della compatibilità, si studieranno anche il metodo **getCompatibility**, chiamato dal primo, e **calculateCompatibility**, chiamato da quest'ultimo, entrambi metodi chiave nel calcolo della compatibilità.

3.2.3 Generazione test da Category Partition

CheckReaderWriterCompatibility

```
public static SchemaPairCompatibility checkReaderWriterCompatibility(Schema reader, Schema writer)
```

Questo metodo valida che lo schema reader può essere usato per decodificare i dati avro scritti con lo schema writer. L'istanza ritornata fornisce informazioni sul risultato della compatibilità della coppia di schemi, descrivendo la relazione tra reader e writer in modo unidirezionale: grazie ad essa si può scoprire se gli schemi sono compatibili o meno, e in caso non lo siano fornisce ulteriori informazioni sul tipo di incompatibilità.

Identificazione delle classi di equivalenza

- *writer*: questo schema è quello usato per serializzare i dati, per questo lo studiamo isolato
⇒ {istanza valida}, {istanza non valida}, {null}
- *reader*: questo schema è usato per deserializzare i dati precedentemente scritti con lo schema writer, per cui fissata un'istanza valida del writer, oltre a vedere la validità del reader dobbiamo studiarne anche la compatibilità con il writer
⇒ {istanza valida compatibile}, {istanza valida incompatibile}, {istanza non valida}, {null}

Boundary analysis

- *writer*:
 - istanza valida: per ogni tipo si prenderà un'istanza di quel tipo di schema
 - istanza non valida: l'istanza lancia eccezioni quando viene chiamato un metodo usato all'interno di metodo sotto test (è stato scelto *getType()*). Non si è usato Mockito per via dell'incompatibilità con Badua, si è invece estesa la classe Schema per fare override del metodo (vedere [Figura 4.38](#))
 - null
- *reader*
 - istanza valida compatibile: per istanza di writer valida si prenderà una o più istanze di reader compatibili, seguendo le linee guida delle specifiche
 - istanza valida non compatibile: per istanza di writer valida si prenderà un'istanza ad essa non compatibile, partendo dai casi di incompatibilità definiti dalle specifiche
 - istanza non valida: l'istanza lancia eccezioni quando viene chiamato un metodo usato all'interno di metodo sotto test (è stato scelto il metodo *getType()*). Non si è usato Mockito per via di problemi con Badua, si è invece estesa la classe Schema per fare override del metodo (vedere [Figura 4.38](#))
 - null

3.2.4 Definizione e risultati test

La test suite implementata si può vedere in [Tabella 4.26](#): si sono provate sia istanze non valide, sia istanze compatibili tra di loro che incompatibili, e le seconde si sono studiate separatamente a seconda che fossero in riferimento a schemi primitivi o complessi dal momento che le incompatibilità cambiano a seconda di questo.

Per le istanze non valide ci si aspettava un'eccezione, per le altre invece un risultato di compatibilità, e in caso gli schemi non fossero compatibili si vanno anche a controllare le incompatibilità che il metodo ha trovato, assicurandosi che siano quelle attese. Tutti i test fino a questa fase sono passati.

3.2.5 Analisi adeguatezza dei test

Dall'analisi di **Jacoco** in [Figura 4.39](#) possiamo vedere che si è ottenuta una **branch coverage** del 66% per il metodo **checkReaderWriterCompatibility**, con solo un branch non coperto, quello relativo al caso in cui `getCompatibility()` è di tipo *RECURSION_IN_PROGRESS*, ma la funzione che genera il risultato, ovvero `getCompatibility`, è costruita appositamente per intercettare il valore e trasformarlo in *COMPATIBLE*, quindi il metodo `checkReaderWriterCompatibility` non lo vedrà mai, quindi questo branch è irraggiungibile.

Andiamo a studiare allora anche il metodo **getCompatibility**: da **Jacoco** scopriamo che soltanto 1 dei 4 branch presenti è coperto, come si può vedere in [Figura 4.40](#): il problema in questo caso è che il metodo `caller` ogni volta crea una nuova istanza della classe `ReaderWriterCompatibilityChecker` su cui chiama `getCompatibility`, di conseguenza la mappa `mMemoizeMap` è sempre vuota, e quindi l'if a LOC 233 entra sempre nel secondo branch, saltando gli altri 3, che perciò non sono raggiungibili dal metodo da testare scelto.

Il risultato finale della compatibilità è dato dal metodo **calculateCompatibility**: da **Jacoco** si rileva per questo metodo una **branch coverage** del 82%, con un totale di 8 branch non coperti, come possiamo vedere in [Figura 4.41](#) e [Figura 4.42](#).

Il branch non coperto a LOC 269 è quello che porta al caso di default, ma dal momento che tutti i valori possibili di `reader.getType()` sono coperti, questo branch è irraggiungibile.

I branch non coperti a LOC 335 sono invece inerenti a combinazioni di tipi di schemi non provati, per cui si aggiungono le combinazioni in [Tabella 4.27](#) per coprire tutti i branch dello switch, tranne l'ultimo, caso equivalente al precedente e perciò irraggiungibile.

I risultati della nuova copertura sono visibili in [Figura 4.43](#) e permettono di arrivare, considerati i casi di branch non raggiungibili, ad una **branch coverage** del 100%.

Dall'analisi di **Badua** per **checkReaderWriterCompatibility** nel [Codice 4.19](#) possiamo vedere che tutte le coppie `def use` non coperte sono quelle che hanno come `use` o `target` la LOC 77, che dal report di **Jacoco** avevamo già notato essere irraggiungibile, per cui tutte queste coppie anche sono irraggiungibili, e la **all uses coverage** è dunque del 100%.

Continuando lo studio con il metodo **getCompatibility**, dal [Codice 4.20](#), troviamo ovviamente le coppie in cui `use` e `target` sono del branch dell'if più esterno che avevamo visto con **Jacoco** essere irraggiungibile, e inoltre la penultima coppia non coperta, ovvero

```
<du var="result" def="232" use="246" covered="0"/>
```

non è raggiungibile dal momento che dopo la definizione di `result` a LOC 232, questa viene ridefinita sempre a LOC 242: in realtà sarebbe raggiungibile, se si entrasse nel primo branch dell'if a LOC 233, e se la condizione dell'if innestato a LOC 234 fosse falsa questa coppia verrebbe raggiunta, ma siccome non si può entrare nel primo, non può avvenire questo caso nel nostro contesto di testing, perciò anche questa coppia è irraggiungibile nel nostro contesto e dunque ci attestiamo anche qui con una **all uses coverage** del 100%.

Infine esaminiamo l'analisi di **Badua** per il metodo **calculateCompatibility**, visualizzabile nel [Codice 4.21](#): le coppie mancate sono solo 9 su 256, raggruppate in [Codice 4.22](#), ma 6 di queste, quelle con `use` o `target` pari a 318 o 387, sono quelle inerenti ai branch di default dei due switch, rispettivamente il primo a LOC 318 e il secondo a LOC 387, e dai ragionamenti fatti precedentemente sono irraggiungibili; dalla coppia

```
<du var="result" def="266" use="332" covered="0"/>
```

invece ci si accorge che non si era considerato il caso in cui il `writer` era una `union` vuota e il `reader` un tipo diverso da una `union`, caso in cui la coppia di schemi è compatibile, per cui si aggiunge anche questo caso di test per coprire la coppia, ma è da sottolineare che la documentazione è agnostica di questo fatto.

Infine, per le ultime due coppie relative alla variabile `assertionsDisabled`, questa variabile viene aggiunta a `compile time` ogni volta che si trova una `assert`, e serve a determinare se le asserzioni sono abilitate o meno a `runtime`: per coprirle basta andare a aggiungere l'opzione `"-da"` alla configurazione di `surefire` di **Badua**.

Con questi miglioramenti sono quini coperte le 3 coppie raggiungibili, e siccome le altre 6 erano non raggiungibili, si calcola una **all uses coverage** del 100% anche in questo caso.

3.2.6 Mutation testing

Dal report di `pitest` in [Figura 4.44](#) si può vedere che tutti i mutanti per il metodo **checkReaderWriterCompatibility** sono stati correttamente uccisi.

Per **getCompatibility** invece l'unico mutante non ucciso è quello del branch dell'if non raggiungibile dall'analisi sulla coverage, come si può vedere in [Figura 4.45](#), quindi è un mutante equivalente nel nostro contesto.

Infine, per il metodo **calculateCompatibility** l'analisi di `pitest`, visualizzabile in [Figura 4.46](#) e [Figura 4.47](#), ci

dice che solo 2 mutanti sopravvivono, entrambi che modificano l'intero che rappresenta il reference token da scrivere dentro il deck (un'oggetto Deque di `java.util`): questo è usato come uno stack per mantenere la posizione corrente in cui si sta calcolando la compatibilità nella gerarchia dello schema e, se un'incompatibilità viene rilevata, gli si associa il valore del deck che rappresenterà univocamente la posizione nello schema da cui è nata l'incompatibilità. Siccome, come si può vedere, questa posizione è rappresentata sempre come un numero non negativo, si migliorano i test per controllare che questo vincolo sia soddisfatto, e in questo modo si rilevano correttamente i mutanti come si può vedere in [Figura 4.48](#).

3.2.7 Stima dell'affidabilità

Ipotizzando una distribuzione uniforme di utilizzo, rispetto all'insieme dei casi di test si può assumere una reliability pari 100%, con nessun test fallito dei 57 implementati.

4 Immagini e tabelle

Figura 4.1: Mock per l'allocator non valido

```
public static ByteBufferAllocator invalidByteBufferAllocator() { 1 usage AlessandroFinocchi
    ByteBufferAllocator bba = mock(ByteBufferAllocator.class);

    when(bba.buffer()).thenReturn(t null);
    when(bba.buffer(anyInt())).thenReturn(t null);
    when(bba.buffer(anyInt(), anyInt())).thenReturn(t null);

    when(bba.directBuffer()).thenReturn(t null);
    when(bba.directBuffer(anyInt())).thenReturn(t null);
    when(bba.directBuffer(anyInt(), anyInt())).thenReturn(t null);

    return bba;
}
```

Figura 4.2: Mock per il file channel non valido

```
public static FileChannel invalidPositionFileChannel() throws IOException { 1 usage AlessandroFinocchi
    FileChannel fc = spy(validFileChannel());
    when(fc.position()).thenReturn(t -1L);
    return fc;
}
```

Figura 4.3: Relazioni tra indici di un ByteBuffer

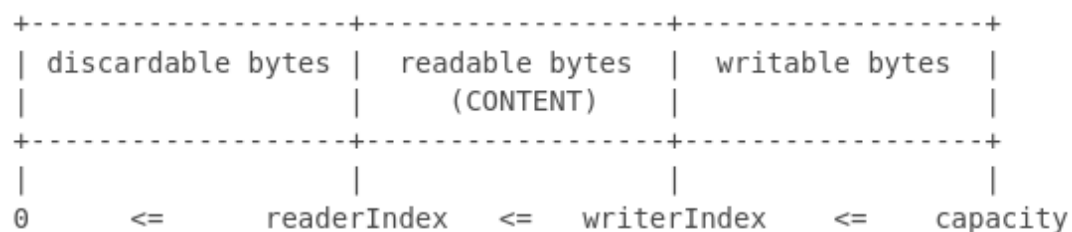


Tabella 4.1: Testing costruttore BufferedChannel

Test	allocator	fc	writeCapacity	readCapacity	unpersistedBytesBound
<i>Variazioni ad allocator</i>					
T1	valid	valid	100	100	1
T2	invalid	valid	100	100	1
T3	null	valid	100	100	1
<i>Variazioni ad fc</i>					
T4	valid	read-only	100	100	1
T5	valid	write-only	100	100	1
T6	valid	closed	100	100	1
T7	valid	invalid pos	100	100	1
T8	valid	null	100	100	1
<i>Variazioni a writeCapacity</i>					
T9	valid	valid	1	100	1
T10	valid	valid	0	100	1
T11	valid	valid	-1	100	1
<i>Variazioni a readCapacity</i>					
T12	valid	valid	100	1	1
T13	valid	valid	100	0	1
T14	valid	valid	100	-1	1
<i>Variazioni a unpersistedBytesBound</i>					
T15	valid	valid	100	100	50
T16	valid	valid	100	100	0
T17	valid	valid	100	100	-1

Tabella 4.2: Testing costruttore BufferedChannel

Test	Risultato atteso	Motivazione
T1	valid instance	Un allocatore unpooled alloca normalmente un buffer
T2	Exception	L'allocatore non è valido poiché genera buffer nulli
T3	Exception	L'allocatore null non permette di allocare alcun buffer
T4	valid instance	Un file channel read-only non deve dar errori ora, ma durante una write dovrà
T5	valid instance	Un file channel write-only non deve dar errori ora, ma durante una read dovrà
T6	Exception	Un file channel chiuso non può essere usato per fare operazioni di lettura o scrittura
T7	Exception	Una posizione non valida genera errori durante operazioni di lettura e scrittura
T8	Exception	Il file channel null non permette di fare operazioni di lettura e scrittura
T9	valid instance	Un buffer di scrittura di dimensione 1 è perfettamente utilizzabile
T10	valid instance	Un buffer di scrittura di dimensione 0 non deve dar problemi ora ma durante una write
T11	Exception	Un buffer di scrittura non può avere capacità negativa
T12	valid instance	Un buffer di lettura di dimensione 1 è perfettamente utilizzabile
T13	valid instance	Un buffer di lettura di dimensione 0 non deve dar problemi ora ma durante una read
T14	Exception	Un buffer di lettura non può avere capacità negativa
T15	valid instance	Questo upperbound è valido
T16	valid instance	Questo upperbound è valido
T17	Exception	Questo upperbound non ha senso che sia un numero negativo

Tabella 4.3: Testing write BufferedChannel

Test	allocator	fc	writeCapacity	readCapacity	unpersistedBytesBound	src
<i>Istanze del costruttore fallite</i>						
W1	invalid	valid	100	100	1	full
W2	valid	invalid pos	100	100	1	full
W3	valid	valid	100	100	-1	full
<i>Contesto di fc non valido</i>						
W4	valid	read-only	100	100	1	full
<i>Contesto di writeCapacity non valida</i>						
W5	valid	valid	0	100	1	full
<i>Variando unpersistedBytesBound (in funzione della lunghezza dati da scrivere, srcContentLen)</i>						
W6	valid	valid	100	100	0	full
W7	valid	valid	100	100	srcContentLen - 1	full
W8	valid	valid	100	100	srcContentLen	full
W9	valid	valid	100	100	srcContentLen + 1	full
<i>Variazioni ad src</i>						
W10	valid	valid	100	100	1	empty
W11	valid	valid	100	100	1	semi-full
W12	valid	valid	100	100	1	full
W13	valid	valid	100	100	1	invalid read index
W14	valid	valid	100	100	1	deallocated
W15	valid	valid	100	100	1	null

Tabella 4.4: Testing write BufferedChannel

Test	Risultato atteso	Motivazione
W1	Exception	Corrisponde al caso di test T2: l'allocatore crea un buffer nullo, quindi al momento della scrittura deve sollevarsi un'eccezione
W2	Exception	Corrisponde al caso di test T7: la posizione del file channel è invalida, e siccome l'upperbound al numero di byte non persistenti è minore di quello che ci si scrive si effettuerà il flush, che quando avverrà scriverà nel file channel in una posizione invalida e quindi si deve sollevare un'eccezione
W3	Exception	Corrisponde al caso di test T20: il numero di byte che possono essere non persistenti è negativo, e siccome il costruttore non genera eccezioni, ci si aspetta che ne venga sollevata una qui almeno
W4	Exception	Il file channel è read-only, e siccome l'upperbound ai byte non persistenti è minore del numero di byte scritti, quando si effettua il flush nel file channel si deve sollevare un'eccezione
W5	Exception	La capacità del buffer di scrittura è zero, ovvero non può essere scritto, e un tale buffer non ha senso che sia usato per operazioni di scrittura quindi deve essere generata un'eccezione
W6	file channel scritto	L'upperbound ai byte non persistenti è minore di quelli che si vogliono scrivere, quindi ci si aspetta verranno scritti nel file channel correttamente
W7	file channel scritto	L'upperbound ai byte non persistenti è minore di quelli che si vogliono scrivere, quindi ci si aspetta verranno scritti nel file channel correttamente
W8	file channel scritto	L'upperbound ai byte non persistenti è uguale a quelli che si vogliono scrivere, quindi ci si aspetta verranno scritti nel file channel correttamente
W9	buffer scritto	L'upperbound ai byte non persistenti è maggiore di quelli che si vogliono scrivere, quindi ci si aspetta non vengano scritti nel file channel ma al tempo stesso che nessuna eccezione sia generata
W10	nulla scritto	La sorgente di dati è vuota, quindi ci si aspetta non venga scritto nulla ne nel buffer ne nel file channel
W11	file channel scritto	La sorgente di dati contiene dei byte da leggere, i quali ci si aspetta vengano scritti nel file channel dal momento che l'upperbound al numero di byte non persistenti è minore del numero di byte che si vogliono scrivere
W12	file channel scritto	La sorgente di dati contiene dei byte da leggere, i quali ci si aspetta vengano scritti nel file channel dal momento che l'upperbound al numero di byte non persistenti è minore del numero di byte che si vogliono scrivere
W13	Exception	La sorgente di dati ha un indice di lettura non valido, per cui quando si leggeranno i dati da essa per scriverli nel buffer di scrittura, ci si aspetta un'eccezione
W14	Exception	La sorgente di dati è deallocata quindi non ci si può leggere
W15	Exception	La sorgente di dati è nulla quindi non ci si può leggere

Tabella 4.5: Testing read BufferedChannel

Test	stato istanza	stato writeBuffer	dest	pos	length
<i>Istanze del costruttore fallite (invalide in contesti di read, fCL = fcContentLen)</i>					
R1	invalid allocator	empty	empty	0	fCL
R2	invalid fc pos	empty	empty	0	fCL
<i>Contesto di fc non valido</i>					
R3	write-only fc	empty	empty	0	1
<i>Contesto di readCapacity non valida</i>					
R4	readCapacity=0	empty	empty	0	1
<i>Variazioni a dest</i>					
R5	valid	empty	empty	0	fCL
R6	valid	empty	semi-full	0	fCL/2
R7	valid	empty	full	0	fCL
R8	valid	empty	invalid write index	0	fCL
R9	valid	empty	deallocated	0	fCL
R10	valid	empty	null	0	fCL
<i>Variazioni a pos e length (in funzione della dimensione del contenuto del buffer di scrittura wbCL e del file channel fCL)</i>					
R11	valid	scritto con wbCL byte	empty	-1	1
R12	valid	scritto con wbCL byte	empty	1	-1
R13	valid	scritto con wbCL byte	empty	0	0
R14	valid	scritto con wbCL byte	empty	0	1
R15	valid	scritto con wbCL byte	empty	0	fCL-1
R16	valid	scritto con wbCL byte	empty	0	fCL
R17	valid	scritto con wbCL byte	empty	1	fCL-1
R18	valid	scritto con wbCL byte	empty	fCL	1
R19	valid	scritto con wbCL byte	empty	fCL+wbCL-1	1
R20	valid	scritto con wbCL byte	empty	0	fCL+1
R21	valid	scritto con wbCL byte	empty	0	fCL+wbCL
R22	valid	scritto con wbCL byte	empty	0	fCL+wbCL+1
R23	valid	scritto con wbCL byte	empty	fCL+wbCL	1

Tabella 4.6: Testing read BufferedChannel

Test	Risultato atteso	Motivazione
R1	Exception	Corrisponde al caso di test T2: l'allocatore crea un buffer nullo, quindi al momento della lettura deve sollevarsi un'eccezione
R2	Exception	Corrisponde al caso di test T7: la posizione del file channel è sempre invalida, e ci si aspetta che, seppur venga comunicata la posizione, alla fine la read usi l'attributo posizione di fc per leggere, per cui ci si aspetta un'eccezione
R3	Exception	Il file channel è write-only, per cui ci si aspetta che le letture non siano ammissibili
R4	Exception	La capacità del buffer di lettura è zero, ovvero non può essere letto, e un tale buffer non ha senso che sia usato per operazioni di lettura, quindi deve essere generata un'eccezione
R5	lettura di fCL byte dal file channel	La destinazione dei dati è vuota, ovvero tutti i byte sono scrivibili, per cui a seguito della read di fCL byte, ci si aspetta che tutti gli fCL byte vengano scritti in dest
R6	lettura di fCL/2 byte dal file channel	La destinazione dei dati è piena a metà, gli rimangono solo fCL/2 byte liberi, e ne vogliono essere letti esattamente fCL/2, quindi ci aspettiamo di leggerli tutti
R7	Exception	La destinazione dei dati è piena, ovvero non ha più spazio scrivibile, quindi ci si aspetta un'eccezione
R8	Exception	La destinazione dei dati ha un indice di scrittura non valido, quindi scrivendoci deve essere generata un'eccezione
R9	Exception	La destinazione dei dati è deallocata, quindi non può essere usata per operazioni di scrittura
R10	Exception	La destinazione dei dati è nulla quindi non ci si può scrivere
R11	Exception	La posizione non è valida, quindi ci si aspetta un'eccezione
R12	Exception	La lunghezza non è valida, quindi ci si aspetta un'eccezione
R13	nessun byte letto dal file channel	La posizione è valida, e se length è 0 ci si aspetta di leggere 0 byte
R14	lettura di fCL byte dal file channel	Ci si aspetta di leggere tutti gli fCL byte e metterli in cache, ma scrivere dentro dest solo il primo byte
R15	lettura di fCL byte dal file channel	Ci si aspetta di leggere tutti gli fCL byte e metterli in cache, ma scrivere dentro dest i byte [0, fCL-1]
R16	lettura di fCL byte dal file channel	Ci si aspetta di leggere tutto il contenuto del file channel e scriverlo in dest
R17	lettura di fCL-1 byte dal buffer	Ci si aspetta di leggere dal secondo all'ultimo byte del file channel e scriverlo in dest
R18	lettura di wbCL byte dal buffer	Ci si aspetta di leggere tutto il contenuto del buffer di scrittura ma di scrivere in dest solo il primo byte
R19	lettura di 1 byte dal buffer	Ci si aspetta di leggere l'ultimo byte del buffer di scrittura
R20	lettura di fCL+wbCL byte da fc e buffer	Ci si aspetta di leggere tutto il file channel e tutto il buffer di scrittura, ma di mettere in dest il contenuto del file channel fino al primo byte nel buffer di scrittura
R21	lettura di fCL+wbCL byte da fc e buffer	Ci si aspetta di leggere tutto il file channel e tutto il buffer di scrittura e scrivere tutto in dest
R22	Exception	Si vuole leggere tutto il file channel e il buffer di scrittura, ma anche il primo byte non leggibile del buffer di scrittura, quindi ci si aspetta un'eccezione
R23	Exception	Si vuole leggere il primo byte non leggibile del buffer di scrittura, quindi ci si aspetta un'eccezione

Tabella 4.7: Testing jacoco write BufferedChannel

Test	allocator	fc	writeCapacity	readCapacity	unpersistedBytesBound	src
<i>sCL = srcContentLen, la dimensione dei dati scritti in src</i>						
J-W1	valid	valid	sCL/2	100	0	full
J-W2	valid	valid	sCL/2 + 1	100	0	full

Tabella 4.8: Testing jacoco write BufferedChannel

Test	Risultato atteso	Motivazione
J-W1	scrittura	Siccome i byte non persistenti sono zero ci si aspetta che i byte scritti vengano scaricati nel file channel
J-W2	scrittura	Siccome i byte non persistenti sono zero ci si aspetta che i byte scritti vengano scaricati nel file channel

Figura 4.4: Risultati Jacoco per la write

```

117. public void write(ByteBuf src) throws IOException {
118.     int copied = 0;
119.     boolean shouldForceWrite = false;
120.     synchronized (this) {
121.         int len = src.readableBytes();
122.         while (copied < len) {
123.             int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124.             writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125.             copied += bytesToCopy;
126.
127.             // if we have run out of buffer space, we should flush to the
128.             // file
129.             if (!writeBuffer.isWritable()) {
130.                 flush();
131.             }
132.             position += copied;
133.             if (doRegularFlushes) {
134.                 unpersistedBytes.addAndGet(copied);
135.                 if (unpersistedBytes.get() >= unpersistedBytesBound) {
136.                     flush();
137.                     shouldForceWrite = true;
138.                 }
139.             }
140.         }
141.     }
142.     if (shouldForceWrite) {
143.         forceWrite(false);
144.     }
145. }

```

(a) Prima del test J-W1

```

117. public void write(ByteBuf src) throws IOException {
118.     int copied = 0;
119.     boolean shouldForceWrite = false;
120.     synchronized (this) {
121.         int len = src.readableBytes();
122.         while (copied < len) {
123.             int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124.             writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125.             copied += bytesToCopy;
126.
127.             // if we have run out of buffer space, we should flush to the
128.             // file
129.             if (!writeBuffer.isWritable()) {
130.                 flush();
131.             }
132.             position += copied;
133.             if (doRegularFlushes) {
134.                 unpersistedBytes.addAndGet(copied);
135.                 if (unpersistedBytes.get() >= unpersistedBytesBound) {
136.                     flush();
137.                     shouldForceWrite = true;
138.                 }
139.             }
140.         }
141.     }
142.     if (shouldForceWrite) {
143.         forceWrite(false);
144.     }
145. }

```

(b) Dopo il test J-W1

Tabella 4.9: Testing jacoco read BufferedChannel

Test	stato istanza	stato writeBuffer	dest	pos	length
<i>fCL = fcContentLen</i>					
J-R1	invalid allocator	empty	empty	fCL	1

Tabella 4.10: Testing jacoco read BufferedChannel

Test	Risultato atteso	Motivazione
J-R1	lettura di 0 byte dal buffer	Si vuole leggere dal buffer, ma questo è nullo, e come si vede da jacoco, quando il buffer è nullo non si ritorna una eccezione ma si fa solo una return, quindi ci si aspetta che il buffer non sia stato scritto e il valore di ritorno sia 0

Figura 4.5: Risultati Jacoco per la read

```

245. public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246.     long prevPos = pos;
247.     while (length > 0) {
248.         // check if it is in the write buffer
249.         if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
250.             int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
251.             int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
252.
253.             if (bytesToCopy == 0) {
254.                 throw new IOException("Read past EOF");
255.             }
256.
257.             dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
258.             pos += bytesToCopy;
259.             length -= bytesToCopy;
260.         } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
261.             // here we reach the end
262.             break;
263.             // first check if there is anything we can grab from the readBuffer
264.         } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
265.             int positionInBuffer = (int) (pos - readBufferStartPosition);
266.             int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
267.             dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
268.             pos += bytesToCopy;
269.             length -= bytesToCopy;
270.             // let's read it
271.         } else {
272.             readBufferStartPosition = pos;
273.
274.             int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
275.                 readBufferStartPosition);
276.             if (readBytes <= 0) {
277.                 throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
278.             }
279.             readBuffer.writerIndex(readBytes);
280.         }
281.     }
282.     return (int) (pos - prevPos);
283. }

```

(a) Prima dell'analisi con jacoco

```

245. public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246.     long prevPos = pos;
247.     while (length > 0) {
248.         // check if it is in the write buffer
249.         if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
250.             int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
251.             int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
252.
253.             if (bytesToCopy == 0) {
254.                 throw new IOException("Read past EOF");
255.             }
256.
257.             dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
258.             pos += bytesToCopy;
259.             length -= bytesToCopy;
260.         } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
261.             // here we reach the end
262.             break;
263.             // first check if there is anything we can grab from the readBuffer
264.         } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
265.             int positionInBuffer = (int) (pos - readBufferStartPosition);
266.             int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
267.             dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
268.             pos += bytesToCopy;
269.             length -= bytesToCopy;
270.             // let's read it
271.         } else {
272.             readBufferStartPosition = pos;
273.
274.             int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
275.                 readBufferStartPosition);
276.             if (readBytes <= 0) {
277.                 throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
278.             }
279.             readBuffer.writerIndex(readBytes);
280.         }
281.     }
282.     return (int) (pos - prevPos);
283. }

```

(b) Dopo l'analisi con jacoco

Codice 4.1: Risultati Badua per la write

```

<method name="write" desc="(Lio/netty/buffer/ByteBuf;)V">
  <du var="this" def="118" use="133" covered="1"/>
  <du var="this" def="118" use="134" target="135" covered="1"/>
  <du var="this" def="118" use="134" target="141" covered="1"/>
  <du var="this" def="118" use="143" covered="1"/>
  <du var="this" def="118" use="135" covered="1"/>
  <du var="this" def="118" use="136" target="137" covered="1"/>
  <du var="this" def="118" use="136" target="141" covered="1"/>
  <du var="this" def="118" use="137" covered="1"/>
  <du var="this" def="118" use="123" covered="1"/>
  <du var="this" def="118" use="124" covered="1"/>
  <du var="this" def="118" use="129" target="130" covered="1"/>
  <du var="this" def="118" use="129" target="132" covered="1"/>
  <du var="this" def="118" use="130" covered="1"/>
  <du var="src" def="118" use="123" covered="1"/>
  <du var="src" def="118" use="124" covered="1"/>
  <du var="this.writeBuffer" def="118" use="123" covered="1"/>
  <du var="this.writeBuffer" def="118" use="124" covered="1"/>
  <du var="this.writeBuffer" def="118" use="129" target="130" covered="1"/>
  <du var="this.writeBuffer" def="118" use="129" target="132" covered="1"/>
  <du var="this.position" def="118" use="133" covered="1"/>
  <du var="this.doRegularFlushes" def="118" use="134" target="135" covered="1"/>
  <du var="this.doRegularFlushes" def="118" use="134" target="141" covered="1"/>
  <du var="this.unpersistedBytes" def="118" use="135" covered="1"/>
  <du var="this.unpersistedBytes" def="118" use="136" target="137" covered="1"/>
  <du var="this.unpersistedBytes" def="118" use="136" target="141" covered="1"/>
  <du var="this.unpersistedBytesBound" def="118" use="136" target="137" covered="1"/>
  <du var="this.unpersistedBytesBound" def="118" use="136" target="141" covered="1"/>
  <du var="copied" def="118" use="122" target="123" covered="1"/>
  <du var="copied" def="118" use="122" target="133" covered="1"/>
  <du var="copied" def="118" use="133" covered="1"/>
  <du var="copied" def="118" use="135" covered="1"/>
  <du var="copied" def="118" use="123" covered="1"/>
  <du var="copied" def="118" use="124" covered="1"/>
  <du var="copied" def="118" use="125" covered="1"/>
  <du var="shouldForceWrite" def="119" use="142" target="143" covered="0"/>
  <du var="shouldForceWrite" def="119" use="142" target="145" covered="1"/>
  <du var="len" def="121" use="122" target="123" covered="1"/>
  <du var="len" def="121" use="122" target="133" covered="1"/>
  <du var="copied" def="125" use="122" target="123" covered="1"/>
  <du var="copied" def="125" use="122" target="133" covered="1"/>
  <du var="copied" def="125" use="133" covered="1"/>
  <du var="copied" def="125" use="135" covered="1"/>
  <du var="copied" def="125" use="123" covered="1"/>
  <du var="copied" def="125" use="124" covered="1"/>
  <du var="copied" def="125" use="125" covered="1"/>
  <du var="shouldForceWrite" def="138" use="142" target="143" covered="1"/>
  <du var="shouldForceWrite" def="138" use="142" target="145" covered="0"/>
  <counter type="DU" missed="2" covered="46"/>
  <counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.2: Risultati Badua per la read

```

<method name="read" desc="(Lio/netty/buffer/ByteBuf;J)I">
  <du var="this" def="246" use="249" target="249" covered="1"/>
  <du var="this" def="246" use="249" target="260" covered="1"/>
  <du var="this" def="246" use="260" target="260" covered="1"/>
  <du var="this" def="246" use="260" target="264" covered="1"/>
  <du var="this" def="246" use="264" target="264" covered="1"/>
  <du var="this" def="246" use="264" target="272" covered="0"/>
  <du var="this" def="246" use="272" covered="1"/>
  <du var="this" def="246" use="274" covered="1"/>
  <du var="this" def="246" use="279" covered="1"/>
  <du var="this" def="246" use="264" target="265" covered="1"/>
  <du var="this" def="246" use="264" target="272" covered="1"/>
  <du var="this" def="246" use="265" covered="1"/>
  <du var="this" def="246" use="266" covered="1"/>
  <du var="this" def="246" use="267" covered="1"/>
  <du var="this" def="246" use="260" target="262" covered="1"/>
  <du var="this" def="246" use="260" target="264" covered="1"/>
  <du var="this" def="246" use="249" target="250" covered="1"/>
  <du var="this" def="246" use="249" target="260" covered="1"/>
  <du var="this" def="246" use="250" covered="1"/>
  <du var="this" def="246" use="251" covered="1"/>
  <du var="this" def="246" use="257" covered="1"/>
  <du var="dest" def="246" use="266" covered="1"/>
  <du var="dest" def="246" use="267" covered="1"/>
  <du var="dest" def="246" use="251" covered="1"/>
  <du var="dest" def="246" use="257" covered="1"/>
  <du var="pos" def="246" use="282" covered="1"/>
  <du var="pos" def="246" use="264" target="264" covered="1"/>
  <du var="pos" def="246" use="264" target="272" covered="0"/>
  <du var="pos" def="246" use="272" covered="1"/>
  <du var="pos" def="246" use="264" target="265" covered="1"/>

```

[illegible]

```

<du var="readBytes" def="274" use="279" covered="1"/>
<counter type="DU" missed="31" covered="87"/>
<counter type="METHOD" missed="0" covered="1"/>
</method>

```

Figura 4.6: Risultati jacoco read test B-R1

```

245. public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246.     long prevPos = pos;
247.     while (length > 0) {
248.         // check if it is in the write buffer
249.         if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
250.             int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
251.             int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
252.
253.             if (bytesToCopy == 0) {
254.                 throw new IOException("Read past EOF");
255.             }
256.
257.             dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
258.             pos += bytesToCopy;
259.             length -= bytesToCopy;
260.         } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
261.             // here we reach the end
262.             break;
263.             // first check if there is anything we can grab from the readBuffer
264.         } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
265.             int positionInBuffer = (int) (pos - readBufferStartPosition);
266.             int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
267.             dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
268.             pos += bytesToCopy;
269.             length -= bytesToCopy;
270.             // let's read it
271.         } else {
272.             readBufferStartPosition = pos;
273.
274.             int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
275.                 readBufferStartPosition);
276.             if (readBytes <= 0) {
277.                 throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
278.             }
279.             readBuffer.writerIndex(readBytes);
280.         }
281.     }
282.     return (int) (pos - prevPos);
283. }

```

Codice 4.3: Prima iterazione badua read

```

<du var="this" def="246" use="264" target="272" covered="0"/>
<du var="pos" def="246" use="264" target="272" covered="0"/>
<du var="this.readBufferStartPosition" def="246" use="264" target="272" covered="0"/>

```

Codice 4.4: Seconda iterazione badua read

```

<du var="this.readBufferStartPosition" def="246" use="264" target="265" covered="0"/>
<du var="this.readBufferStartPosition" def="246" use="265" covered="0"/>

```

Codice 4.5: Terza iterazione badua read

```

<du var="pos" def="258" use="264" target="264" covered="0"/>
<du var="pos" def="258" use="264" target="272" covered="0"/>
<du var="pos" def="258" use="272" covered="0"/>
<du var="pos" def="258" use="264" target="265" covered="0"/>
<du var="pos" def="258" use="264" target="272" covered="0"/>
<du var="pos" def="258" use="265" covered="0"/>
<du var="pos" def="258" use="268" covered="0"/>
<du var="length" def="259" use="269" covered="0"/>

```

Codice 4.6: Quarta iterazione badua read

```

<du var="pos" def="258" use="260" target="262" covered="0"/>
<du var="pos" def="258" use="260" target="264" covered="0"/>

```

Codice 4.7: Quinta iterazione badua read

```

<du var="pos" def="268" use="264" target="265" covered="0"/>

```

Codice 4.8: Sesta iterazione badua read

```

<du var="pos" def="258" use="249" target="260" covered="0"/>

```

Tabella 4.11: Test aggiunti a read dopo badua

Test	stato istanza	stato writeBuffer	dest	pos	length
<i>fCL = fcContentLen, wbCL = wbContentLen</i>					
B-R1	valid	empty	empty	0	fCL/2
B-R2	valid	empty	empty	1	6
B-R3	valid with both capacity half of the content write buffer written	empty	scritto con wbCL byte	0	fCL+wbCL

Tabella 4.12: Test aggiunti a read dopo badua

Test	Risultato atteso	Motivazione
B-R1	lettura byte [0,fCL/2] dal file channel	Ci si aspetta di leggere due volte il file channel e caricarlo in cache dal primo byte, e quindi ritornare fCL come numero di byte letti, ma scrivere in dest solo i primi fCL/2
B-R2	lettura byte [1,fCL/2+1] dal file channel	Ci si aspetta di leggere due volte il file channel e caricarlo in cache dal secondo byte, e quindi ritornare fCL-1 come numero di byte letti, ma scrivere in dest i byte [1, fCL/2-1]
B-R3	lettura di tutti i byte	Seppur la le capacità dei due buffer sono metà del contenuto che vogliono leggere, ci si aspetta semplicemente che vengano fatte più iterazioni ma che venga comunque letto tutto

Figura 4.7: Report di pitest per costruttore

```

88     public BufferedChannel(ByteBufAllocator allocator, FileChannel fc, int writeCapacity, int readCapacity,
89                             long unpersistedBytesBound) throws IOException {
90         super(fc, readCapacity);
91         this.writeCapacity = writeCapacity;
92         this.position = fc.position();
93         1 this.writeBufferStartPosition.set(position);
94         this.writeBuffer = allocator.directBuffer(writeCapacity);
95         this.unpersistedBytes = new AtomicLong(0);
96         this.unpersistedBytesBound = unpersistedBytesBound;
97         2 this.doRegularFlushes = unpersistedBytesBound > 0;
98     }

```

Figura 4.8: Report di pitest per write (prima)

```

117     public void write(ByteBuf src) throws IOException {
118         int copied = 0;
119         boolean shouldForceWrite = false;
120         synchronized (this) {
121             int len = src.readableBytes();
122             2 while (copied < len) {
123                 1 int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124                 writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125                 1 copied += bytesToCopy;
126             }
127             // if we have run out of buffer space, we should flush to the
128             // file
129             1 if (!writeBuffer.isWritable()) {
130                 1 flush();
131             }
132         }
133         1 position += copied;
134         1 if (doRegularFlushes) {
135             unpersistedBytes.addAndGet(copied);
136             2 if (unpersistedBytes.get() >= unpersistedBytesBound) {
137                 1 flush();
138                 shouldForceWrite = true;
139             }
140         }
141     }
142     1 if (shouldForceWrite) {
143         forceWrite(false);
144     }
145 }

```


Figura 4.9: Diagramma write

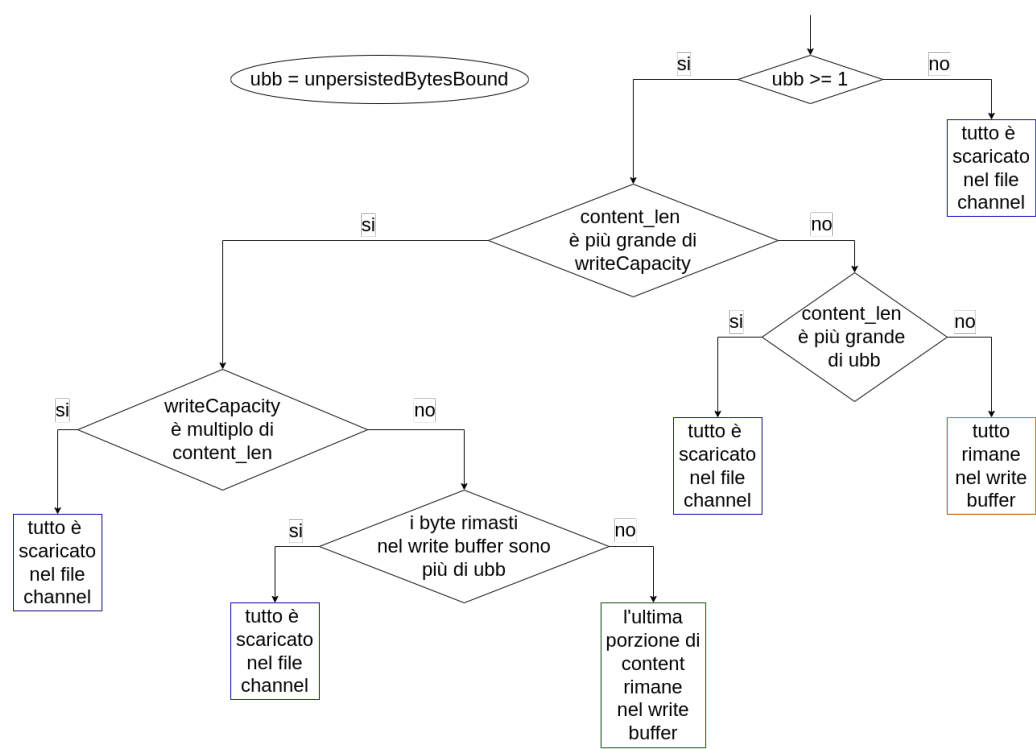


Tabella 4.13: Testing pitest write BufferedChannel

Test	allocator	fc	writeCapacity	readCapacity	unpersistedBytesBound	src
CL = content_len > 4, la dimensione dei dati scritti in src						
P-W1	valid	valid	CL-2	1	1	full
P-W2	valid	valid	CL-2	1	3	full

Tabella 4.14: Testing pitest write BufferedChannel

Test	Risultato atteso	Motivazione
P-W1	scrittura	La capacità del buffer di scrittura è più piccola di 2 byte rispetto i byte che vi vengono scritti, quindi si riempirà una volta e verrà scaricato nel file channel, la seconda volta verranno scritti nel buffer gli ultimi 2 byte, i quali verranno scaricati anch'essi nel file channel perchè superano l'upperbound al numero di byte non persistenti
P-W2	scrittura	La capacità del buffer di scrittura è più piccola di 2 byte rispetto i byte che vi vengono scritti, quindi si riempirà una volta e verrà scaricato nel file channel, la seconda volta verranno scritti nel buffer gli ultimi 2 byte, i quali non verranno scaricati però nel file channel poichè non superano l'upperbound al numero di byte non persistenti

Figura 4.10: Report di pitest per write (dopo)

```

117 public void write(ByteBuf src) throws IOException {
118     int copied = 0;
119     boolean shouldForceWrite = false;
120     synchronized (this) {
121         int len = src.readableBytes();
122         while (copied < len) {
123             int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124             writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125             copied += bytesToCopy;
126
127             // if we have run out of buffer space, we should flush to the
128             // file
129             if (!writeBuffer.isWritable()) {
130                 flush();
131             }
132         }
133         position += copied;
134         if (doRegularFlushes) {
135             unpersistedBytes.addAndGet(copied);
136             if (unpersistedBytes.get() >= unpersistedBytesBound) {
137                 flush();
138                 shouldForceWrite = true;
139             }
140         }
141     }
142     if (shouldForceWrite) {
143         forceWrite(false);
144     }
145 }

```

Figura 4.11: Report di pitest per read (prima)

```

245 public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246     long prevPos = pos;
247     while (length > 0) {
248         // check if it is in the write buffer
249         if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
250             int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
251             int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
252
253             if (bytesToCopy == 0) {
254                 throw new IOException("Read past EOF");
255             }
256
257             dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
258             pos += bytesToCopy;
259             length -= bytesToCopy;
260         } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
261             // here we reach the end
262             break;
263             // first check if there is anything we can grab from the readBuffer
264         } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
265             int positionInBuffer = (int) (pos - readBufferStartPosition);
266             int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
267             dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
268             pos += bytesToCopy;
269             length -= bytesToCopy;
270             // let's read it
271         } else {
272             readBufferStartPosition = pos;
273
274             int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
275                 readBufferStartPosition);
276             if (readBytes <= 0) {
277                 throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
278             }
279             readBuffer.writerIndex(readBytes);
280         }
281     }
282     return (int) (pos - prevPos);
283 }

```

Tabella 4.15: Test aggiunti a read dopo pitest

Test	stato istanza	stato writeBuffer	dest	pos	length
<i>fCL = fCContentLen</i>					
P-R1	valid with readCapacity=fCL/2	empty	fCL/2	1	1

Tabella 4.16: Test aggiunti a read dopo pitest

Test	Risultato atteso	Motivazione
P-R1	lettura byte fCL/2-esimo dal file channel	Ci si aspetta di leggere solo 1 byte perchè sarà l'ultimo del buffer di scrittura

Figura 4.12: Report di pitest per read (dopo)

```

264 5      } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
265 1          int positionInBuffer = (int) (pos - readBufferStartPosition);
266 1          int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
267          dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
268 1          pos += bytesToCopy;
269 1          length -= bytesToCopy;

```

Tabella 4.17: Testing costruttore WriteCache

Test	allocator	maxCacheSize	maxSegmentSize	Risultato atteso	Motivazione
<i>Variazioni ad allocator</i>					
T1	invalid	1	1	Exception	L'allocatore alloca buffer nulli
T2	null	1	1	Exception	L'allocatore è null
<i>Variazioni a maxCacheSize</i>					
T3	valid	-1	1	Exception	La size massima della cache è invalida
T4	valid	0	1	Exception	La size massima della cache non ha senso che sia zero
T5	valid	1	1	valid instance	La size massima della cache è valida
<i>Variazioni a maxSegmentSize</i>					
T6	valid	512	-1	Exception	La size massima dei segmenti è invalida
T7	valid	512	0	Exception	La size massima dei segmenti non ha senso che sia zero
T8	valid	512	1	valid instance	La size massima dei segmenti è valida
T9	valid	512	100	Exception	La size massima dei segmenti non è potenza di 2
T10	valid	512	128	valid instance	La size massima dei segmenti è valida
T11	valid	512	512	valid instance	La size massima dei segmenti è valida
T12	valid	512	600	Exception	La size massima dei segmenti è maggiore di maxCacheSize

Tabella 4.18: Testing put WriteCache

Test	stato istanza	ledgerId	entryId	entry	Risultato atteso	Motivazione
<i>Istanze del costruttore fallite</i>						
P1	maxCacheSize=0	1	1	non empty	false	La cache size è 0 perciò nulla può essere scritto
<i>Variazioni a ledgerId</i>						
P2	valid	-1	1	non empty	Exception	Il ledgerId non può essere negativo
P3	valid	0	1	non empty	true	Il ledgerId è valido
<i>Variazioni a entryId</i>						
P4	valid	1	-1	non empty	Exception	L'entryId non può essere negativo
P5	valid	1	0	non empty	true	L'entryId è valido
<i>Variazioni ad entry</i>						
P6	valid	1	1	invalid read index	Exception	L'entry ha un indice di lettura non valido, leggervi causerà un'eccezione
P7	valid	1	1	deallocated	Exception	L'entry è deallocata, leggervi causerà un'eccezione
P8	valid	1	1	null	Exception	L'entry è null, leggervi causerà un'eccezione
<i>Variazioni ad entry in funzione di maxSegmentSize</i>						
P9	valid	1	1	non empty size ≤ maxSegSize	true	I byte da scrivere entrano in un segmento, quindi ci si aspetta che la put vada a buon fine
P10	valid	1	1	non empty size > maxSegSize	false	I byte da scrivere non entrano in un segmento, quindi ci si aspetta che la put non vada a buon fine
P11	valid	1	1	empty	true	Vogliamo scrivere 0 byte quindi l'operazione andrà sicuramente a buon fine

Figura 4.13: Risultati jacoco costruttore

```

94. public WriteCache(ByteBufAllocator allocator, long maxCacheSize, int maxSegmentSize) {
95.     checkArgument(maxSegmentSize > 0);
96.
97.     long alignedMaxSegmentSize = alignToPowerOfTwo(maxSegmentSize);
98.     checkArgument(maxSegmentSize == alignedMaxSegmentSize, "Max segment size needs to be in form of 2^n");
99.
100.    this.allocator = allocator;
101.    this.maxCacheSize = maxCacheSize;
102.    this.maxSegmentSize = (int) maxSegmentSize;
103.    this.segmentOffsetMask = maxSegmentSize - 1;
104.    this.segmentOffsetBits = 63 - Long.numberOfLeadingZeros(maxSegmentSize);
105.
106.    this.segmentsCount = 1 + (int) (maxCacheSize / maxSegmentSize);
107.
108.    this.cacheSegments = new ByteBuf[segmentsCount];
109.
110.    for (int i = 0; i < segmentsCount - 1; i++) {
111.        // All intermediate segments will be full-size
112.        cacheSegments[i] = Unpooled.directBuffer(maxSegmentSize, maxSegmentSize);
113.    }
114.
115.    int lastSegmentSize = (int) (maxCacheSize % maxSegmentSize);
116.    cacheSegments[segmentsCount - 1] = Unpooled.directBuffer(lastSegmentSize, lastSegmentSize);
117. }

```

Figura 4.14: Risultati jacoco put (prima)

```

135. public boolean put(long ledgerId, long entryId, ByteBuf entry) {
136.     int size = entry.readableBytes();
137.
138.     // Align to 64 bytes so that different threads will not contend the same L1
139.     // cache line
140.     int alignedSize = align64(size);
141.
142.     long offset;
143.     int localOffset;
144.     int segmentIdx;
145.
146.     while (true) {
147.         offset = cacheOffset.getAndAdd(alignedSize);
148.         localOffset = (int) (offset & segmentOffsetMask);
149.         segmentIdx = (int) (offset >>> segmentOffsetBits);
150.
151.         if ((offset + size) > maxCacheSize) {
152.             // Cache is full
153.             return false;
154.         } else if (maxSegmentSize - localOffset < size) {
155.             // If an entry is at the end of a segment, we need to get a new offset and try
156.             // again in next segment
157.             continue;
158.         } else {
159.             // Found a good offset
160.             break;
161.         }
162.     }
163.
164.     cacheSegments[segmentIdx].setBytes(localOffset, entry, entry.readerIndex(), entry.readableBytes());
165.
166.     // Update last entryId for ledger. This logic is to handle writes for the same
167.     // ledger coming out of order and from different thread, though in practice it
168.     // should not happen and the compareAndSet should be always uncontended.
169.     while (true) {
170.         long currentLastEntryId = lastEntryMap.get(ledgerId);
171.         if (currentLastEntryId > entryId) {
172.             // A newer entry is already there
173.             break;
174.         }
175.
176.         if (lastEntryMap.compareAndSet(ledgerId, currentLastEntryId, entryId)) {
177.             break;
178.         }
179.     }
180.
181.     index.put(ledgerId, entryId, offset, size);
182.     cacheCount.increment();
183.     cacheSize.addAndGet(size);
184.     return true;
185. }

```

Tabella 4.19: Testing jacoco put WriteCache

Test	stato istanza	ledgerId	entryId	entry	Risultato atteso	Motivazione
Nel ledgerId 1 è stata inserita un entry con id 2						
P1	valid, non empty	1	1	non empty size ≤ maxSegSize	true	La configurazione è valida, ma la lastEntryMap non verrà aggiornata
P2	valid, non empty	1	3	non empty size ≤ maxSegSize	true	La configurazione è valida, inoltre la lastEntryMap verrà aggiornata

Figura 4.15: Risultati jacoco put (dopo)

```

135.     public boolean put(long ledgerId, long entryId, ByteBuf entry) {
136.         int size = entry.readableBytes();
137.
138.         // Align to 64 bytes so that different threads will not contend the same L1
139.         // cache line
140.         int alignedSize = align64(size);
141.
142.         long offset;
143.         int localOffset;
144.         int segmentIdx;
145.
146.         while (true) {
147.             offset = cacheOffset.getAndAdd(alignedSize);
148.             localOffset = (int) (offset & segmentOffsetMask);
149.             segmentIdx = (int) (offset >>> segmentOffsetBits);
150.
151.             if ((offset + size) > maxCacheSize) {
152.                 // Cache is full
153.                 return false;
154.             } else if (maxSegmentSize - localOffset < size) {
155.                 // If an entry is at the end of a segment, we need to get a new offset and try
156.                 // again in next segment
157.                 continue;
158.             } else {
159.                 // Found a good offset
160.                 break;
161.             }
162.         }
163.
164.         cacheSegments[segmentIdx].setBytes(localOffset, entry, entry.readerIndex(), entry.readableBytes());
165.
166.         // Update last entryId for ledger. This logic is to handle writes for the same
167.         // ledger coming out of order and from different thread, though in practice it
168.         // should not happen and the compareAndSet should be always uncontended.
169.         while (true) {
170.             long currentLastEntryId = lastEntryMap.get(ledgerId);
171.             if (currentLastEntryId > entryId) {
172.                 // A newer entry is already there
173.                 break;
174.             }
175.
176.             if (lastEntryMap.compareAndSet(ledgerId, currentLastEntryId, entryId)) {
177.                 break;
178.             }
179.         }
180.
181.         index.put(ledgerId, entryId, offset, size);
182.         cacheCount.increment();
183.         cacheSize.addAndGet(size);
184.         return true;
185.     }

```

Codice 4.9: Risultati Badua per la put

```

<method name="put" desc="(J)Lio/netty/buffer/ByteBuf;Z">
  <du var="this" def="136" use="147" covered="1"/>
  <du var="this" def="136" use="148" covered="1"/>
  <du var="this" def="136" use="149" covered="1"/>
  <du var="this" def="136" use="151" target="153" covered="1"/>
  <du var="this" def="136" use="151" target="154" covered="1"/>
  <du var="this" def="136" use="154" target="157" covered="1"/>
  <du var="this" def="136" use="154" target="164" covered="1"/>
  <du var="this" def="136" use="164" covered="1"/>
  <du var="this" def="136" use="170" covered="1"/>
  <du var="this" def="136" use="176" target="177" covered="1"/>
  <du var="this" def="136" use="176" target="179" covered="0"/>
  <du var="this" def="136" use="181" covered="1"/>
  <du var="this" def="136" use="182" covered="1"/>
  <du var="this" def="136" use="183" covered="1"/>
  <du var="ledgerId" def="136" use="170" covered="1"/>
  <du var="ledgerId" def="136" use="176" target="177" covered="1"/>
  <du var="ledgerId" def="136" use="176" target="179" covered="0"/>
  <du var="ledgerId" def="136" use="181" covered="1"/>
  <du var="entryId" def="136" use="171" target="173" covered="1"/>

```

```

<du var="entryId" def="136" use="171" target="176" covered="1"/>
<du var="entryId" def="136" use="176" target="177" covered="1"/>
<du var="entryId" def="136" use="176" target="179" covered="0"/>
<du var="entryId" def="136" use="181" covered="1"/>
<du var="entry" def="136" use="164" covered="1"/>
<du var="this.cacheOffset" def="136" use="147" covered="1"/>
<du var="this.segmentOffsetMask" def="136" use="148" covered="1"/>
<du var="this.segmentOffsetBits" def="136" use="149" covered="1"/>
<du var="this.maxCacheSize" def="136" use="151" target="153" covered="1"/>
<du var="this.maxCacheSize" def="136" use="151" target="154" covered="1"/>
<du var="this.maxSegmentSize" def="136" use="154" target="157" covered="1"/>
<du var="this.maxSegmentSize" def="136" use="154" target="164" covered="1"/>
<du var="this.cacheSegments" def="136" use="164" covered="1"/>
<du var="this.lastEntryMap" def="136" use="170" covered="1"/>
<du var="this.lastEntryMap" def="136" use="176" target="177" covered="1"/>
<du var="this.lastEntryMap" def="136" use="176" target="179" covered="0"/>
<du var="this.index" def="136" use="181" covered="1"/>
<du var="this.cacheCount" def="136" use="182" covered="1"/>
<du var="this.cacheSize" def="136" use="183" covered="1"/>
<du var="size" def="136" use="151" target="153" covered="1"/>
<du var="size" def="136" use="151" target="154" covered="1"/>
<du var="size" def="136" use="154" target="157" covered="1"/>
<du var="size" def="136" use="154" target="164" covered="1"/>
<du var="size" def="136" use="181" covered="1"/>
<du var="size" def="136" use="183" covered="1"/>
<du var="alignedSize" def="140" use="147" covered="1"/>
<du var="offset" def="147" use="151" target="153" covered="1"/>
<du var="offset" def="147" use="151" target="154" covered="1"/>
<du var="offset" def="147" use="181" covered="1"/>
<du var="localOffset" def="148" use="154" target="157" covered="1"/>
<du var="localOffset" def="148" use="154" target="164" covered="1"/>
<du var="localOffset" def="148" use="164" covered="1"/>
<du var="segmentIdx" def="149" use="164" covered="1"/>
<du var="currentLastEntryId" def="170" use="171" target="173" covered="1"/>
<du var="currentLastEntryId" def="170" use="171" target="176" covered="1"/>
<du var="currentLastEntryId" def="170" use="176" target="177" covered="1"/>
<du var="currentLastEntryId" def="170" use="176" target="179" covered="0"/>
<counter type="DU" missed="5" covered="51"/>
<counter type="METHOD" missed="0" covered="1"/>
</method>

```

Figura 4.16: Risultati pitest costruttore (prima)

```

94 public WriteCache(ByteBufAllocator allocator, long maxCacheSize, int maxSegmentSize) {
95     checkArgument(maxSegmentSize > 0);
96
97     long alignedMaxSegmentSize = alignToPowerOfTwo(maxSegmentSize);
98     checkArgument(maxSegmentSize == alignedMaxSegmentSize, "Max segment size needs to be in form of 2^n");
99
100     this.allocator = allocator;
101     this.maxCacheSize = maxCacheSize;
102     this.maxSegmentSize = (int) maxSegmentSize;
103     this.segmentOffsetMask = maxSegmentSize - 1;
104     this.segmentOffsetBits = 63 - Long.numberOfLeadingZeros(maxSegmentSize);
105
106     this.segmentsCount = 1 + (int) (maxCacheSize / maxSegmentSize);
107
108     this.cacheSegments = new ByteBuf[segmentsCount];
109
110     for (int i = 0; i < segmentsCount - 1; i++) {
111         // All intermediate segments will be full-size
112         cacheSegments[i] = Unpooled.directBuffer(maxSegmentSize, maxSegmentSize);
113     }
114
115     int lastSegmentSize = (int) (maxCacheSize % maxSegmentSize);
116     cacheSegments[segmentsCount - 1] = Unpooled.directBuffer(lastSegmentSize, lastSegmentSize);
117 }

```

Figura 4.17: Risultati pitest costruttore (dopo)

```

94 public WriteCache(ByteBufAllocator allocator, long maxCacheSize, int maxSegmentSize) {
95     checkArgument(maxSegmentSize > 0);
96
97     long alignedMaxSegmentSize = alignToPowerOfTwo(maxSegmentSize);
98     checkArgument(maxSegmentSize == alignedMaxSegmentSize, "Max segment size needs to be in form of 2^n");
99
100     this.allocator = allocator;
101     this.maxCacheSize = maxCacheSize;
102     this.maxSegmentSize = (int) maxSegmentSize;
103     this.segmentOffsetMask = maxSegmentSize - 1;
104     this.segmentOffsetBits = 63 - Long.numberOfLeadingZeros(maxSegmentSize);
105
106     this.segmentsCount = 1 + (int) (maxCacheSize / maxSegmentSize);
107
108     this.cacheSegments = new ByteBuf[segmentsCount];
109
110     for (int i = 0; i < segmentsCount - 1; i++) {
111         // All intermediate segments will be full-size
112         cacheSegments[i] = Unpooled.directBuffer(maxSegmentSize, maxSegmentSize);
113     }
114
115     int lastSegmentSize = (int) (maxCacheSize % maxSegmentSize);
116     cacheSegments[segmentsCount - 1] = Unpooled.directBuffer(lastSegmentSize, lastSegmentSize);
117 }

```

Figura 4.18: Risultati ptest put (prima)

```

135 public boolean put(long ledgerId, long entryId, ByteBuf entry) {
136     int size = entry.readableBytes();
137
138     // Align to 64 bytes so that different threads will not contend the same L1
139     // cache line
140     int alignedSize = align64(size);
141
142     long offset;
143     int localOffset;
144     int segmentIdx;
145
146     while (true) {
147         offset = cacheOffset.getAndAdd(alignedSize);
148         localOffset = (int) (offset & segmentOffsetMask);
149         segmentIdx = (int) (offset >>> segmentOffsetBits);
150
151         if ((offset + size) > maxCacheSize) {
152             // Cache is full
153             return false;
154         } else if (maxSegmentSize - localOffset < size) {
155             // If an entry is at the end of a segment, we need to get a new offset and try
156             // again in next segment
157             continue;
158         } else {
159             // Found a good offset
160             break;
161         }
162     }
163
164     cacheSegments[segmentIdx].setBytes(localOffset, entry, entry.readerIndex(), entry.readableBytes());
165
166     // Update last entryId for ledger. This logic is to handle writes for the same
167     // ledger coming out of order and from different thread, though in practice it
168     // should not happen and the compareAndSet should be always uncontended.
169     while (true) {
170         long currentLastEntryId = lastEntryMap.get(ledgerId);
171         if (currentLastEntryId > entryId) {
172             // A newer entry is already there
173             break;
174         }
175
176         if (lastEntryMap.compareAndSet(ledgerId, currentLastEntryId, entryId)) {
177             break;
178         }
179     }
180
181     index.put(ledgerId, entryId, offset, size);
182     cacheCount.increment();
183     cacheSize.addAndGet(size);
184     return true;
185 }

```

Tabella 4.20: Testing ptest put WriteCache

Test	stato istanza	ledgerId	entryId	entry	Risultato atteso	Motivazione
<i>Cache vuota con un solo segmento</i>						
P-P1	valid, empty maxSegSize= 2*maxCacheSize	1	1	non empty size=maxSegSize	false	La cache è troppo piccola per l'entry, l'inserimento non andrà a buon fine
<i>È già stata inserita una entry di dimensione pari a maxSegSize</i>						
P-P2	valid, 1 seg written maxSegSize=128 maxCacheSize=256	1	1	non empty size=maxSegSize	true	La configurazione è valida, e c'è lo spazio per aggiungere entry
<i>È già stata inserita una entry di dimensione pari a maxSegSize/2</i>						
P-P3	valid, half seg written, maxSegSize=128 maxCacheSize=256	1	1	non empty size=maxSegSize	true	vedi Sotto-sezione 2.2.6
P-P4	valid, half seg written, maxSegSize=128 maxCacheSize=256	1	1	non empty size=maxSegSize	false	vedi Sotto-sezione 2.2.6

Figura 4.19: Risultati pitest put (dopo)

```
135 public boolean put(long ledgerId, long entryId, ByteBuf entry) {
136     int size = entry.readableBytes();
137
138     // Align to 64 bytes so that different threads will not contend the same L1
139     // cache line
140     int alignedSize = align64(size);
141
142     long offset;
143     int localOffset;
144     int segmentIdx;
145
146     while (true) {
147         offset = cacheOffset.getAndAdd(alignedSize);
148         localOffset = (int) (offset & segmentOffsetMask);
149         segmentIdx = (int) (offset >>> segmentOffsetBits);
150
151         if ((offset + size) > maxCacheSize) {
152             // Cache is full
153             return false;
154         } else if (maxSegmentSize - localOffset < size) {
155             // If an entry is at the end of a segment, we need to get a new offset and try
156             // again in next segment
157             continue;
158         } else {
159             // Found a good offset
160             break;
161         }
162     }
163
164     cacheSegments[segmentIdx].setBytes(localOffset, entry, entry.readerIndex(), entry.readableBytes());
165
166     // Update last entryId for ledger. This logic is to handle writes for the same
167     // ledger coming out of order and from different thread, though in practice it
168     // should not happen and the compareAndSet should be always uncontended.
169     while (true) {
170         long currentLastEntryId = lastEntryMap.get(ledgerId);
171         if (currentLastEntryId > entryId) {
172             // A newer entry is already there
173             break;
174         }
175
176         if (lastEntryMap.compareAndSet(ledgerId, currentLastEntryId, entryId)) {
177             break;
178         }
179     }
180
181     index.put(ledgerId, entryId, offset, size);
182     cacheCount.increment();
183     cacheSize.addAndGet(size);
184     return true;
185 }
```

Figura 4.20: ORD tra SingleDirectoryDbLedgerStorage e WriteCache

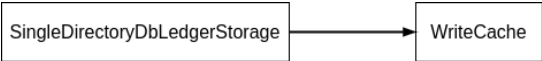


Figura 4.21: Utilizzo Mockito per IT

```
doReturn(true).when(currentWriteCache).put(anyLong(), anyLong(), any());
storage.addEntry(entry);
verify(currentWriteCache, times(1)).put(1, 2, entry);
```

Tabella 4.21: IT SingleDirectoryDbLedgerStorage

Test	Spiegazione
testWriteCacheStubbed1	Si crea uno stub di writeCache che ritorna true alla prima chiamata, di conseguenza si verifica che il metodo put venga chiamato solo una volta.
testWriteCacheStubbed2	Si crea uno stub di writeCache che ritorna false alla prima chiamata, di conseguenza si verifica che il metodo put venga chiamato due, la seconda per il flushing.
testWriteCacheRotation	Si controlla che la cache venga ruotata, e di conseguenza la put della entry che provoca la rotazione deve avvenire almeno due volte.

Figura 4.22: Sostituto del mock di names

```
1 package org.apache.avro;
2
3 public class InvalidSchemaNames extends Schema.Names{
4     @Override 5 usages AlessandroFinocchi
5     public String space() {
6         throw new NullPointerException();
7     }
8
9     @Override 2 usages AlessandroFinocchi
10    public void space(String space) {
11        throw new NullPointerException();
12    }
13
14    @Override AlessandroFinocchi
15    public Schema get(String o) {
16        throw new NullPointerException();
17    }
18
19    @Override AlessandroFinocchi
```

Tabella 4.22: Testing parse Schema

schema	names	Risultato atteso	Motivazione
Variazioni a schema			
valid	valid	valid instance	Il json rispetta le specifiche, quindi ci aspettiamo un'istanza di schema corretta
without mandatory field	valid	Exception	Nel nodo json manca un campo obbligatorio, quindi ci aspettiamo un errore
invalid mandatory field	valid	Exception	Nel nodo json un campo ha un valore non supportato dalle specifiche, quindi ci aspettiamo un errore
null	valid	Exception	Il nodo json è null quindi il suo parsing non deve andare a buon fine
Variazioni a names			
valid	invalid	Exception	La mappa di namespace restituisce sempre un eccezione, quindi ci aspettiamo un errore
valid	null	Exception	La mappa di namespace è null, quindi ci aspettiamo un errore

Figura 4.23: Risultati jacoco parse

```
2101. static Schema parse(JsonNode schema, Names names) {
2102.     if (schema == null) {
2103.         throw new SchemaParseException("Cannot parse <null> schema");
2104.     }
2105.
2106.     Schema result = Schema.parseNamesDeclared(schema, names, names.space);
2107.     Schema.parseCompleteSchema(schema, names, names.space);
2108.
2109.     return result;
2110. }
```

Figura 4.24: Risultati 1 jacoco parseNamesDeclared (prima)

```

1879. static Schema parseNamesDeclared(JsonNode schema, Names names, String currentNameSpace) {
1880.     ◆ if (schema == null) {
1881.         return null;
1882.     }
1883.     ◆ if (schema.isObject()) {
1884.
1885.         String type = Schema.getOptionalText(schema, "type");
1886.         Name name = null;
1887.
1888.         String doc = null;
1889.         Schema result = null;
1890.         final boolean isTypeError = "error".equals(type);
1891.         final boolean isTypeRecord = "record".equals(type);
1892.         final boolean isTypeEnum = "enum".equals(type);
1893.         final boolean isTypeFixed = "fixed".equals(type);
1894.
1895.         ◆ if (isTypeRecord || isTypeError || isTypeEnum || isTypeFixed) {
1896.             String space = getOptionalText(schema, "namespace");
1897.             doc = getOptionalText(schema, "doc");
1898.             ◆ if (space == null)
1899.                 space = currentNameSpace;
1900.             name = new Name(getRequiredText(schema, "name", "No name in schema"), space);
1901.         }
1902.         ◆ if (isTypeRecord || isTypeError) { // record
1903.             result = new RecordSchema(name, doc, isTypeError);
1904.             names.add(result);
1905.             JsonNode fieldsNode = schema.get("fields");
1906.
1907.             ◆ if (fieldsNode == null || !fieldsNode.isArray())
1908.                 throw new SchemaParseException("Record has no fields: " + schema);
1909.             ◆ exploreFields(fieldsNode, names, name != null ? name.space : null);
1910.
1911.             ◆ } else if (isTypeEnum) { // enum
1912.                 JsonNode symbolsNode = schema.get("symbols");
1913.                 ◆ if (symbolsNode == null || !symbolsNode.isArray())
1914.                     throw new SchemaParseException("Enum has no symbols: " + schema);
1915.                 LockableArrayList<String> symbols = new LockableArrayList<>(symbolsNode.size());
1916.                 ◆ for (JsonNode n : symbolsNode)
1917.                     symbols.add(n.textValue());
1918.                 JsonNode enumDefault = schema.get("default");
1919.                 String defaultSymbol = null;
1920.                 ◆ if (enumDefault != null)
1921.                     defaultSymbol = enumDefault.textValue();
1922.                 result = new EnumSchema(name, doc, symbols, defaultSymbol);
1923.                 names.add(result);
1924.             ◆ } else if (type.equals("array")) { // array
1925.                 JsonNode itemsNode = schema.get("items");
1926.                 ◆ if (itemsNode == null)
1927.                     throw new SchemaParseException("Array has no items type: " + schema);
1928.                 final Schema items = Schema.parseNamesDeclared(itemsNode, names, currentNameSpace);
1929.                 result = Schema.createArray(items);
1930.             ◆ } else if (type.equals("map")) { // map
1931.                 JsonNode valuesNode = schema.get("values");
1932.                 ◆ if (valuesNode == null)
1933.                     throw new SchemaParseException("Map has no values type: " + schema);
1934.                 final Schema values = Schema.parseNamesDeclared(valuesNode, names, currentNameSpace);

```

Figura 4.25: Risultati 2 jacoco parseNamesDeclared (prima)

```

1932.  if (valuesNode == null)
1933.      throw new SchemaParseException("Map has no values type: " + schema);
1934.      final Schema values = Schema.parseNamesDeclared(valuesNode, names, currentNamespace);
1935.      result = Schema.createMap(values);
1936.  } else if (isTypeFixed) { // fixed
1937.      JsonNode sizeNode = schema.get("size");
1938.      if (sizeNode == null || !sizeNode.isInt())
1939.          throw new SchemaParseException("Invalid or no size: " + schema);
1940.      result = new FixedSchema(name, doc, sizeNode.intValue());
1941.      if (name != null)
1942.          names.add(result);
1943.      } else if (PRIMITIVES.containsKey(type)) {
1944.          result = Schema.create(PRIMITIVES.get(type));
1945.      }
1946.      if (result != null) {
1947.          Set<String> reserved = SCHEMA_RESERVED;
1948.          if (isTypeEnum) {
1949.              reserved = ENUM_RESERVED;
1950.          }
1951.          Schema.addProperties(schema, reserved, result);
1952.      }
1953.      return result;
1954.  } else if (schema.isArray()) {
1955.      List<Schema> subs = new ArrayList<>(schema.size());
1956.      schema.forEach((JsonNode item) -> {
1957.          Schema sub = Schema.parseNamesDeclared(item, names, currentNamespace);
1958.          if (sub != null) {
1959.              subs.add(sub);
1960.          }
1961.      });
1962.      return Schema.createUnion(subs);
1963.  } else if (schema.isTextual()) {
1964.      String value = schema.asText();
1965.      return names.get(value);
1966.  }
1967.  return null;
1968.  }

```

Figura 4.26: Metodo getRequiredText di Schema

```

/**
 * Extracts text value associated to key from the container JsonNode, and throws
 * {@link SchemaParseException} if it doesn't exist.
 *
 * @param container Container where to find key.
 * @param key Key to look for in container.
 * @param error String to prepend to the SchemaParseException.
 */
private static String getRequiredText(JsonNode container, String key, String error) {
    String out = getOptionalText(container, key);
    if (null == out) {
        throw new SchemaParseException(error + "Key '" + key + "' not found in container");
    }
    return out;
}

```

Tabella 4.23: Testing jacoco parseNamesDeclared Schema

schema	names	Risultato atteso	Motivazione
ERROR	valid	valid instance	Il json rispetta le specifiche, quindi ci aspettiamo un'istanza di schema corretta
NO_FIELD_RECORD	valid	Exception	Il record deve avere un campo field, perciò ci si aspetta un errore
NO_ARRAY_FIELD_RECORD	valid	Exception	Il campo field di un record deve essere un array, perciò ci si aspetta un errore
NO_SYMBOLS_ENUM	valid	Exception	Una enum deve avere almeno un simbolo, perciò ci si aspetta un errore
NO_ARRAY_SYMBOLS_ENUM	valid	Exception	I simboli delle enum devono essere dentro un array, perciò ci si aspetta un errore
DEFAULT_VALUE_ENUM	valid	valid instance	Il json rispetta le specifiche, quindi ci aspettiamo un'istanza di schema corretta
NO_ITEMS_ARRAY	valid	Exception	Un array deve avere il campo item, perciò ci si aspetta un errore
NO_VALUES_MAP	valid	Exception	Una mappa deve avere un campo values, perciò ci si aspetta un errore
NO_SIZE_FIXED	valid	Exception	Uno schema fixed deve avere un campo size, perciò ci si aspetta un errore
NO_INT_SIZE_FIXED	valid	Exception	Il campo size di un fixed deve essere un'intero, perciò ci si aspetta un errore
NON_TEXTUAL	valid	Exception	Lo schema che si vuole parsare non è testuale, perciò ci si aspetta un errore

Figura 4.27: Risultati 1 jacoco parseNamesDeclared (dopo)

```

1879. static Schema parseNamesDeclared(JsonNode schema, Names names, String currentNameSpace) {
1880.     if (schema == null) {
1881.         return null;
1882.     }
1883.     if (schema.isObject()) {
1884.
1885.         String type = Schema.getOptionalText(schema, "type");
1886.         Name name = null;
1887.
1888.         String doc = null;
1889.         Schema result = null;
1890.         final boolean isTypeError = "error".equals(type);
1891.         final boolean isTypeRecord = "record".equals(type);
1892.         final boolean isTypeEnum = "enum".equals(type);
1893.         final boolean isTypeFixed = "fixed".equals(type);
1894.
1895.         if (isTypeRecord || isTypeError || isTypeEnum || isTypeFixed) {
1896.             String space = getOptionalText(schema, "namespace");
1897.             doc = getOptionalText(schema, "doc");
1898.             if (space == null)
1899.                 space = currentNameSpace;
1900.             name = new Name(getRequiredText(schema, "name", "No name in schema"), space);
1901.         }
1902.         if (isTypeRecord || isTypeError) { // record
1903.             result = new RecordSchema(name, doc, isTypeError);
1904.             names.add(result);
1905.             JsonNode fieldsNode = schema.get("fields");
1906.
1907.             if (fieldsNode == null || !fieldsNode.isArray())
1908.                 throw new SchemaParseException("Record has no fields: " + schema);
1909.             exploreFields(fieldsNode, names, name != null ? name.space : null);
1910.
1911.         } else if (isTypeEnum) { // enum
1912.             JsonNode symbolsNode = schema.get("symbols");
1913.             if (symbolsNode == null || !symbolsNode.isArray())
1914.                 throw new SchemaParseException("Enum has no symbols: " + schema);
1915.             LockableArrayList<String> symbols = new LockableArrayList<>(symbolsNode.size());
1916.             for (JsonNode n : symbolsNode)
1917.                 symbols.add(n.textValue());
1918.             JsonNode enumDefault = schema.get("default");
1919.             String defaultSymbol = null;
1920.             if (enumDefault != null)
1921.                 defaultSymbol = enumDefault.textValue();
1922.             result = new EnumSchema(name, doc, symbols, defaultSymbol);
1923.             names.add(result);
1924.         } else if (type.equals("array")) { // array
1925.             JsonNode itemsNode = schema.get("items");
1926.             if (itemsNode == null)
1927.                 throw new SchemaParseException("Array has no items type: " + schema);
1928.             final Schema items = Schema.parseNamesDeclared(itemsNode, names, currentNameSpace);
1929.             result = Schema.createArray(items);
1930.         } else if (type.equals("map")) { // map
1931.             JsonNode valuesNode = schema.get("values");
1932.             if (valuesNode == null)
1933.                 throw new SchemaParseException("Map has no values type: " + schema);
1934.             final Schema values = Schema.parseNamesDeclared(valuesNode, names, currentNameSpace);

```

Figura 4.28: Risultati 2 jacoco parseNamesDeclared (dopo)

```

1932.  ◆   if (valuesNode == null)
1933.      throw new SchemaParseException("Map has no values type: " + schema);
1934.      final Schema values = Schema.parseNamesDeclared(valuesNode, names, currentNamespace);
1935.      result = Schema.createMap(values);
1936.  ◆   } else if (isTypeFixed) { // fixed
1937.      JsonNode sizeNode = schema.get("size");
1938.  ◆   if (sizeNode == null || !sizeNode.isInt())
1939.      throw new SchemaParseException("Invalid or no size: " + schema);
1940.      result = new FixedSchema(name, doc, sizeNode.intValue());
1941.  ◆   if (name != null)
1942.      names.add(result);
1943.  ◆   } else if (PRIMITIVES.containsKey(type)) {
1944.      result = Schema.create(PRIMITIVES.get(type));
1945.  }
1946.  ◆   if (result != null) {
1947.      Set<String> reserved = SCHEMA_RESERVED;
1948.  ◆   if (isTypeEnum) {
1949.      reserved = ENUM_RESERVED;
1950.  }
1951.      Schema.addProperty(schema, reserved, result);
1952.  }
1953.      return result;
1954.  ◆   } else if (schema.isArray()) {
1955.      List<Schema> subs = new ArrayList<>(schema.size());
1956.      schema.forEach((JsonNode item) -> {
1957.          Schema sub = Schema.parseNamesDeclared(item, names, currentNamespace);
1958.  ◆   if (sub != null) {
1959.      subs.add(sub);
1960.  }
1961.      });
1962.      return Schema.createUnion(subs);
1963.  ◆   } else if (schema.isTextual()) {
1964.      String value = schema.asText();
1965.      return names.get(value);
1966.  }
1967.      return null;
1968.  }

```

Figura 4.29: Risultati 1 jacoco parseCompleteSchema

```

2019.  static Schema parseCompleteSchema(JsonNode schema, Names names, String currentSpace) {
2020.  ◆   if (schema == null) {
2021.      throw new SchemaParseException("Cannot parse <null> schema");
2022.  }
2023.  ◆   if (schema.isTextual()) {
2024.      String type = schema.asText();
2025.      Schema avroSchema = names.get(type);
2026.  ◆   if (avroSchema == null) {
2027.      avroSchema = names.get(currentSpace + "." + type);
2028.  }
2029.      return avroSchema;
2030.  }
2031.  ◆   if (schema.isArray()) {
2032.      List<Schema> schemas = StreamSupport.stream(schema.spliterator(), false)
2033.          .map((JsonNode sub) -> parseCompleteSchema(sub, names, currentSpace)).collect(Collectors.toList());
2034.      return Schema.createUnion(schemas);
2035.  }
2036.  ◆   if (schema.isObject()) {
2037.      Schema result = null;
2038.      String type = getRequiredText(schema, "type", "No type");
2039.      Name name = null;
2040.
2041.      final boolean isTypeError = "error".equals(type);
2042.      final boolean isTypeRecord = "record".equals(type);
2043.      final boolean isTypeArray = "array".equals(type);
2044.
2045.  ◆   if (isTypeRecord || isTypeError || "enum".equals(type) || "fixed".equals(type)) {
2046.      // named schema
2047.      String space = getOptionalText(schema, "namespace");

```

Figura 4.30: Risultati 2 jacoco parseCompleteSchema

```

2048.
2049.  ◆ if (space == null)
2050.      space = currentSpace;
2051.      name = new Name(getRequiredText(schema, "name", "No name in schema"), space);
2052.
2053.      result = names.get(name);
2054.  ◆ if (result == null) {
2055.      throw new SchemaParseException("Unparsed field type " + name);
2056.  }
2057.
2058.  ◆ if (isTypeRecord || isTypeError) {
2059.  ◆ if (result != null && !result.hasFields()) {
2060.      final List<Field> fields = new ArrayList<>();
2061.      JsonNode fieldsNode = schema.get("fields");
2062.  ◆ if (fieldsNode == null || !fieldsNode.isArray())
2063.      throw new SchemaParseException("Record has no fields: " + schema);
2064.
2065.  ◆ for (JsonNode field : fieldsNode) {
2066.      Field f = Field.parse(field, names, name.space);
2067.
2068.      fields.add(f);
2069.  ◆ if (f.schema.getLogicalType() == null && getOptionalText(field, LOGICAL_TYPE_PROP) != null)
2070.      LOG.warn(
2071.          "Ignored the {}.{}.logicalType property (\"{}{}\"). It should probably be nested inside
2072.          name, f.name, getOptionalText(field, "logicalType");
2073.      )
2074.      result.setFields(fields);
2075.  }
2076.  ◆ } else if (isTypeArray) {
2077.      JsonNode items = schema.get("items");
2078.      Schema schemaItems = parseCompleteSchema(items, names, currentSpace);
2079.      result = Schema.createArray(schemaItems);
2080.  ◆ } else if ("map".equals(type)) {
2081.      JsonNode values = schema.get("values");
2082.      Schema mapItems = parseCompleteSchema(values, names, currentSpace);
2083.      result = Schema.createMap(mapItems);
2084.  ◆ } else if (result == null) {
2085.      result = names.get(currentSpace + "." + type);
2086.  ◆ if (result == null) {
2087.      result = names.get(type);
2088.  }
2089.  }
2090.
2091.      Set<String> reserved = SCHEMA_RESERVED;
2092.  ◆ if ("enum".equals(type)) {
2093.      reserved = ENUM_RESERVED;
2094.  }
2095.      Schema.addProperties(schema, reserved, result);
2096.      return result;
2097.  }
2098.  return null;
2099.  }

```

Codice 4.10: Risultati Badua parseNamesDeclared

```

<method name="parseNamesDeclared"
  desc="(Lcom/fasterxml/jackson/databind/JsonNode;Lorg/apache/avro/Schema$Names;Ljava/lang/String;)Lorg/apache/avro/Schema;">
  <du var="schema" def="1880" use="1880" target="1881" covered="0"/>
  <du var="schema" def="1880" use="1880" target="1883" covered="1"/>
  <du var="schema" def="1880" use="1883" target="1885" covered="1"/>
  <du var="schema" def="1880" use="1883" target="1954" covered="1"/>
  <du var="schema" def="1880" use="1954" target="1955" covered="1"/>
  <du var="schema" def="1880" use="1954" target="1963" covered="1"/>
  <du var="schema" def="1880" use="1963" target="1964" covered="1"/>
  <du var="schema" def="1880" use="1963" target="1967" covered="1"/>
  <du var="schema" def="1880" use="1964" covered="1"/>
  <du var="schema" def="1880" use="1955" covered="1"/>
  <du var="schema" def="1880" use="1956" covered="1"/>
  <du var="schema" def="1880" use="1885" covered="1"/>
  <du var="schema" def="1880" use="1896" covered="1"/>
  <du var="schema" def="1880" use="1897" covered="1"/>
  <du var="schema" def="1880" use="1900" covered="1"/>
  <du var="schema" def="1880" use="1905" covered="1"/>
  <du var="schema" def="1880" use="1908" covered="1"/>
  <du var="schema" def="1880" use="1951" covered="1"/>
  <du var="schema" def="1880" use="1937" covered="1"/>
  <du var="schema" def="1880" use="1939" covered="1"/>
  <du var="schema" def="1880" use="1931" covered="1"/>

```



```
<du var="schema" def="1880" use="1933" covered="1"/>
<du var="schema" def="1880" use="1925" covered="1"/>
<du var="schema" def="1880" use="1927" covered="1"/>
<du var="schema" def="1880" use="1912" covered="1"/>
<du var="schema" def="1880" use="1914" covered="1"/>
<du var="schema" def="1880" use="1918" covered="1"/>
<du var="names" def="1880" use="1965" covered="1"/>
<du var="names" def="1880" use="1956" covered="1"/>
<du var="names" def="1880" use="1904" covered="1"/>
<du var="names" def="1880" use="1909" covered="1"/>
<du var="names" def="1880" use="1942" covered="1"/>
<du var="names" def="1880" use="1934" covered="1"/>
<du var="names" def="1880" use="1928" covered="1"/>
<du var="names" def="1880" use="1923" covered="1"/>
<du var="currentNameSpace" def="1880" use="1956" covered="1"/>
<du var="currentNameSpace" def="1880" use="1934" covered="1"/>
<du var="currentNameSpace" def="1880" use="1928" covered="1"/>
<du var="currentNameSpace" def="1880" use="1899" covered="1"/>
<du var="PRIMITIVES" def="1880" use="1943" target="1944" covered="1"/>
<du var="PRIMITIVES" def="1880" use="1943" target="1946" covered="1"/>
<du var="PRIMITIVES" def="1880" use="1944" covered="1"/>
<du var="SCHEMA_RESERVED" def="1880" use="1947" covered="1"/>
<du var="ENUM_RESERVED" def="1880" use="1949" covered="1"/>
<du var="type" def="1885" use="1924" target="1925" covered="1"/>
<du var="type" def="1885" use="1924" target="1930" covered="1"/>
<du var="type" def="1885" use="1930" target="1931" covered="1"/>
<du var="type" def="1885" use="1930" target="1936" covered="1"/>
<du var="type" def="1885" use="1943" target="1944" covered="1"/>
<du var="type" def="1885" use="1943" target="1946" covered="1"/>
<du var="type" def="1885" use="1944" covered="1"/>
<du var="name" def="1886" use="1903" covered="0"/>
<du var="name" def="1886" use="1909" target="1909" covered="0"/>
<du var="name" def="1886" use="1909" target="1909" covered="0"/>
<du var="name" def="1886" use="1909" covered="0"/>
<du var="name" def="1886" use="1940" covered="0"/>
<du var="name" def="1886" use="1941" target="1942" covered="0"/>
<du var="name" def="1886" use="1941" target="1943" covered="0"/>
<du var="name" def="1886" use="1922" covered="0"/>
<du var="doc" def="1888" use="1903" covered="0"/>
<du var="doc" def="1888" use="1940" covered="0"/>
<du var="doc" def="1888" use="1922" covered="0"/>
<du var="result" def="1889" use="1946" target="1947" covered="0"/>
<du var="result" def="1889" use="1946" target="1953" covered="1"/>
<du var="result" def="1889" use="1953" covered="1"/>
<du var="result" def="1889" use="1951" covered="0"/>
<du var="isTypeError" def="1890" use="1903" covered="1"/>
<du var="isTypeError" def="1890" use="1902" target="1903" covered="1"/>
<du var="isTypeError" def="1890" use="1902" target="1911" covered="1"/>
<du var="isTypeError" def="1890" use="1895" target="1895" covered="1"/>
<du var="isTypeError" def="1890" use="1895" target="1896" covered="1"/>
<du var="isTypeRecord" def="1891" use="1895" target="1895" covered="1"/>
<du var="isTypeRecord" def="1891" use="1895" target="1896" covered="1"/>
<du var="isTypeRecord" def="1891" use="1902" target="1902" covered="1"/>
<du var="isTypeRecord" def="1891" use="1902" target="1903" covered="1"/>
<du var="isTypeEnum" def="1892" use="1948" target="1949" covered="1"/>
<du var="isTypeEnum" def="1892" use="1948" target="1951" covered="1"/>
<du var="isTypeEnum" def="1892" use="1911" target="1912" covered="1"/>
<du var="isTypeEnum" def="1892" use="1911" target="1924" covered="1"/>
<du var="isTypeEnum" def="1892" use="1895" target="1895" covered="1"/>
<du var="isTypeEnum" def="1892" use="1895" target="1896" covered="1"/>
<du var="isTypeFixed" def="1893" use="1936" target="1937" covered="1"/>
<du var="isTypeFixed" def="1893" use="1936" target="1943" covered="1"/>
<du var="isTypeFixed" def="1893" use="1895" target="1896" covered="1"/>
<du var="isTypeFixed" def="1893" use="1895" target="1902" covered="1"/>
<du var="space" def="1896" use="1898" target="1899" covered="1"/>
<du var="space" def="1896" use="1898" target="1900" covered="1"/>
<du var="space" def="1896" use="1900" covered="1"/>
<du var="doc" def="1897" use="1903" covered="1"/>
<du var="doc" def="1897" use="1940" covered="1"/>
<du var="doc" def="1897" use="1922" covered="1"/>
<du var="space" def="1899" use="1900" covered="1"/>
<du var="name" def="1900" use="1903" covered="1"/>
<du var="name" def="1900" use="1909" target="1909" covered="0"/>
<du var="name" def="1900" use="1909" target="1909" covered="1"/>
<du var="name" def="1900" use="1909" covered="1"/>
<du var="name" def="1900" use="1940" covered="1"/>
<du var="name" def="1900" use="1941" target="1942" covered="1"/>
<du var="name" def="1900" use="1941" target="1943" covered="0"/>
<du var="name" def="1900" use="1922" covered="1"/>
<du var="result" def="1903" use="1946" target="1947" covered="1"/>
<du var="result" def="1903" use="1946" target="1953" covered="0"/>
<du var="result" def="1903" use="1953" covered="1"/>
<du var="result" def="1903" use="1951" covered="1"/>
<du var="fieldsNode" def="1905" use="1907" target="1907" covered="1"/>
<du var="fieldsNode" def="1905" use="1907" target="1908" covered="1"/>
<du var="fieldsNode" def="1905" use="1907" target="1908" covered="1"/>
<du var="fieldsNode" def="1905" use="1907" target="1909" covered="1"/>
```

```

<du var="fieldsNode" def="1905" use="1909" covered="1"/>
<du var="symbolsNode" def="1912" use="1913" target="1913" covered="1"/>
<du var="symbolsNode" def="1912" use="1913" target="1914" covered="1"/>
<du var="symbolsNode" def="1912" use="1913" target="1914" covered="1"/>
<du var="symbolsNode" def="1912" use="1913" target="1915" covered="1"/>
<du var="symbolsNode" def="1912" use="1915" covered="1"/>
<du var="symbolsNode" def="1912" use="1916" covered="1"/>
<du var="symbols" def="1915" use="1922" covered="1"/>
<du var="symbols" def="1915" use="1917" covered="1"/>
<du var="enumDefault" def="1918" use="1920" target="1921" covered="1"/>
<du var="enumDefault" def="1918" use="1920" target="1922" covered="1"/>
<du var="enumDefault" def="1918" use="1921" covered="1"/>
<du var="defaultSymbol" def="1919" use="1922" covered="1"/>
<du var="defaultSymbol" def="1921" use="1922" covered="1"/>
<du var="result" def="1922" use="1946" target="1947" covered="1"/>
<du var="result" def="1922" use="1946" target="1953" covered="0"/>
<du var="result" def="1922" use="1953" covered="1"/>
<du var="result" def="1922" use="1951" covered="1"/>
<du var="itemsNode" def="1925" use="1926" target="1927" covered="1"/>
<du var="itemsNode" def="1925" use="1926" target="1928" covered="1"/>
<du var="itemsNode" def="1925" use="1928" covered="1"/>
<du var="result" def="1929" use="1946" target="1947" covered="1"/>
<du var="result" def="1929" use="1946" target="1953" covered="0"/>
<du var="result" def="1929" use="1953" covered="1"/>
<du var="result" def="1929" use="1951" covered="1"/>
<du var="valuesNode" def="1931" use="1932" target="1933" covered="1"/>
<du var="valuesNode" def="1931" use="1932" target="1934" covered="1"/>
<du var="valuesNode" def="1931" use="1934" covered="1"/>
<du var="result" def="1935" use="1946" target="1947" covered="1"/>
<du var="result" def="1935" use="1946" target="1953" covered="0"/>
<du var="result" def="1935" use="1953" covered="1"/>
<du var="result" def="1935" use="1951" covered="1"/>
<du var="sizeNode" def="1937" use="1938" target="1938" covered="1"/>
<du var="sizeNode" def="1937" use="1938" target="1939" covered="1"/>
<du var="sizeNode" def="1937" use="1938" target="1939" covered="1"/>
<du var="sizeNode" def="1937" use="1938" target="1940" covered="1"/>
<du var="sizeNode" def="1937" use="1940" covered="1"/>
<du var="result" def="1940" use="1946" target="1947" covered="1"/>
<du var="result" def="1940" use="1946" target="1953" covered="0"/>
<du var="result" def="1940" use="1953" covered="1"/>
<du var="result" def="1940" use="1951" covered="1"/>
<du var="result" def="1940" use="1942" covered="1"/>
<du var="result" def="1944" use="1946" target="1947" covered="1"/>
<du var="result" def="1944" use="1946" target="1953" covered="0"/>
<du var="result" def="1944" use="1953" covered="1"/>
<du var="result" def="1944" use="1951" covered="1"/>
<du var="reserved" def="1947" use="1951" covered="1"/>
<du var="reserved" def="1949" use="1951" covered="1"/>
<counter type="DU" missed="22" covered="137"/>
<counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.11: DU mancate in parseNamesDeclared

```

<du var="schema" def="1880" use="1880" target="1881" covered="0"/>

```

Codice 4.12: DU mancate in parseNamesDeclared

```

<du var="name" def="1886" use="1903" covered="0"/>
<du var="name" def="1886" use="1909" target="1909" covered="0"/>
<du var="name" def="1886" use="1909" target="1909" covered="0"/>
<du var="name" def="1886" use="1909" covered="0"/>
<du var="name" def="1886" use="1940" covered="0"/>
<du var="name" def="1886" use="1941" target="1942" covered="0"/>
<du var="name" def="1886" use="1941" target="1943" covered="0"/>
<du var="name" def="1886" use="1922" covered="0"/>

```

Codice 4.13: DU mancate in parseNamesDeclared

```

<du var="doc" def="1888" use="1903" covered="0"/>
<du var="doc" def="1888" use="1940" covered="0"/>
<du var="doc" def="1888" use="1922" covered="0"/>

```

Codice 4.14: DU mancate in parseNamesDeclared

```

<du var="result" def="1889" use="1946" target="1947" covered="0"/>
<du var="result" def="1889" use="1951" covered="0"/>

```

Codice 4.15: DU mancate in parseNamesDeclared

```

<du var="name" def="1900" use="1909" target="1909" covered="0"/>
<du var="name" def="1900" use="1941" target="1943" covered="0"/>

```

Codice 4.16: DU mancate in parseNamesDeclared

```
<du var="result" def="1903" use="1946" target="1953" covered="0"/>
<du var="result" def="1922" use="1946" target="1953" covered="0"/>
<du var="result" def="1929" use="1946" target="1953" covered="0"/>
<du var="result" def="1935" use="1946" target="1953" covered="0"/>
<du var="result" def="1940" use="1946" target="1953" covered="0"/>
<du var="result" def="1944" use="1946" target="1953" covered="0"/>
```

Codice 4.17: Risultati Badua parseCompleteSchema

```
<method name="parseCompleteSchema"
  desc="(Lcom/fasterxml/jackson/databind/JsonNode;Lorg/apache/avro/Schema$Names;Ljava/lang/String;)Lorg/apache/avro/Schema;">
  <du var="schema" def="2020" use="2020" target="2021" covered="0"/>
  <du var="schema" def="2020" use="2020" target="2023" covered="1"/>
  <du var="schema" def="2020" use="2023" target="2024" covered="1"/>
  <du var="schema" def="2020" use="2023" target="2031" covered="1"/>
  <du var="schema" def="2020" use="2031" target="2032" covered="1"/>
  <du var="schema" def="2020" use="2031" target="2036" covered="1"/>
  <du var="schema" def="2020" use="2036" target="2037" covered="1"/>
  <du var="schema" def="2020" use="2036" target="2098" covered="1"/>
  <du var="schema" def="2020" use="2038" covered="1"/>
  <du var="schema" def="2020" use="2047" covered="1"/>
  <du var="schema" def="2020" use="2051" covered="1"/>
  <du var="schema" def="2020" use="2095" covered="1"/>
  <du var="schema" def="2020" use="2061" covered="1"/>
  <du var="schema" def="2020" use="2063" covered="0"/>
  <du var="schema" def="2020" use="2081" covered="1"/>
  <du var="schema" def="2020" use="2077" covered="1"/>
  <du var="schema" def="2020" use="2033" covered="1"/>
  <du var="schema" def="2020" use="2024" covered="1"/>
  <du var="names" def="2020" use="2053" covered="1"/>
  <du var="names" def="2020" use="2066" covered="1"/>
  <du var="names" def="2020" use="2085" covered="1"/>
  <du var="names" def="2020" use="2087" covered="1"/>
  <du var="names" def="2020" use="2082" covered="1"/>
  <du var="names" def="2020" use="2078" covered="1"/>
  <du var="names" def="2020" use="2033" covered="1"/>
  <du var="names" def="2020" use="2025" covered="1"/>
  <du var="names" def="2020" use="2027" covered="1"/>
  <du var="currentSpace" def="2020" use="2085" covered="1"/>
  <du var="currentSpace" def="2020" use="2082" covered="1"/>
  <du var="currentSpace" def="2020" use="2078" covered="1"/>
  <du var="currentSpace" def="2020" use="2050" covered="1"/>
  <du var="currentSpace" def="2020" use="2033" covered="1"/>
  <du var="currentSpace" def="2020" use="2027" covered="1"/>
  <du var="LOG" def="2020" use="2070" covered="0"/>
  <du var="SCHEMA_RESERVED" def="2020" use="2091" covered="1"/>
  <du var="ENUM_RESERVED" def="2020" use="2093" covered="1"/>
  <du var="type" def="2024" use="2027" covered="1"/>
  <du var="avroSchema" def="2025" use="2026" target="2027" covered="1"/>
  <du var="avroSchema" def="2025" use="2026" target="2029" covered="1"/>
  <du var="avroSchema" def="2025" use="2029" covered="1"/>
  <du var="avroSchema" def="2027" use="2029" covered="1"/>
  <du var="result" def="2037" use="2059" target="2059" covered="0"/>
  <du var="result" def="2037" use="2059" target="2091" covered="0"/>
  <du var="result" def="2037" use="2095" covered="0"/>
  <du var="result" def="2037" use="2096" covered="0"/>
  <du var="result" def="2037" use="2059" target="2060" covered="0"/>
  <du var="result" def="2037" use="2059" target="2091" covered="0"/>
  <du var="result" def="2037" use="2074" covered="0"/>
  <du var="result" def="2037" use="2084" target="2085" covered="1"/>
  <du var="result" def="2037" use="2084" target="2091" covered="0"/>
  <du var="type" def="2038" use="2092" target="2093" covered="1"/>
  <du var="type" def="2038" use="2092" target="2095" covered="1"/>
  <du var="type" def="2038" use="2080" target="2081" covered="1"/>
  <du var="type" def="2038" use="2080" target="2084" covered="1"/>
  <du var="type" def="2038" use="2085" covered="1"/>
  <du var="type" def="2038" use="2087" covered="1"/>
  <du var="type" def="2038" use="2045" target="2045" covered="1"/>
  <du var="type" def="2038" use="2045" target="2047" covered="1"/>
  <du var="type" def="2038" use="2045" target="2047" covered="1"/>
  <du var="type" def="2038" use="2045" target="2058" covered="1"/>
  <du var="name" def="2039" use="2066" covered="0"/>
  <du var="name" def="2039" use="2070" covered="0"/>
  <du var="isTypeError" def="2041" use="2058" target="2059" covered="1"/>
  <du var="isTypeError" def="2041" use="2058" target="2076" covered="1"/>
  <du var="isTypeError" def="2041" use="2045" target="2045" covered="1"/>
  <du var="isTypeError" def="2041" use="2045" target="2047" covered="1"/>
  <du var="isTypeRecord" def="2042" use="2045" target="2045" covered="1"/>
  <du var="isTypeRecord" def="2042" use="2045" target="2047" covered="1"/>
  <du var="isTypeRecord" def="2042" use="2058" target="2058" covered="1"/>
  <du var="isTypeRecord" def="2042" use="2058" target="2059" covered="1"/>
  <du var="isTypeArray" def="2043" use="2076" target="2077" covered="1"/>
  <du var="isTypeArray" def="2043" use="2076" target="2080" covered="1"/>
  <du var="space" def="2047" use="2049" target="2050" covered="1"/>
  <du var="space" def="2047" use="2049" target="2051" covered="1"/>
```

```

<du var="space" def="2047" use="2051" covered="1"/>
<du var="space" def="2050" use="2051" covered="1"/>
<du var="name" def="2051" use="2066" covered="1"/>
<du var="name" def="2051" use="2070" covered="0"/>
<du var="name" def="2051" use="2055" covered="0"/>
<du var="result" def="2053" use="2054" target="2055" covered="0"/>
<du var="result" def="2053" use="2054" target="2058" covered="1"/>
<du var="result" def="2053" use="2059" target="2059" covered="1"/>
<du var="result" def="2053" use="2059" target="2091" covered="0"/>
<du var="result" def="2053" use="2095" covered="1"/>
<du var="result" def="2053" use="2096" covered="1"/>
<du var="result" def="2053" use="2059" target="2060" covered="1"/>
<du var="result" def="2053" use="2059" target="2091" covered="0"/>
<du var="result" def="2053" use="2074" covered="1"/>
<du var="result" def="2053" use="2084" target="2085" covered="0"/>
<du var="result" def="2053" use="2084" target="2091" covered="1"/>
<du var="fields" def="2060" use="2074" covered="1"/>
<du var="fields" def="2060" use="2068" covered="1"/>
<du var="fieldsNode" def="2061" use="2062" target="2062" covered="1"/>
<du var="fieldsNode" def="2061" use="2062" target="2063" covered="0"/>
<du var="fieldsNode" def="2061" use="2062" target="2063" covered="0"/>
<du var="fieldsNode" def="2061" use="2062" target="2065" covered="1"/>
<du var="fieldsNode" def="2061" use="2065" covered="1"/>
<du var="field" def="2065" use="2069" target="2070" covered="0"/>
<du var="field" def="2065" use="2069" target="2073" covered="1"/>
<du var="field" def="2065" use="2072" covered="0"/>
<du var="f" def="2066" use="2069" target="2069" covered="1"/>
<du var="f" def="2066" use="2069" target="2073" covered="0"/>
<du var="f" def="2066" use="2072" covered="0"/>
<du var="result" def="2079" use="2095" covered="1"/>
<du var="result" def="2079" use="2096" covered="1"/>
<du var="result" def="2083" use="2095" covered="1"/>
<du var="result" def="2083" use="2096" covered="1"/>
<du var="result" def="2085" use="2086" target="2087" covered="1"/>
<du var="result" def="2085" use="2086" target="2091" covered="0"/>
<du var="result" def="2085" use="2095" covered="0"/>
<du var="result" def="2085" use="2096" covered="0"/>
<du var="result" def="2087" use="2095" covered="1"/>
<du var="result" def="2087" use="2096" covered="1"/>
<du var="reserved" def="2091" use="2095" covered="1"/>
<du var="reserved" def="2093" use="2095" covered="1"/>
<counter type="DU" missed="28" covered="90"/>
<counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.18: DU mancate in parseCompleteSchema

```

<du var="schema" def="2020" use="2020" target="2021" covered="0"/>
<du var="schema" def="2020" use="2063" covered="0"/>
<du var="LOG" def="2020" use="2070" covered="0"/>
<du var="result" def="2037" use="2059" target="2059" covered="0"/>
<du var="result" def="2037" use="2059" target="2091" covered="0"/>
<du var="result" def="2037" use="2095" covered="0"/>
<du var="result" def="2037" use="2096" covered="0"/>
<du var="result" def="2037" use="2059" target="2060" covered="0"/>
<du var="result" def="2037" use="2059" target="2091" covered="0"/>
<du var="result" def="2037" use="2074" covered="0"/>
<du var="result" def="2037" use="2084" target="2091" covered="0"/>
<du var="name" def="2039" use="2066" covered="0"/>
<du var="name" def="2039" use="2070" covered="0"/>
<du var="name" def="2051" use="2070" covered="0"/>
<du var="name" def="2051" use="2055" covered="0"/>
<du var="result" def="2053" use="2054" target="2055" covered="0"/>
<du var="result" def="2053" use="2059" target="2091" covered="0"/>
<du var="result" def="2053" use="2059" target="2091" covered="0"/>
<du var="result" def="2053" use="2084" target="2085" covered="0"/>
<du var="fieldsNode" def="2061" use="2062" target="2063" covered="0"/>
<du var="fieldsNode" def="2061" use="2062" target="2063" covered="0"/>
<du var="field" def="2065" use="2069" target="2070" covered="0"/>
<du var="field" def="2065" use="2072" covered="0"/>
<du var="f" def="2066" use="2069" target="2073" covered="0"/>
<du var="f" def="2066" use="2072" covered="0"/>
<du var="result" def="2085" use="2086" target="2091" covered="0"/>
<du var="result" def="2085" use="2095" covered="0"/>
<du var="result" def="2085" use="2096" covered="0"/>

```

Figura 4.31: Risultati 1 pitest parseNamesDeclared (prima)

```

1879 static Schema parseNamesDeclared(JsonNode schema, Names names, String currentNameSpace) {
1880 1 if (schema == null) {
1881 return null;
1882 }
1883 1 if (schema.isObject()) {
1884
1885 String type = Schema.getOptionalText(schema, "type");
1886 Name name = null;
1887
1888 String doc = null;
1889 Schema result = null;
1890 final boolean isTypeError = "error".equals(type);
1891 final boolean isTypeRecord = "record".equals(type);
1892 final boolean isTypeEnum = "enum".equals(type);
1893 final boolean isTypeFixed = "fixed".equals(type);
1894
1895 4 if (isTypeRecord || isTypeError || isTypeEnum || isTypeFixed) {
1896 String space = getOptionalText(schema, "namespace");
1897 doc = getOptionalText(schema, "doc");
1898 1 if (space == null)
1899 space = currentNameSpace;
1900 name = new Name(getRequiredText(schema, "name", "No name in schema"), space);
1901 }
1902 2 if (isTypeRecord || isTypeError) { // record
1903 result = new RecordSchema(name, doc, isTypeError);
1904 1 names.add(result);
1905 JsonNode fieldsNode = schema.get("fields");
1906
1907 2 if (fieldsNode == null || !fieldsNode.isArray())
1908 throw new SchemaParseException("Record has no fields: " + schema);
1909 2 exploreFields(fieldsNode, names, name != null ? name.space : null);
1910
1911 1 } else if (isTypeEnum) { // enum
1912 JsonNode symbolsNode = schema.get("symbols");
1913 2 if (symbolsNode == null || !symbolsNode.isArray())
1914 throw new SchemaParseException("Enum has no symbols: " + schema);
1915 LockableArrayList<String> symbols = new LockableArrayList<>(symbolsNode.size());
1916 for (JsonNode n : symbolsNode)
1917 symbols.add(n.textValue());
1918 JsonNode enumDefault = schema.get("default");
1919 String defaultSymbol = null;
1920 1 if (enumDefault != null)
1921 defaultSymbol = enumDefault.textValue();
1922 result = new EnumSchema(name, doc, symbols, defaultSymbol);
1923 1 names.add(result);
1924 1 } else if (type.equals("array")) { // array
1925 JsonNode itemsNode = schema.get("items");

```


Figura 4.32: Risultati 2 pitest parseNamesDeclared (prima)

```

1926 1 if (itemsNode == null)
1927     throw new SchemaParseException("Array has no items type: " + schema);
1928     final Schema items = Schema.parseNamesDeclared(itemsNode, names, currentNameSpace);
1929     result = Schema.createArray(items);
1930 1 } else if (type.equals("map")) { // map
1931     JsonNode valuesNode = schema.get("values");
1932 1 if (valuesNode == null)
1933     throw new SchemaParseException("Map has no values type: " + schema);
1934     final Schema values = Schema.parseNamesDeclared(valuesNode, names, currentNameSpace);
1935     result = Schema.createMap(values);
1936 1 } else if (isTypeFixed) { // fixed
1937     JsonNode sizeNode = schema.get("size");
1938 2 if (sizeNode == null || !sizeNode.isInt())
1939     throw new SchemaParseException("Invalid or no size: " + schema);
1940     result = new FixedSchema(name, doc, sizeNode.intValue());
1941 1 if (name != null)
1942 1     names.add(result);
1943 1 } else if (PRIMITIVES.containsKey(type)) {
1944     result = Schema.create(PRIMITIVES.get(type));
1945 }
1946 1 if (result != null) {
1947     Set<String> reserved = SCHEMA_RESERVED;
1948 1 if (isTypeEnum) {
1949     reserved = ENUM_RESERVED;
1950 }
1951 1 Schema.addProperty(schema, reserved, result);
1952 }
1953 1 return result;
1954 1 } else if (schema.isArray()) {
1955     List<Schema> subs = new ArrayList<>(schema.size());
1956 1 schema.forEach((JsonNode item) -> {
1957     Schema sub = Schema.parseNamesDeclared(item, names, currentNameSpace);
1958 1 if (sub != null) {
1959     subs.add(sub);
1960 }
1961 });
1962 1 return Schema.createUnion(subs);
1963 1 } else if (schema.isTextual()) {
1964     String value = schema.asText();
1965 1 return names.get(value);
1966 }
1967 return null;
1968 }

```

Tabella 4.24: Testing pitest parseNamesDeclared Schema

schema	names	Risultato atteso	Motivazione
RECORD_WITH_RECORD_FIELDS	valid	valid instance	Il json rispetta le specifiche, quindi ci aspettiamo un'istanza di schema corretta
LOGICAL_TYPE_DATE	valid	valid instance	Il json rispetta le specifiche, quindi ci aspettiamo un'istanza di schema corretta

Figura 4.33: Risultati 1 pitest parseNamesDeclared (dopo)

```

1907 2 if (fieldsNode == null || !fieldsNode.isArray())
1908     throw new SchemaParseException("Record has no fields: " + schema);
1909 2 exploreFields(fieldsNode, names, name != null ? name.space : null);
1910

```

Figura 4.34: Risultati 2 pitest parseNamesDeclared (dopo)

```

1945 }
1946 1 if (result != null) {
1947     Set<String> reserved = SCHEMA_RESERVED;
1948 1 if (isTypeEnum) {
1949     reserved = ENUM_RESERVED;
1950 }
1951 1 Schema.addProperty(schema, reserved, result);
1952 }

```

Figura 4.35: Risultati piteest parseCompleteSchema (prima)

```

2019 static Schema parseCompleteSchema(JsonNode schema, Names names, String currentSpace) {
2020     if (schema == null) {
2021         throw new SchemaParseException("Cannot parse <null> schema");
2022     }
2023     if (schema.isTextual()) {
2024         String type = schema.asText();
2025         Schema avroSchema = names.get(type);
2026         if (avroSchema == null) {
2027             avroSchema = names.get(currentSpace + "." + type);
2028         }
2029         return avroSchema;
2030     }
2031     if (schema.isArray()) {
2032         List<Schema> schemas = StreamSupport.stream(schema.spliterator(), false)
2033             .map((JsonNode sub) -> parseCompleteSchema(sub, names, currentSpace)).collect(Collectors.toList());
2034         return Schema.createUnion(schemas);
2035     }
2036     if (schema.isObject()) {
2037         Schema result = null;
2038         String type = getRequiredText(schema, "type", "No type");
2039         Name name = null;
2040
2041         final boolean isTypeError = "error".equals(type);
2042         final boolean isTypeRecord = "record".equals(type);
2043         final boolean isTypeArray = "array".equals(type);
2044
2045         if (isTypeRecord || isTypeError || "enum".equals(type) || "fixed".equals(type)) {
2046             // named schema
2047             String space = getOptionalText(schema, "namespace");
2048
2049             if (space == null)
2050                 space = currentSpace;
2051             name = new Name(getRequiredText(schema, "name", "No name in schema"), space);
2052
2053             result = names.get(name);
2054             if (result == null) {
2055                 throw new SchemaParseException("Unparsed field type " + name);
2056             }
2057         }
2058         if (isTypeRecord || isTypeError) {
2059             if (result != null && !result.hasFields()) {
2060                 final List<Field> fields = new ArrayList<>();
2061                 JsonNode fieldsNode = schema.get("fields");
2062                 if (fieldsNode == null || !fieldsNode.isArray())
2063                     throw new SchemaParseException("Record has no fields: " + schema);
2064
2065                 for (JsonNode field : fieldsNode) {
2066                     Field f = Field.parse(field, names, name.space);
2067
2068                     fields.add(f);
2069                 }
2070                 if (f.schema.getLogicalType() == null && getOptionalText(field, LOGICAL_TYPE_PROP) != null)
2071                     LOG.warn(
2072                         "Ignored the {}.{}.logicalType property (\"{}{}\"). It should probably be nested inside the \"type\" for the field.",
2073                         name, f.name, getOptionalText(field, "logicalType"));
2074                 result.setFields(fields);
2075             }
2076         }
2077         else if (isTypeArray) {
2078             JsonNode items = schema.get("items");
2079             Schema schemaItems = parseCompleteSchema(items, names, currentSpace);
2080             result = Schema.createArray(schemaItems);
2081         }
2082         else if ("map".equals(type)) {
2083             JsonNode values = schema.get("values");
2084             Schema mapItems = parseCompleteSchema(values, names, currentSpace);
2085             result = Schema.createMap(mapItems);
2086         }
2087         else if (result == null) {
2088             result = names.get(currentSpace + "." + type);
2089             if (result == null) {
2090                 result = names.get(type);
2091             }
2092         }
2093     }
2094
2095     Set<String> reserved = SCHEMA_RESERVED;
2096     if ("enum".equals(type)) {
2097         reserved = ENUM_RESERVED;
2098     }
2099     Schema.addProperties(schema, reserved, result);
2100     return result;
2101 }

```

Figura 4.36: Power mock per il campo statico LOG

```

Logger mockLogger = spy(Schema.LOG);
Whitebox.setInternalState(Schema.class, "LOG", mockLogger);

Schema.parse(schemaJsonNode, schemaNames);

String schemaType = schemaJsonNode.get("type").toString();
boolean isRecord = schemaType.equals("\"record\"");
boolean isError = schemaType.equals("\"error\"");

if(isRecord || isError) verify(mockLogger, times(1)).warn(anyString(), any(Object[].class));

```


Tabella 4.25: Testing pitest parseCompleteSchema Schema

schema	names	Risultato atteso	Motivazione
record with logicalType property not nested in type field	valid	valid instance	La proprietà logicalType dovrebbe essere dentro un oggetto type in questi casi, ma non è obbligatorio quindi ci si aspetta solo un warning proveniente dal logger

Figura 4.37: Risultati pitest parseCompleteSchema (dopo)

```

2068      fields.add(f);
2069      if (f.schema.getLogicalType() == null && getOptionalText(field, LOGICAL_TYPE_PROP) != null)
2070          LOG.warn(
2071              "Ignored the {}.{}.logicalType property \"{}\". It should probably be nested inside the \"type\" for the field.",
2072              name, f.name, getOptionalText(field, "logicalType"));
2073      }
2074      result.setFields(fields);

```

Figura 4.38: Mock per schema non valid

```

public class InvalidSchema extends Schema { 1 usage new *

    InvalidSchema(Type type) { 1 usage new *
        super(type);
    }

    @Override new *
    public Type getType() {
        throw new NullPointerException();
    }
}

```

Tabella 4.26: Test suite checkReaderWriterCompatibility

reader	writer	Risultato atteso	Motivazione
<i>Schemi compatibili</i>			
valid compatible	valid	COMPATIBLE	Reader e writer uguali, quindi devono essere compatibili
<i>Schemi invalidi</i>			
null	null	Exception	La coppia è invalida, quindi ci si aspetta un errore
null	valid	Exception	La coppia è invalida, quindi ci si aspetta un errore
valid	null	Exception	La coppia è invalida, quindi ci si aspetta un errore
invalid	invalid	Exception	La coppia è invalida, quindi ci si aspetta un errore
invalid	valid	Exception	La coppia è invalida, quindi ci si aspetta un errore
valid	invalid	Exception	La coppia è invalida, quindi ci si aspetta un errore
<i>Schemi incompatibili primitivi</i>			
primitive valid promotable	valid	COMPATIBLE	Il writer è promovibile al reader come da specifica (es: writer int e reader long), quindi devono essere compatibili
primitive valid non promotable	valid	INCOMPATIBLE: TYPE_MISMATCH	Il writer non è promovibile al reader come da specifica (es: writer long e reader int), quindi devono essere incompatibili per via di un mismatch nel tipo
<i>Schemi incompatibili complessi</i>			
complex valid incompatible	complex valid	INCOMPATIBLE: TYPE_MISMATCH	Il reader è dello stesso tipo complesso del writer o una union, ma c'è mismatch nei tipi quindi devono essere incompatibili
named complex incompatible	named valid	INCOMPATIBLE: NAME_MISMATCH	Il reader è dello stesso tipo complesso e named del writer (record, enum e fixed), ma hanno nomi diversi quindi devono essere incompatibili
enum valid incompatible	enum valid	INCOMPATIBLE: MISSING_ENUM_SYMBOLS	La enumerazione reader è come quella writer, ma un simbolo del writer non è presente nel reader e quest'ultimo non ha un valore di default, quindi devono essere incompatibili
fixed valid incompatible	fixed valid	INCOMPATIBLE: FIXED_SIZE_MISMATCH	Il fixed reader è come il writer ma ha una size diversa, quindi devono essere incompatibili
record valid incompatible	record valid	INCOMPATIBLE: READER_FIELD_MISSING_DEFAULT_VALUE	Il record reader è come il writer, ma un campo nel reader non è presente nel writer, e per esso non c'è un valore di default, quindi devono essere incompatibili
valid incompatible	union valid	INCOMPATIBLE: MISSING_UNION_BRANCH	Il writer è una union, e il reader è o una union di tipi diversi o un tipo non presente nel writer, quindi devono essere incompatibili

Figura 4.39: Risultati jacoco checkReaderWriterCompatibility

```

60. public static SchemaPairCompatibility checkReaderWriterCompatibility(final Schema reader, final Schema writer) {
61.     final SchemaCompatibilityResult compatibility = new ReaderWriterCompatibilityChecker().getCompatibility(reader,
62.     writer);
63.
64.     final String message;
65.     switch (compatibility.getCompatibility()) {
66.     case INCOMPATIBLE: {
67.         message = String.format(
68.             "Data encoded using writer schema:%n%s%n" + "will or may fail to decode using reader schema:%n%s%n",
69.             writer.toString(true), reader.toString(true));
70.         break;
71.     }
72.     case COMPATIBLE: {
73.         message = READER_WRITER_COMPATIBLE_MESSAGE;
74.         break;
75.     }
76.     default:
77.         throw new AvroRuntimeException("Unknown compatibility: " + compatibility);
78.     }
79.
80.     return new SchemaPairCompatibility(compatibility, reader, writer, message);
81. }

```

Figura 4.40: Risultati jacoco getCompatibility

```

227. private SchemaCompatibilityResult getCompatibility(String referenceToken, final Schema reader, final Schema writer,
228.     final Deque<String> location) {
229.     location.addFirst(referenceToken);
230.     LOG.debug("Checking compatibility of reader {} with writer {}", reader, writer);
231.     final ReaderWriter pair = new ReaderWriter(reader, writer);
232.     SchemaCompatibilityResult result = mMemoizeMap.get(pair);
233.     if (result != null) {
234.         if (result.getCompatibility() == SchemaCompatibilityType.RECURSION_IN_PROGRESS) {
235.             // Break the recursion here.
236.             // schemas are compatible unless proven incompatible:
237.             result = SchemaCompatibilityResult.compatible();
238.         }
239.     } else {
240.         // Mark this reader/writer pair as "in progress":
241.         mMemoizeMap.put(pair, SchemaCompatibilityResult.recursionInProgress());
242.         result = calculateCompatibility(reader, writer, location);
243.         mMemoizeMap.put(pair, result);
244.     }
245.     location.removeFirst();
246.     return result;
247. }

```

Figura 4.41: Risultati 1 jacoco calculateCompatibility(prima)

```

262. private SchemaCompatibilityResult calculateCompatibility(final Schema reader, final Schema writer,
263. final Deque<String> location) {
264.     assert (reader != null);
265.     assert (writer != null);
266.     SchemaCompatibilityResult result = SchemaCompatibilityResult.compatible();
267.
268.     if (reader.getType() == writer.getType()) {
269.         switch (reader.getType()) {
270.             case NULL:
271.             case BOOLEAN:
272.             case INT:
273.             case LONG:
274.             case FLOAT:
275.             case DOUBLE:
276.             case BYTES:
277.             case STRING: {
278.                 return result;
279.             }
280.             case ARRAY: {
281.                 return result
282.                     .mergedWith(getCompatibility("items", reader.getElementType(), writer.getElementType(), location));
283.             }
284.             case MAP: {
285.                 return result.mergedWith(getCompatibility("values", reader.getValueType(), writer.getValueType(), location));
286.             }
287.             case FIXED: {
288.                 result = result.mergedWith(checkSchemaNames(reader, writer, location));
289.                 return result.mergedWith(checkFixedSize(reader, writer, location));
290.             }
291.             case ENUM: {
292.                 result = result.mergedWith(checkSchemaNames(reader, writer, location));
293.                 return result.mergedWith(checkReaderEnumContainsAllWriterEnumSymbols(reader, writer, location));
294.             }
295.             case RECORD: {
296.                 result = result.mergedWith(checkSchemaNames(reader, writer, location));
297.                 return result.mergedWith(checkReaderWriterRecordFields(reader, writer, location));
298.             }
299.             case UNION: {
300.                 // Check that each individual branch of the writer union can be decoded:
301.                 int i = 0;
302.                 for (final Schema writerBranch : writer.getTypes()) {
303.                     location.addFirst(Integer.toString(i));
304.                     SchemaCompatibilityResult compatibility = getCompatibility(reader, writerBranch);
305.                     if (compatibility.getCompatibility() == SchemaCompatibilityType.INCOMPATIBLE) {
306.                         String message = String.format("reader union lacking writer type: %s", writerBranch.getType());
307.                         result = result.mergedWith(SchemaCompatibilityResult.incompatible(
308.                             SchemaIncompatibilityType.MISSING_UNION_BRANCH, reader, writer, message, asList(location)));
309.                     }
310.                     location.removeFirst();
311.                     i++;
312.                 }
313.                 // Each schema in the writer union can be decoded with the reader:
314.                 return result;
315.             }
316.             default: {
317.                 throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
318.             }
319.         }
320.     }
321. }
322. } else {
323.     // Reader and writer have different schema types:

```

Figura 4.42: Risultati 2 jacoco calculateCompatibility(prima)

```

324.
325. // Reader compatible with all branches of a writer union is compatible
326. ◆ if (writer.getType() == Schema.Type.UNION) {
327.     int index = 0;
328.     ◆ for (Schema s : writer.getTypes()) {
329.         result = result.mergedWith(getCompatibility(Integer.toString(index), reader, s, location));
330.         index++;
331.     }
332.     return result;
333. }
334.
335. ◆ switch (reader.getType()) {
336.     case NULL:
337.         return result.mergedWith(typeMismatch(reader, writer, location));
338.     case BOOLEAN:
339.         return result.mergedWith(typeMismatch(reader, writer, location));
340.     case INT:
341.         return result.mergedWith(typeMismatch(reader, writer, location));
342.     case LONG: {
343.         ◆ return (writer.getType() == Type.INT) ? result : result.mergedWith(typeMismatch(reader, writer, location));
344.     }
345.     case FLOAT: {
346.         ◆ return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG)) ? result
347.             : result.mergedWith(typeMismatch(reader, writer, location));
348.     }
349.     case DOUBLE: {
350.         ◆ return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG) || (writer.getType() == Type.FLOAT))
351.             ? result
352.             : result.mergedWith(typeMismatch(reader, writer, location));
353.     }
354.     case BYTES: {
355.         ◆ return (writer.getType() == Type.STRING) ? result : result.mergedWith(typeMismatch(reader, writer, location));
356.     }
357.     case STRING: {
358.         ◆ return (writer.getType() == Type.BYTES) ? result : result.mergedWith(typeMismatch(reader, writer, location));
359.     }
360.     case ARRAY:
361.         return result.mergedWith(typeMismatch(reader, writer, location));
362.     case MAP:
363.         return result.mergedWith(typeMismatch(reader, writer, location));
364.     case FIXED:
365.         return result.mergedWith(typeMismatch(reader, writer, location));
366.     case ENUM:
367.         return result.mergedWith(typeMismatch(reader, writer, location));
368.     case RECORD:
369.         return result.mergedWith(typeMismatch(reader, writer, location));
370.     case UNION: {
371.         ◆ for (final Schema readerBranch : reader.getTypes()) {
372.             SchemaCompatibilityResult compatibility = getCompatibility(readerBranch, writer);
373.             ◆ if (compatibility.getCompatibility() == SchemaCompatibilityType.COMPATIBLE) {
374.                 return result;
375.             }
376.         }
377.     }
378.     // No branch in the reader union has been found compatible with the writer
379.     // schema:
380.     String message = String.format("reader union lacking writer type: %s", writer.getType());
381.     return result.mergedWith(SchemaCompatibilityResult
382.         .incompatible(SchemaIncompatibilityType.MISSING_UNION_BRANCH, reader, writer, message, asList(location)));
383. }
384.
385. default: {
386.     throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
387. }
388. }
389. }
390. }
391. }

```

Tabella 4.27: Test jacoco calculateCompatibility

reader	writer	Risultato atteso	Motivazione
<i>Schemi primitivi incompatibili</i>			
boolean	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
bytes	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
double	string	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
<i>Schemi complessi incompatibili</i>			
array	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
map	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
fixed	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
enum	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica
fixed	int	INCOMPATIBLE: TYPE_MISMATCH	I tipi sono incompatibili da specifica

Figura 4.43: Risultati 2 jacoco calculateCompatibility(dopo)

```

325. // Reader compatible with all branches of a writer union is compatible
326. ◆ if (writer.getType() == Schema.Type.UNION) {
327.     int index = 0;
328.     ◆ for (Schema s : writer.getTypes()) {
329.         result = result.mergedWith(getCompatibility(Integer.toString(index), reader, s, location));
330.         index++;
331.     }
332.     return result;
333. }
334.
335. ◆ switch (reader.getType()) {
336.     case NULL:
337.         return result.mergedWith(typeMismatch(reader, writer, location));
338.     case BOOLEAN:
339.         return result.mergedWith(typeMismatch(reader, writer, location));
340.     case INT:
341.         return result.mergedWith(typeMismatch(reader, writer, location));
342.     case LONG: {
343.         ◆ return (writer.getType() == Type.INT) ? result : result.mergedWith(typeMismatch(reader, writer, location));
344.     }
345.     case FLOAT: {
346.         ◆ return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG)) ? result
347.             : result.mergedWith(typeMismatch(reader, writer, location));
348.     }
349.     case DOUBLE: {
350.         ◆ return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG) || (writer.getType() == Type.FLOAT))
351.             ? result
352.             : result.mergedWith(typeMismatch(reader, writer, location));
353.     }
354.     case BYTES: {
355.         ◆ return (writer.getType() == Type.STRING) ? result : result.mergedWith(typeMismatch(reader, writer, location));
356.     }
357.     case STRING: {
358.         ◆ return (writer.getType() == Type.BYTES) ? result : result.mergedWith(typeMismatch(reader, writer, location));
359.     }
360.     case ARRAY:
361.         return result.mergedWith(typeMismatch(reader, writer, location));
362.     case MAP:
363.         return result.mergedWith(typeMismatch(reader, writer, location));
364.     case FIXED:
365.         return result.mergedWith(typeMismatch(reader, writer, location));
366.     case ENUM:
367.         return result.mergedWith(typeMismatch(reader, writer, location));
368.     case RECORD:
369.         return result.mergedWith(typeMismatch(reader, writer, location));
370.     case UNION: {
371.         ◆ for (final Schema readerBranch : reader.getTypes()) {
372.             SchemaCompatibilityResult compatibility = getCompatibility(readerBranch, writer);
373.             ◆ if (compatibility.getCompatibility() == SchemaCompatibilityType.COMPATIBLE) {
374.                 return result;
375.             }
376.         }
377.     }
378.     // No branch in the reader union has been found compatible with the writer
379.     // schema:
380.     String message = String.format("reader union lacking writer type: %s", writer.getType());
381.     return result.mergedWith(SchemaCompatibilityResult
382.         .incompatible(SchemaIncompatibilityType.MISSING_UNION_BRANCH, reader, writer, message, asList(location)));
383. }
384.
385. default: {
386.     throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
387. }
388. }
389. }
390. }
391. }

```

Codice 4.19: Risultati Badua checkReaderWriterCompatibility

```

<method name="checkReaderWriterCompatibility"
  desc="(Lorg/apache/avro/Schema;Lorg/apache/avro/Schema;)Lorg/apache/avro/SchemaCompatibility$SchemaPairCompatibility;">
  <du var="reader" def="61" use="80" covered="1"/>
  <du var="reader" def="61" use="69" covered="1"/>
  <du var="writer" def="61" use="80" covered="1"/>
  <du var="writer" def="61" use="69" covered="1"/>
  <du var="$SwitchMap$org$apache$avro$SchemaCompatibility$SchemaCompatibilityType" def="61" use="65" target="77" covered="0"/>
  <du var="$SwitchMap$org$apache$avro$SchemaCompatibility$SchemaCompatibilityType" def="61" use="65" target="67" covered="1"/>
  <du var="$SwitchMap$org$apache$avro$SchemaCompatibility$SchemaCompatibilityType" def="61" use="65" target="73" covered="1"/>
  <du var="compatibility" def="61" use="65" target="77" covered="0"/>
  <du var="compatibility" def="61" use="65" target="67" covered="1"/>
  <du var="compatibility" def="61" use="65" target="73" covered="1"/>
  <du var="compatibility" def="61" use="80" covered="1"/>
  <du var="compatibility" def="61" use="77" covered="0"/>
  <du var="message" def="67" use="80" covered="1"/>
  <du var="message" def="73" use="80" covered="1"/>
  <counter type="DU" missed="3" covered="11"/>
  <counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.20: Risultati Badua getCompatibility

```

<method name="getCompatibility"
  desc="(Ljava/lang/String;Lorg/apache/avro/Schema;Lorg/apache/avro/Schema;Ljava/util/Deque;)Lorg/apache/avro/SchemaCompatibility$SchemaCompatibilityResult;">
  <du var="this" def="229" use="241" covered="1"/>
  <du var="this" def="229" use="242" covered="1"/>
  <du var="this" def="229" use="243" covered="1"/>
  <du var="reader" def="229" use="242" covered="1"/>
  <du var="writer" def="229" use="242" covered="1"/>
  <du var="location" def="229" use="242" covered="1"/>
  <du var="location" def="229" use="245" covered="1"/>
  <du var="this.mMemoizeMap" def="229" use="241" covered="1"/>
  <du var="this.mMemoizeMap" def="229" use="243" covered="1"/>
  <du var="RECURSION_IN_PROGRESS" def="229" use="234" target="237" covered="0"/>
  <du var="RECURSION_IN_PROGRESS" def="229" use="234" target="245" covered="0"/>
  <du var="pair" def="231" use="241" covered="1"/>
  <du var="pair" def="231" use="243" covered="1"/>
  <du var="result" def="232" use="233" target="234" covered="0"/>
  <du var="result" def="232" use="233" target="241" covered="1"/>
  <du var="result" def="232" use="234" target="237" covered="0"/>
  <du var="result" def="232" use="234" target="245" covered="0"/>
  <du var="result" def="232" use="246" covered="0"/>
  <du var="result" def="237" use="246" covered="0"/>
  <du var="result" def="242" use="246" covered="1"/>
  <counter type="DU" missed="7" covered="13"/>
  <counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.21: Risultati Badua calculateCompatibility

```

<method name="calculateCompatibility"
  desc="(Lorg/apache/avro/Schema;Lorg/apache/avro/Schema;Ljava/util/Deque;)Lorg/apache/avro/SchemaCompatibility$SchemaCompatibilityResult;">
  <du var="this" def="264" use="374" covered="1"/>
  <du var="this" def="264" use="371" covered="1"/>
  <du var="this" def="264" use="369" covered="1"/>
  <du var="this" def="264" use="367" covered="1"/>
  <du var="this" def="264" use="365" covered="1"/>
  <du var="this" def="264" use="363" covered="1"/>
  <du var="this" def="264" use="359" covered="1"/>
  <du var="this" def="264" use="356" covered="1"/>
  <du var="this" def="264" use="351" covered="1"/>
  <du var="this" def="264" use="346" covered="1"/>
  <du var="this" def="264" use="343" covered="1"/>
  <du var="this" def="264" use="341" covered="1"/>
  <du var="this" def="264" use="339" covered="1"/>
  <du var="this" def="264" use="337" covered="1"/>
  <du var="this" def="264" use="329" covered="1"/>
  <du var="this" def="264" use="304" covered="1"/>
  <du var="this" def="264" use="296" covered="1"/>
  <du var="this" def="264" use="297" covered="1"/>
  <du var="this" def="264" use="292" covered="1"/>
  <du var="this" def="264" use="293" covered="1"/>
  <du var="this" def="264" use="288" covered="1"/>
  <du var="this" def="264" use="289" covered="1"/>
  <du var="this" def="264" use="285" covered="1"/>
  <du var="this" def="264" use="281" covered="1"/>
  <du var="reader" def="264" use="268" target="269" covered="1"/>
  <du var="reader" def="264" use="268" target="326" covered="1"/>
  <du var="reader" def="264" use="335" target="387" covered="0"/>
  <du var="reader" def="264" use="335" target="337" covered="1"/>
  <du var="reader" def="264" use="335" target="339" covered="1"/>
  <du var="reader" def="264" use="335" target="341" covered="1"/>
  <du var="reader" def="264" use="335" target="343" covered="1"/>
  <du var="reader" def="264" use="335" target="346" covered="1"/>
  <du var="reader" def="264" use="335" target="351" covered="1"/>
  <du var="reader" def="264" use="335" target="356" covered="1"/>
  <du var="reader" def="264" use="335" target="359" covered="1"/>
  <du var="reader" def="264" use="335" target="363" covered="1"/>
  <du var="reader" def="264" use="335" target="365" covered="1"/>
  <du var="reader" def="264" use="335" target="367" covered="1"/>
  <du var="reader" def="264" use="335" target="369" covered="1"/>
  <du var="reader" def="264" use="335" target="371" covered="1"/>
  <du var="reader" def="264" use="335" target="373" covered="1"/>
  <du var="reader" def="264" use="373" covered="1"/>
  <du var="reader" def="264" use="382" covered="1"/>
  <du var="reader" def="264" use="371" covered="1"/>
  <du var="reader" def="264" use="369" covered="1"/>
  <du var="reader" def="264" use="367" covered="1"/>
  <du var="reader" def="264" use="365" covered="1"/>
  <du var="reader" def="264" use="363" covered="1"/>
  <du var="reader" def="264" use="359" covered="1"/>
  <du var="reader" def="264" use="356" covered="1"/>
  <du var="reader" def="264" use="351" covered="1"/>
  <du var="reader" def="264" use="346" covered="1"/>
  <du var="reader" def="264" use="343" covered="1"/>
  <du var="reader" def="264" use="341" covered="1"/>
  <du var="reader" def="264" use="339" covered="1"/>
  <du var="reader" def="264" use="337" covered="1"/>

```

[illegible]

[illegible]

```

<du var="i" def="301" use="303" covered="1"/>
<du var="i" def="301" use="311" covered="1"/>
<du var="writerBranch" def="302" use="306" covered="1"/>
<du var="compatibility" def="304" use="305" target="306" covered="1"/>
<du var="compatibility" def="304" use="305" target="310" covered="1"/>
<du var="result" def="307" use="314" covered="1"/>
<du var="result" def="307" use="307" covered="1"/>
<du var="i" def="311" use="303" covered="1"/>
<du var="i" def="311" use="311" covered="1"/>
<du var="index" def="327" use="329" covered="1"/>
<du var="index" def="327" use="330" covered="1"/>
<du var="result" def="329" use="332" covered="1"/>
<du var="result" def="329" use="329" covered="1"/>
<du var="index" def="330" use="329" covered="1"/>
<du var="index" def="330" use="330" covered="1"/>
<du var="compatibility" def="374" use="375" target="376" covered="1"/>
<du var="compatibility" def="374" use="375" target="378" covered="1"/>
<counter type="DU" missed="9" covered="247"/>
<counter type="METHOD" missed="0" covered="1"/>
</method>

```

Codice 4.22: DU mancate in calculateCompatibility

```

<du var="reader" def="264" use="387" covered="0"/>
<du var="reader" def="264" use="269" target="318" covered="0"/>
<du var="reader" def="264" use="318" covered="0"/>
<du var="$assertionsDisabled" def="264" use="264" target="265" covered="0"/>
<du var="$assertionsDisabled" def="264" use="265" target="266" covered="0"/>
<du var="$SwitchMap$org$apache$avro$Schema$Type" def="264" use="335" target="387" covered="0"/>
<du var="$SwitchMap$org$apache$avro$Schema$Type" def="264" use="269" target="318" covered="0"/>
<du var="result" def="266" use="332" covered="0"/>

```

Figura 4.44: Risultati pitest checkReaderWriterCompatibility

```

60 public static SchemaPairCompatibility checkReaderWriterCompatibility(final Schema reader, final Schema writer) {
61     final SchemaCompatibilityResult compatibility = new ReaderWriterCompatibilityChecker().getCompatibility(reader,
62     writer);
63
64     final String message;
65     switch (compatibility.getCompatibility()) {
66     case INCOMPATIBLE: {
67         message = String.format(
68             "Data encoded using writer schema:%n%s%n" + "will or may fail to decode using reader schema:%n%s%n",
69             writer.toString(true), reader.toString(true));
70         break;
71     }
72     case COMPATIBLE: {
73         message = READER_WRITER_COMPATIBLE_MESSAGE;
74         break;
75     }
76     default:
77         throw new AvroRuntimeException("Unknown compatibility: " + compatibility);
78     }
79
80     return new SchemaPairCompatibility(compatibility, reader, writer, message);
81 }

```

Figura 4.45: Risultati pitest getCompatibility

```

227 private SchemaCompatibilityResult getCompatibility(String referenceToken, final Schema reader, final Schema writer,
228     final Deque<String> location) {
229     location.addFirst(referenceToken);
230     LOG.debug("Checking compatibility of reader {} with writer {}", reader, writer);
231     final ReaderWriter pair = new ReaderWriter(reader, writer);
232     SchemaCompatibilityResult result = mMemoizeMap.get(pair);
233     if (result != null) {
234         if (result.getCompatibility() == SchemaCompatibilityType.RECURSION_IN_PROGRESS) {
235             // Break the recursion here.
236             // schemas are compatible unless proven incompatible:
237             result = SchemaCompatibilityResult.compatible();
238         }
239     } else {
240         // Mark this reader/writer pair as "in progress":
241         mMemoizeMap.put(pair, SchemaCompatibilityResult.recursionInProgress());
242         result = calculateCompatibility(reader, writer, location);
243         mMemoizeMap.put(pair, result);
244     }
245     location.removeFirst();
246     return result;
247 }

```

Figura 4.46: Risultati 1 pitest calculateCompatibility (prima)

```

262 private SchemaCompatibilityResult calculateCompatibility(final Schema reader, final Schema writer,
263     final Deque<String> location) {
264     assert (reader != null);
265     assert (writer != null);
266     SchemaCompatibilityResult result = SchemaCompatibilityResult.compatible();
267
268 1   if (reader.getType() == writer.getType()) {
269       switch (reader.getType()) {
270           case NULL:
271           case BOOLEAN:
272           case INT:
273           case LONG:
274           case FLOAT:
275           case DOUBLE:
276           case BYTES:
277           case STRING: {
278 1       return result;
279           }
280           case ARRAY: {
281 1       return result
282           .mergedWith(getCompatibility("items", reader.getElementType(), writer.getElementType(), location));
283           }
284           case MAP: {
285 1       return result.mergedWith(getCompatibility("values", reader.getValueType(), writer.getValueType(), location));
286           }
287           case FIXED: {
288               result = result.mergedWith(checkSchemaNames(reader, writer, location));
289 1       return result.mergedWith(checkFixedSize(reader, writer, location));
290           }
291           case ENUM: {
292               result = result.mergedWith(checkSchemaNames(reader, writer, location));
293 1       return result.mergedWith(checkReaderEnumContainsAllWriterEnumSymbols(reader, writer, location));
294           }
295           case RECORD: {
296               result = result.mergedWith(checkSchemaNames(reader, writer, location));
297 1       return result.mergedWith(checkReaderWriterRecordFields(reader, writer, location));
298           }
299           case UNION: {
300               // Check that each individual branch of the writer union can be decoded:
301               int i = 0;
302               for (final Schema writerBranch : writer.getTypes()) {
303 1               location.addFirst(Integer.toString(i));
304                   SchemaCompatibilityResult compatibility = getCompatibility(reader, writerBranch);
305 1               if (compatibility.getCompatibility() == SchemaCompatibilityType.INCOMPATIBLE) {
306                   String message = String.format("reader union lacking writer type: %s", writerBranch.getType());
307                   result = result.mergedWith(SchemaCompatibilityResult.incompatible(
308                       SchemaIncompatibilityType.MISSING_UNION_BRANCH, reader, writer, message, asList(location)));
309               }
310               location.removeFirst();
311 1               i++;
312               }
313               // Each schema in the writer union can be decoded with the reader:
314 1       return result;
315           }
316       }
317       default: {
318           throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
319       }
320   }
321 }
322 } else {
323     // Reader and writer have different schema types:
324

```

Figura 4.47: Risultati pitest calculateCompatibility (prima)

```

325 // Reader compatible with all branches of a writer union is compatible
326 1 if (writer.getType() == Schema.Type.UNION) {
327     int index = 0;
328     for (Schema s : writer.getTypes()) {
329         result = result.mergedWith(getCompatibility(Integer.toString(index), reader, s, location));
330 1 index++;
331     }
332 1 return result;
333 }
334
335 switch (reader.getType()) {
336 case NULL:
337 1 return result.mergedWith(typeMismatch(reader, writer, location));
338 case BOOLEAN:
339 1 return result.mergedWith(typeMismatch(reader, writer, location));
340 case INT:
341 1 return result.mergedWith(typeMismatch(reader, writer, location));
342 case LONG: {
343 2 return (writer.getType() == Type.INT) ? result : result.mergedWith(typeMismatch(reader, writer, location));
344 }
345 case FLOAT: {
346 3 return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG)) ? result
347 : result.mergedWith(typeMismatch(reader, writer, location));
348 }
349 case DOUBLE: {
350 4 return ((writer.getType() == Type.INT) || (writer.getType() == Type.LONG) || (writer.getType() == Type.FLOAT))
351 ? result
352 : result.mergedWith(typeMismatch(reader, writer, location));
353 }
354 case BYTES: {
355 2 return (writer.getType() == Type.STRING) ? result : result.mergedWith(typeMismatch(reader, writer, location));
356 }
357 case STRING: {
358 2 return (writer.getType() == Type.BYTES) ? result : result.mergedWith(typeMismatch(reader, writer, location));
359 }
360 }
361 case ARRAY:
362 1 return result.mergedWith(typeMismatch(reader, writer, location));
363 case MAP:
364 1 return result.mergedWith(typeMismatch(reader, writer, location));
365 case FIXED:
366 1 return result.mergedWith(typeMismatch(reader, writer, location));
367 case ENUM:
368 1 return result.mergedWith(typeMismatch(reader, writer, location));
369 case RECORD:
370 1 return result.mergedWith(typeMismatch(reader, writer, location));
371 case UNION: {
372     for (final Schema readerBranch : reader.getTypes()) {
373         SchemaCompatibilityResult compatibility = getCompatibility(readerBranch, writer);
374         if (compatibility.getCompatibility() == SchemaCompatibilityType.COMPATIBLE) {
375 1 return result;
376 }
377 }
378 }
379 // No branch in the reader union has been found compatible with the writer
380 // schema:
381 String message = String.format("reader union lacking writer type: %s", writer.getType());
382 1 return result.mergedWith(SchemaCompatibilityResult
383 .incompatible(SchemaIncompatibilityType.MISSING_UNION_BRANCH, reader, writer, message, asList(location)));
384 }
385 default: {
386     throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
387 }
388 }
389 }
390 }
391 }

```

Figura 4.48: Risultati pitest calculateCompatibility (dopo)

```

311 1 i++;
312 }
313 // Each schema in the writer union can be decoded with the reader:
314 1 return result;
315 }
316
317 default: {
318     throw new AvroRuntimeException("Unknown schema type: " + reader.getType());
319 }
320 }
321
322 } else {
323 // Reader and writer have different schema types:
324
325 // Reader compatible with all branches of a writer union is compatible
326 1 if (writer.getType() == Schema.Type.UNION) {
327     int index = 0;
328     for (Schema s : writer.getTypes()) {
329         result = result.mergedWith(getCompatibility(Integer.toString(index), reader, s, location));
330 1 index++;

```